

Computational Methods in Business and Finance

Course Notes for CS 335

Fall 2026

Instructor: Professor Leili Rafiee Sevyeri

David Cheriton R. School of Computer Science

University of Waterloo

©Peter Forsyth 2015¹

These notes have partially been funded by



¹David R. Cheriton School of Computer Science, University of Waterloo, Waterloo, Ontario, Canada, N2L 3G1, paforsyt@uwaterloo.ca, www.scicom.uwaterloo.ca/ paforsyt, tel: (519) 888-4567x34415, fax: (519) 885-1208

Contents

1	Pricing Options: Black-Scholes Model	7
1.1	The First Options Trade	7
1.2	Background	7
1.3	Definitions	8
1.4	A Simple Example: The Two State Tree	8
1.5	A Hedging Strategy	9
1.6	Brownian Motion	10
1.7	Geometric Brownian motion with drift	14
1.7.1	Ito's Lemma	18
1.7.2	Some uses of Ito's Lemma	19
1.7.3	Some more uses of Ito's Lemma	20
1.7.4	Integration by Parts	22
1.8	The Black-Scholes Analysis	22
1.9	Hedging in Continuous Time	23
1.10	The option price	24
1.11	American early exercise	24
2	More on Ito's Lemma	27
3	The Binomial Model	31
3.1	A Binomial Model Based on the Risk-Neutral Walk	31
3.2	A No-arbitrage Lattice	33
3.3	A Drifting Lattice	35
3.3.1	Numerical Comparison: No-arbitrage Lattice and Drifting Lattice	37
3.4	Smoothing the Payoff	39
3.4.1	Richardson extrapolation	42
3.5	MATLAB and Python Implementations	42
3.6	Dynamic Programming	46
4	Introduction to Computational Mathematics	49
4.1	Sources of Error	50
4.2	Floating Point Numbers and Operations	53
4.2.1	A Binary Computer	53
4.2.2	Standard floating point systems	54
4.2.3	Machine Precision	55
4.2.4	Floating Point Operations	56
4.3	Conditioning of a Mathematical Problem	57
4.4	Stability of a Numerical Algorithm	62

5	Root Finding	65
5.1	Introduction	65
5.2	Four Algorithms for Root Finding	66
5.2.1	Bisection Method	66
5.2.2	Fixed Point Iteration	67
5.2.3	Newton's Method	68
5.2.4	Secant Method	69
5.2.5	Stopping Criteria for Iterative Functions	69
5.3	Rate of Convergence	70
5.4	Convergence Theory	71
5.4.1	Fixed Point Iteration	71
5.4.2	Newton's Method	79
5.4.3	Secant Method	79
5.4.4	Overview	79
6	The Risk Neutral World	81
7	Monte Carlo Methods	85
7.1	Monte Carlo Error Estimators	87
7.2	Random Numbers and Monte Carlo	87
7.3	The Box-Muller Algorithm	89
7.4	An improved Box Muller	90
7.5	Speeding up Monte Carlo	92
7.6	Estimating the mean and variance	93
7.7	Low Discrepancy Sequences	94
7.8	Correlated Random Numbers	95
7.9	Integration of Stochastic Differential Equations	97
7.9.1	The Brownian Bridge	97
7.9.2	Strong and Weak Convergence	99
7.10	MATLAB and Monte Carlo Simulation	101
8	Discrete Hedging	107
8.1	Delta Hedging	107
8.2	Gamma Hedging	108
8.3	Vega Hedging	111
8.4	A Stop-Loss Strategy	112
8.5	Profit and Loss: probability density, VAR and CVAR	113
9	Some Investing Facts	115
9.1	Stocks for the Long Run?	115
9.1.1	GBM is Risky	116
9.2	Volatility Pumping	119
9.2.1	Constant Proportions Strategy	120
9.2.2	Leveraged Two Times Bull/Bear ETFs	122
9.3	More on Volatility Pumping	124
9.4	Constant Proportion Portfolio Insurance	125
9.4.1	Covered Call Writing	126
9.4.2	Stop Loss, Start Gain	128

10 Jump Diffusion	131
10.1 The Poisson Process	132
10.2 The Jump Diffusion Pricing Equation	133
10.3 Alternate Derivation of the Jump Diffusion Pricing Equation	135
10.4 Simulating Jump Diffusion	138
10.4.1 Compensated Drift	139
10.4.2 Contingent Claims Pricing	140
10.5 MATLAB Code: Jump Diffusion	140
11 Regime Switching	143
11.1 Mean Variance Portfolio Optimization	145
11.2 Special Cases	146
11.3 The Portfolio Allocation Problem	147
11.4 Adding a Risk-free asset	149
11.5 Criticism	151
11.6 Individual Securities	152
12 Derivative Contracts on non-traded Assets and Real Options	157
12.1 Derivative Contracts	157
12.2 A Forward Contract	160
12.2.1 Convenience Yield	161
12.2.2 Volatility of Forward Prices	162
13 Further Reading	163
13.1 General Interest	163
13.2 More Background	163
13.3 More Technical	163

Chapter 1

Pricing Options: Black-Scholes Model

1.1 The First Options Trade

“Men wanted for hazardous journey, small wages, bitter cold, long months of complete darkness, constant dangers, safe return doubtful. Honour and recognition in case of success.” Advertisement placed by Earnest Shackleton in 1914. He received 5000 replies. An example of extreme risk-seeking behaviour. Hedging with options is used to mitigate risk, and would not appeal to members of Shackleton’s expedition.

Many people think that options and futures are recent inventions. However, options have a long history, going back to ancient Greece.

As recorded by Aristotle in *Politics*, the fifth century BC philosopher Thales of Miletus took part in a sophisticated trading strategy. The main point of this trade was to confirm that philosophers could become rich if they so chose. This is perhaps the first rejoinder to the famous question *“If you are so smart, why aren’t you rich?”* which has dogged academics throughout the ages.

Thales observed that the weather was very favourable to a good olive crop, which would result in a bumper harvest of olives. If there had been an established Athens Board of Olives Exchange, Thales could have simply sold olive futures short (a surplus of olives would cause the price of olives to go down). Since the exchange did not exist, Thales put a deposit on all the olive presses surrounding Miletus. When the olive crop was harvested, demand for olive presses reached enormous proportions (olives were not a storable commodity). Thales then sublet the presses for a profit. Note that by placing a deposit on the presses, Thales was actually manufacturing an option on the olive crop, i.e., the most he could lose was his deposit. If he had sold short olive futures, he would have been liable to an unlimited loss, in the event that the olive crop turned out bad, and the price of olives went up. In other words, he had an option on a future of a non-storable commodity.

1.2 Background

Over the past few years derivative securities (options, futures, and forward contracts) have become essential tools for corporations and investors alike. Derivatives facilitate the transfer of financial risks. As such, they may be used to hedge risk exposures or to assume risks in the anticipation of profits. To take a simple yet instructive example, a gold mining firm is exposed to fluctuations in the price of gold. The firm could use a forward contract to fix the price of its future sales. This would protect the firm against a fall in the price of gold, but it would also sacrifice the upside potential from a gold price increase. This could be upside preserved by using options instead of a forward contract.

Individual investors can also use derivatives as part of their investment strategies. This can be done through direct trading on financial exchanges. In addition, it is quite common for financial products to include some form of embedded derivative. Any insurance contract can be viewed as a *put* option. Consequently, any

investment which provides some kind of protection actually includes an option feature. Standard examples include deposit insurance guarantees on savings accounts as well as the provision of being able to redeem a savings bond at par at any time. These types of embedded options are becoming increasingly common and increasingly complex. A prominent current example are investment guarantees being offered by insurance companies (“segregated funds”) and mutual funds. In such contracts, the initial investment is guaranteed, and gains can be locked-in (reset) a fixed number of times per year at the option of the contract holder. This is actually a very complex put option, known as a shout option. How much should an investor be willing to pay for this insurance? Determining the fair market value of these sorts of contracts is a problem in *option pricing*.

1.3 Definitions

Let’s consider some simple European put/call options. At some time T in the future (the expiry or exercise date) the holder has the right, but not the obligation, to

- Buy an asset at a prescribed price K (the exercise or strike price). This is a call option.
- Sell the asset at a prescribed price K (the exercise or strike price). This is a put option.

At expiry time T , we know with certainty what the value of the option is, in terms of the price of the underlying asset S ,

$$\begin{aligned}\text{Payoff} &= \max(S - K, 0) \text{ for a call} \\ \text{Payoff} &= \max(K - S, 0) \text{ for a put}\end{aligned}\tag{1.3.1}$$

Note that the payoff from an option is always non-negative, since the holder has a right but not an obligation. This contrasts with a forward contract, where the holder *must* buy or sell at a prescribed price.

1.4 A Simple Example: The Two State Tree

This example is taken from *Options, futures, and other derivatives*, by John Hull. Suppose the value of a stock is currently \$20. It is known that at the end of three months, the stock price will be either \$22 or \$18. We assume that the stock pays no dividends, and we would like to value a European call option to buy the stock in three months for \$21. This option can have only two possible values in three months: if the stock price is \$22, the option is worth \$1, if the stock price is \$18, the option is worth zero. This is illustrated in Figure 1.4.1.

In order to price this option, we can set up an imaginary portfolio consisting of the option and the stock, in such a way that there is no uncertainty about the value of the portfolio at the end of three months. Since the portfolio has no risk, the return earned by this portfolio must be the risk-free rate.

Consider a portfolio consisting of a long (positive) position of δ shares of stock, and short (negative) one call option. We will compute δ so that the portfolio is riskless. If the stock moves up to \$22 or goes down to \$18, then the value of the portfolio is

$$\begin{aligned}\text{Value if stock goes up} &= \$22\delta - 1 \\ \text{Value if stock goes down} &= \$18\delta - 0\end{aligned}\tag{1.4.1}$$

We can equate these expressions to find $\delta = .25$, which results in the value of the portfolio satisfying

$$\begin{aligned}\text{Value if stock goes up} &= \$22\delta - 1 = \$4.50 \\ \text{Value if stock goes down} &= \$18\delta - 0 = \$4.50\end{aligned}\tag{1.4.2}$$

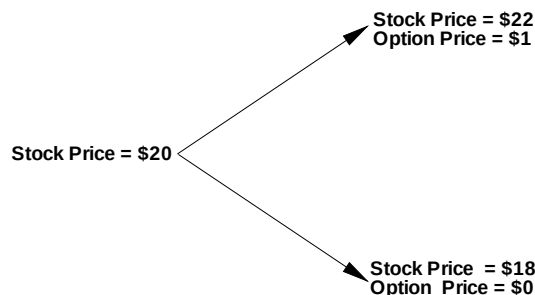


Figure 1.4.1: A simple case where the stock value can either be \$22 or \$18, with a European call option, $K = \$21$.

So, regardless of whether the stock moves up or down, the value of the portfolio is \$4.50. A risk-free portfolio must earn the risk-free rate. Suppose the current risk-free rate is 12%, then the value of the portfolio *today* must be the present value of \$4.50, or

$$4.50 \times e^{-.12 \times .25} = 4.367$$

The value of the stock today is \$20. Let the value of the option be V . The value of the portfolio is

$$\begin{aligned} 20 \times .25 - V &= 4.367 \\ \rightarrow V &= .633 \end{aligned}$$

1.5 A Hedging Strategy

So, if we sell the above option (we hold a short position in the option), then we can hedge this position in the following way. Today, we sell the option for \$.633, borrow \$4.367 from the bank at the risk free rate (this means that we have to pay the bank back \$4.50 in three months), which gives us \$5.00 in cash. Then, we buy .25 shares at \$20.00 (the current price of the stock). In three months time, one of two things happens

- The stock goes up to \$22, and our stock holding is now worth \$5.50. We pay the option holder \$1.00, which leaves us with \$4.50, just enough to pay off the bank loan.
- The stock goes down to \$18.00, so the call option is worthless. The value of the stock holding is now \$4.50, which is just enough to pay off the bank loan.

Consequently, in this simple situation, we see that the *theoretical* price of the option is the cost for the seller to set up the portfolio, which will precisely pay off the option holder and any bank loans required to set up the hedge, at the expiry of the option. In other words, this is the price which a hedger requires to ensure that there is always just enough money at the end to net out at zero gain or loss. If the market price of the option was higher than this value, the seller could sell at the higher price and lock in an instantaneous risk-free gain. Alternatively, if the market price of the option was lower than the theoretical, or *fair market* value, it would be possible to lock in a risk-free gain by selling the portfolio short. Any such arbitrage opportunities are rapidly exploited in the market, so that for most investors, we can assume that such opportunities are

not possible (the *no arbitrage condition*), and therefore that the market price of the option should be the theoretical price.

Note that this hedge works regardless of whether or not the stock goes up or down. Once we set up this hedge, we don't have a care in the world. The value of the option is also independent of the probability that the stock goes up to \$22 or down to \$18. This is somewhat counterintuitive.

1.6 Brownian Motion

Before we consider a model for stock price movements, let's consider the idea of Brownian motion with drift. Suppose X is a random variable, and in time $t \rightarrow t + dt$, $X \rightarrow X + dX$, where

$$dX = \alpha dt + \sigma dZ \quad (1.6.1)$$

where αdt is the drift term, σ is the volatility, and dZ is a random term. The dZ term has the form

$$dZ = \phi \sqrt{dt} \quad (1.6.2)$$

where ϕ is a random variable drawn from a normal distribution with mean zero and variance one ($\phi \sim N(0, 1)$, i.e., ϕ is normally distributed).

If E is the expectation operator, then

$$E(\phi) = 0 \quad E(\phi^2) = 1 . \quad (1.6.3)$$

Now in a time interval dt , we have

$$\begin{aligned} E(dX) &= E(\alpha dt) + E(\sigma dZ) \\ &= \alpha dt + \sigma E[\phi \sqrt{dt}] \\ &= \alpha dt , \end{aligned} \quad (1.6.4)$$

and the variance of dX , denoted by $Var(dX)$ is

$$\begin{aligned} Var(dX) &= E([dX - E(dX)]^2) \\ &= E([\alpha dt + \sigma dZ - \alpha dt]^2) \\ &= E([\sigma dZ]^2) \\ &= E([\sigma \phi \sqrt{dt}]^2) \\ &= \sigma^2 dt E(\phi^2) \\ &= \sigma^2 dt . \end{aligned} \quad (1.6.5)$$

Let's look at a discrete model to understand the process of this Brownian motion more completely. To do so we will consider a discrete lattice of points. Let $X = X_0$ at $t = 0$. Suppose that at $t = \Delta t$,

$$\begin{aligned} X_0 &\rightarrow X_0 + \Delta h \quad ; \quad \text{with probability } p \\ X_0 &\rightarrow X_0 - \Delta h \quad ; \quad \text{with probability } q \end{aligned} \quad (1.6.6)$$

where $p + q = 1$. Assume that:

- X follows a Markov process, i.e., the probability distribution in the future depends only on where it is now.
- The probability of an up or down move is independent of what happened in the past.
- X can move only up or down by Δh .

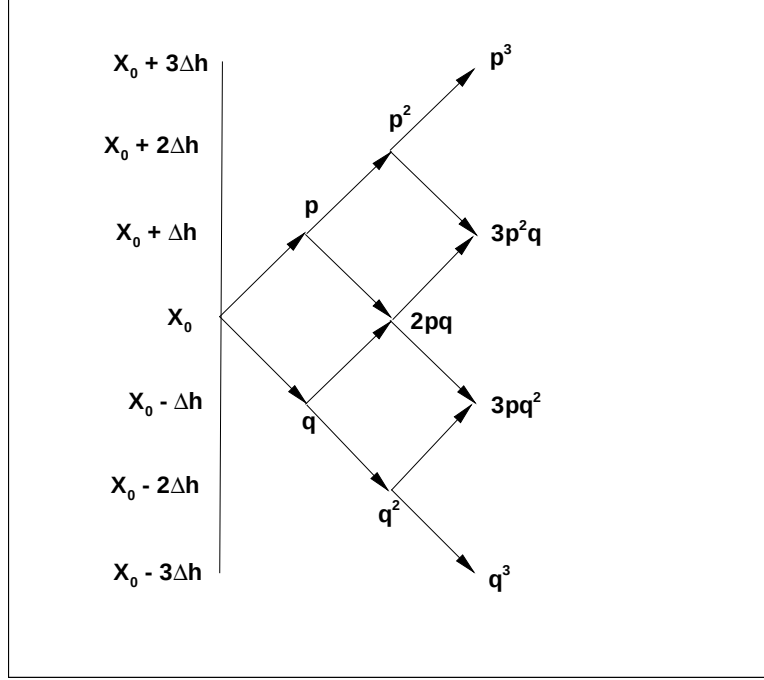


Figure 1.6.1: Probabilities of reaching the discrete lattice points for the first three moves.

At any lattice point $X_0 + i\Delta h$, the probability of an up move is p , and the probability of a down move is q . The probabilities of reaching any particular lattice point (value of X) for the first three moves are shown in Figure 1.6.1. Each move takes place over the time interval $t \rightarrow t + \Delta t$.

Let ΔX be the change in X over the interval $t \rightarrow t + \Delta t$. Then

$$\begin{aligned}
 E(\Delta X) &= p(\Delta h) + q(-\Delta h) \\
 E(\Delta X) &= (p - q)\Delta h \\
 E([\Delta X]^2) &= p(\Delta h)^2 + q(-\Delta h)^2 \\
 &= (\Delta h)^2,
 \end{aligned} \tag{1.6.7}$$

so that the variance of ΔX is (over $t \rightarrow t + \Delta t$)

$$\begin{aligned}
 Var(\Delta X) &= E([\Delta X]^2) - [E(\Delta X)]^2 \\
 &= (\Delta h)^2 - (p - q)^2(\Delta h)^2 \\
 &= (1 - (p - q)^2)(\Delta h)^2 \\
 &= (1 - (p - (1 - p))^2)(\Delta h)^2 \\
 &= 4p(1 - p)(\Delta h)^2 \\
 &= 4pq(\Delta h)^2.
 \end{aligned} \tag{1.6.8}$$

where we used the fact that $q = 1 - p$. Now, suppose we consider the distribution of X after n moves, so that $t = n\Delta t$. The probability of j up moves, and $(n - j)$ down moves, denoted $P(n, j)$, is

$$P(n, j) = \frac{n!}{j!(n - j)!} p^j q^{n-j} \tag{1.6.9}$$

which is just a binomial distribution. Now, if X_n is the value of X after n steps on the lattice, then

$$\begin{aligned}
 E(X_n - X_0) &= nE(\Delta X) \\
 Var(X_n - X_0) &= nVar(\Delta X),
 \end{aligned} \tag{1.6.10}$$

which follows from the properties of a binomial distribution, (each up or down move is independent of previous moves). Consequently, from equations (1.6.7, 1.6.8, 1.6.10) we obtain

$$\begin{aligned}
E(X_n - X_0) &= n(p - q)\Delta h \\
&= \frac{t}{\Delta t}(p - q)\Delta h \\
Var(X_n - X_0) &= n4pq(\Delta h)^2 \\
&= \frac{t}{\Delta t}4pq(\Delta h)^2
\end{aligned} \tag{1.6.11}$$

Now, we would like to take the limit as $\Delta t \rightarrow 0$ in such a way that the mean and variance of X , after a finite time t , is independent of Δt , and we would like to recover

$$\begin{aligned}
dX &= \alpha dt + \sigma dZ \\
E(dX) &= \alpha dt \\
Var(dX) &= \sigma^2 dt
\end{aligned} \tag{1.6.12}$$

as $\Delta t \rightarrow 0$. Now, since $0 \leq p, q \leq 1$, we need to choose $\Delta h = Const \sqrt{\Delta t}$. Otherwise, from equation (1.6.11) we get that $Var(X_n - X_0)$ is either 0 or infinite after a finite time. (Stock variances do not have either of these properties, so this is obviously not a very interesting case).

Let's choose $\Delta h = \sigma\sqrt{\Delta t}$, which gives (from equation (1.6.11))

$$\begin{aligned}
E(X_n - X_0) &= (p - q)\frac{\sigma t}{\sqrt{\Delta t}} \\
Var(X_n - X_0) &= t4pq\sigma^2
\end{aligned} \tag{1.6.13}$$

Now, for $E(X_n - X_0)$ to be independent of Δt as $\Delta t \rightarrow 0$, we must have

$$(p - q) = Const \sqrt{\Delta t} \tag{1.6.14}$$

If we choose

$$p - q = \frac{\alpha}{\sigma}\sqrt{\Delta t} \tag{1.6.15}$$

we get

$$\begin{aligned}
p &= \frac{1}{2} \left[1 + \frac{\alpha}{\sigma}\sqrt{\Delta t} \right] \\
q &= \frac{1}{2} \left[1 - \frac{\alpha}{\sigma}\sqrt{\Delta t} \right]
\end{aligned} \tag{1.6.16}$$

Now, putting together equations (1.6.13-1.6.16) gives

$$\begin{aligned}
E(X_n - X_0) &= \alpha t \\
Var(X_n - X_0) &= t\sigma^2 \left(1 - \frac{\alpha^2}{\sigma^2} \Delta t \right) \\
&= t\sigma^2 ; \Delta t \rightarrow 0 .
\end{aligned} \tag{1.6.17}$$

Now, let's imagine that $X(t_n) - X(t_0) = X_n - X_0$ is very small, so that $X_n - X_0 \simeq dX$ and $t_n - t_0 \simeq dt$, so that equation (1.6.17) becomes

$$\begin{aligned}
E(dX) &= \alpha dt \\
Var(dX) &= \sigma^2 dt .
\end{aligned} \tag{1.6.18}$$

which agrees with the equations (1.6.4-1.6.5) associated with Brownian motion. Hence, in the limit as $\Delta t \rightarrow 0$, we can interpret the random walk for X on the discrete lattice (with these parameters) as the solution to the stochastic differential equation (SDE)

$$\begin{aligned} dX &= \alpha dt + \sigma dZ \\ dZ &= \phi \sqrt{dt}. \end{aligned} \quad (1.6.19)$$

Consider the case where $\alpha = 0, \sigma = 1$, so that $dX = dZ \simeq Z(t_i) - Z(t_{i-1}) = Z_i - Z_{i-1} = X_i - X_{i-1}$. Now we can write

$$\int_0^t dZ = \lim_{\Delta t \rightarrow 0} \sum_i (Z_{i+1} - Z_i) = (Z_n - Z_0). \quad (1.6.20)$$

From equation (1.6.17) using $(\alpha = 0, \sigma = 1)$ we have

$$\begin{aligned} E(Z_n - Z_0) &= 0 \\ \text{Var}(Z_n - Z_0) &= t. \end{aligned} \quad (1.6.21)$$

Now, if n is large ($\Delta t \rightarrow 0$), recall that the binomial distribution (1.6.9) tends to a normal distribution. From equation (1.6.21), we have that the mean of this distribution is zero, with variance t , so that

$$\begin{aligned} (Z_n - Z_0) &\sim N(0, t) \\ &= \int_0^t dZ. \end{aligned} \quad (1.6.22)$$

In other words, after a finite time t , $\int_0^t dZ$ is normally distributed with mean zero and variance t (the limit of a binomial distribution is a normal distribution).

Recall that, in this case, we have that $Z_i - Z_{i-1} = \Delta h = \sqrt{\Delta t}$ with probability p and $Z_i - Z_{i-1} = -\sqrt{\Delta t}$ with probability q . Note that $(Z_i - Z_{i-1})^2 = \Delta t$, with certainty, so that we can write

$$(Z_i - Z_{i-1})^2 \simeq (dZ)^2 = \Delta t. \quad (1.6.23)$$

To summarize

- We can interpret the SDE

$$\begin{aligned} dX &= \alpha dt + \sigma dZ \\ dZ &= \phi \sqrt{dt}. \end{aligned} \quad (1.6.24)$$

as the limit of a discrete random walk on a lattice as the timestep tends to zero.

- $\text{Var}(dZ) = dt$; otherwise, after any finite time, the $\text{Var}(X_n - X_0)$ is either zero or infinite.
- We can *integrate* the random term dZ to obtain

$$\begin{aligned} \int_0^t dZ &= Z(t) - Z(0) \\ &\sim N(0, t). \end{aligned} \quad (1.6.25)$$

Going back to our lattice example, we can notice that the total distance traveled over any finite interval of time becomes infinite. Observe that

$$E(|\Delta X|) = \Delta h \quad (1.6.26)$$

T	1.0
σ	.2
α	.10
X_{init}	0
Number of simulations	50000
Number of timesteps	400

Table 1.6.1: *Data used in simulation of discrete walk on a lattice.*

Variable	Mean	Standard Deviation
$X(T)$	0.10093	0.20035
$e^{X(T)}$	.22813	1.1286

Table 1.6.2: *Test results: discrete lattice walk, data in Table 1.6.1.*

so that the the total distance traveled in n steps is

$$\begin{aligned} n\Delta h &= \frac{t}{\Delta t} \Delta h \\ &= \frac{t\sigma}{\sqrt{\Delta t}} \end{aligned} \quad (1.6.27)$$

which goes to infinity as $\Delta t \rightarrow 0$. Similarly,

$$\frac{\Delta X}{\Delta t} = \pm\infty. \quad (1.6.28)$$

Consequently, Brownian motion is very jagged at every timescale. These paths are not differentiable, i.e., $\frac{dx}{dt}$ does not exist, so we cannot speak of

$$E\left(\frac{dX}{dt}\right) \quad (1.6.29)$$

but we can possibly define

$$\frac{E(dX)}{dt}. \quad (1.6.30)$$

We can verify that taking the limit as $\Delta t \rightarrow 0$ on the discrete lattice converges to the normal density. Consider the data in Table 1.6.1. The random walk on the lattice was simulated using a Monte Carlo approach. Starting at X_0 , the particle was moved up with probability p and down with probability $(1 - p)$, in consistent with (1.6.16). A random number was used to determine the actual move. At the next node, this was repeated, until we obtain the position of X after n steps, X_n . This is repeated many times. We can then determine the mean and variance of these outcomes (see Table 1.6.2). The mean and variance of e^X have also been included, since this is relevant for the case of Geometric Brownian Motion, which will be studied in the next Section. A histogram of the outcomes is shown in Figure 1.6.2.

The MATLAB m-file used to generate the walk on the lattice is given in Algorithm 1.7.3. Equivalent Python/Numpy code is given in Algorithm 1.7.4.

1.7 Geometric Brownian motion with drift

Of course, the actual path followed by stock is more complex than the simple situation described above. More realistically, we assume that the relative changes in stock prices (the returns) follow Brownian motion

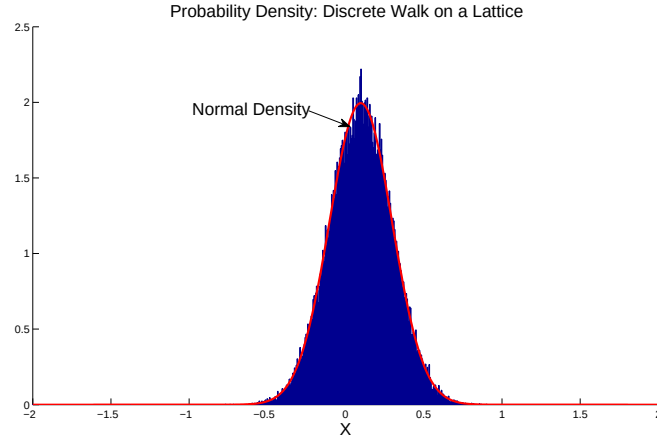


Figure 1.6.2: Normalized histogram of discrete lattice walk simulations. Normal density with mean 0.1, standard deviation 0.2 also shown.

with drift. We suppose that in an infinitesimal time dt , the stock price S changes to $S + dS$, where

$$\frac{dS}{S} = \mu dt + \sigma dZ \quad (1.7.1)$$

where μ is the drift rate, σ is the volatility, and dZ is the increment of a Wiener process,

$$dZ = \phi \sqrt{dt} \quad (1.7.2)$$

where $\phi \sim N(0,1)$. Equations (1.7.1) and (1.7.2) are called geometric Brownian motion with drift. So, superimposed on the upward (relative) drift is a (relative) random walk. The degree of randomness is given by the volatility σ . Figure 1.7.1 gives an illustration of ten realizations of this random process for two different values of the volatility. In this case, we assume that the drift rate μ equals the risk free rate.

Vectorized M file For Lattice Walk

```
function [X_new] = walk_sim( N_sim,N,...
    mu, T, sigma, X_init)

%
% N_sim    number of simulations
% N        number of timesteps
% X_init    initial value
% T        expiry time
% sigma     volatility
% mu        drift
%
% lattice factors
%
%
delt = T/N; % timestep size
up = sigma*sqrt(delt);
down = - sigma*sqrt(delt);

p = 1./2.*( 1. + mu/sigma*sqrt( delt ) );

X_new = zeros(N_sim,1);
X_new(1:N_sim,1) = X_init;

ptest = zeros(N_sim, 1);

for i=1:N % timestep loop
    % now, for each timestep, generate info for
    % all simulations

    ptest(:,1) = rand(N_sim,1);
    ptest(:,1) = (ptest(:,1) <= p); % = 1 if up move
                                     % = 0 if down move
    X_new(:,1) = X_new(:,1) + ptest(:,1)*up + (1.-ptest(:,1))*down;

    % end of generation of all data for all simulations
    % for this timestep
end % timestep loop
```

(1.7.3)

Vectorized Python/Numpy code For Lattice Walk

```
def walk_sim(N_sim, N, mu, T, sigma, X_init):
    # N_sim - number of simulations
    # N - number of timesteps
    # mu - drift
    # T - expiry time
    # sigma - volatility
    # X_init - initial value

    import numpy as np
    import math

    delt = T/N
    up = sigma*math.sqrt(delt)
    down = -sigma*math.sqrt(delt)

    p = 1./2.*(1. + mu/sigma*math.sqrt(delt))

    X_new = np.ones(N_sim)*X_init

    ptest = np.zeros(N_sim)

    for i in np.arange(N): #timestep loop
        # now for each timestep, generate info
        # for all simulations

        ptest = np.random.uniform(0,1,N_sim)
        ptest = (ptest <= p)
        X_new = X_new + ptest*up + (1-ptest)*down
        # note: asterisks here perform *elementwise*
        # multiplication on numpy vectors, not
        # a dot product.

    # end of generation of data for all simulations
    # for this timestep
    return X_new
```

(1.7.4)

Note that

$$\begin{aligned}
 E(dS) &= E(\sigma S dZ + \mu S dt) \\
 &= \mu S dt \\
 &\quad \text{since } E(dZ) = 0
 \end{aligned}
 \tag{1.7.5}$$

and that the variance of dS is

$$\begin{aligned}
 Var[dS] &= E(dS^2) - [E(dS)]^2 \\
 &= E(\sigma^2 S^2 dZ^2) \\
 &= \sigma^2 S^2 dt
 \end{aligned}
 \tag{1.7.6}$$

so that σ is a measure of the degree of randomness of the stock price movement.

Equation (1.7.1) is a *stochastic differential equation*. The normal rules of calculus don't apply, since for

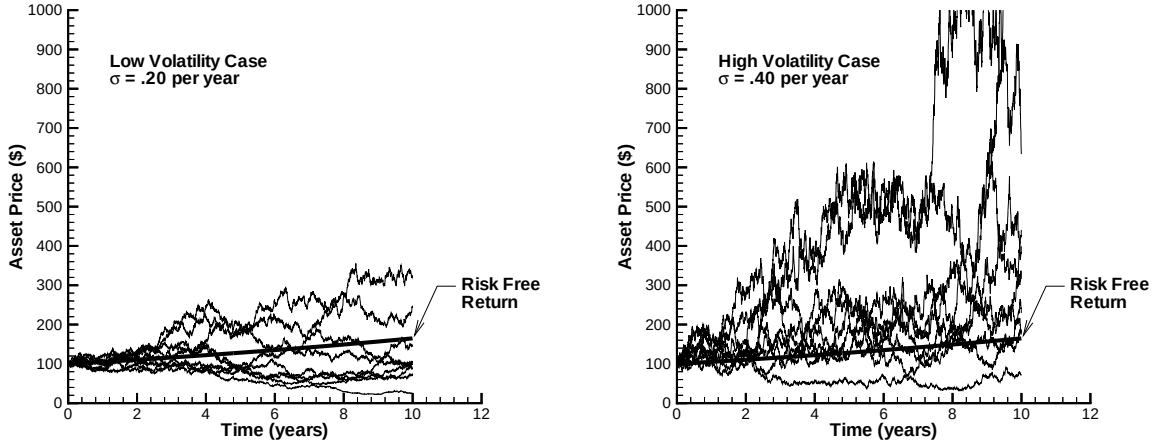


Figure 1.7.1: Realizations of asset price following geometric Brownian motion. Left: low volatility case; right: high volatility case. Risk-free rate of return $r = .05$.

example

$$\frac{dZ}{dt} = \phi \frac{1}{\sqrt{dt}} \\ \rightarrow \infty \text{ as } dt \rightarrow 0 \text{ .}$$

The study of these sorts of equations uses results from stochastic calculus. However, for our purposes, we need only one result, which is Ito's Lemma (see *Derivatives: the theory and practice of financial engineering*, by P. Wilmott). Suppose we have some function $G = G(S, t)$, where S follows the stochastic process equation (1.7.1), then, in small time increment dt , $G \rightarrow G + dG$, where

$$dG = \left(\mu S \frac{\partial G}{\partial S} + \frac{\sigma^2 S^2}{2} \frac{\partial^2 G}{\partial S^2} + \frac{\partial G}{\partial t} \right) dt + \sigma S \frac{\partial G}{\partial S} dZ \quad (1.7.7)$$

An informal derivation of this result is given in the following section.

1.7.1 Ito's Lemma

We give an informal derivation of Ito's lemma (1.7.7). Suppose we have a variable S which follows

$$dS = a(S, t)dt + b(S, t)dZ \quad (1.7.8)$$

where dZ is the increment of a Weiner process.

Now since

$$dZ^2 = \phi^2 dt \quad (1.7.9)$$

where ϕ is a random variable drawn from a normal distribution with mean zero and unit variance, we have that, if E is the expectation operator, then

$$E(\phi) = 0 \quad E(\phi^2) = 1 \quad (1.7.10)$$

so that the expected value of dZ^2 is

$$E(dZ^2) = dt \quad (1.7.11)$$

Now, it can be shown (see Section 2) that in the limit as $dt \rightarrow 0$, we have that $\phi^2 dt$ becomes non-stochastic, so that with probability one

$$dZ^2 \rightarrow dt \quad \text{as } dt \rightarrow 0 \quad (1.7.12)$$

Now, supposing we have some function $G = G(S, t)$, then a Taylor series implies

$$dG = G_S dS + G_t dt + G_{SS} \frac{dS^2}{2} + \dots \quad (1.7.13)$$

where for brevity, we've used subscripts to indicate partial derivatives. Now (from (1.7.8))

$$\begin{aligned} (dS)^2 &= (a dt + b dZ)^2 \\ &= a^2 dt^2 + 2ab dZ dt + b^2 dZ^2 \end{aligned} \quad (1.7.14)$$

Since $dZ = O(\sqrt{dt})$ and $dZ^2 \rightarrow dt$, equation (1.7.14) becomes

$$(dS)^2 = b^2 dZ^2 + O((dt)^{3/2}) \quad (1.7.15)$$

or

$$(dS)^2 \rightarrow b^2 dt \quad \text{as } dt \rightarrow 0 \quad (1.7.16)$$

Now, equations (1.7.8, 1.7.13, 1.7.16) give

$$\begin{aligned} dG &= G_S dS + G_t dt + G_{SS} \frac{dS^2}{2} + \dots \\ &= G_S (a dt + b dZ) + dt (G_t + G_{SS} \frac{b^2}{2}) \\ &= G_S b dZ + (a G_S + G_{SS} \frac{b^2}{2} + G_t) dt \end{aligned} \quad (1.7.17)$$

So, we have the final result that if

$$dS = a(S, t) dt + b(S, t) dZ \quad (1.7.18)$$

and if $G = G(S, t)$, then

$$dG = G_S b dZ + \left(a G_S + G_{SS} \frac{b^2}{2} + G_t \right) dt \quad (1.7.19)$$

Equation (1.7.7) can be deduced by setting $a = \mu S$ and $b = \sigma S$ in equation (1.7.19).

1.7.2 Some uses of Ito's Lemma

In certain cases, Ito's Lemma can be used to solve an SDE. Suppose we have

$$dS = \mu dt + \sigma dZ \quad (1.7.20)$$

If $\mu, \sigma = \text{Const}$, then this can be integrated (from $t = 0$ to $t = t$) exactly to give

$$S(t) = S(0) + \mu t + \sigma(Z(t) - Z(0)) \quad (1.7.21)$$

and from equation (1.6.25)

$$Z(t) - Z(0) \sim N(0, t) \quad (1.7.22)$$

Note that when we say that we solve a stochastic differential equation exactly, this means that we have an expression for the *distribution* of $S(T)$.

Suppose instead we have the more usual geometric Brownian motion

$$dS = \mu S dt + \sigma S dZ \quad (1.7.23)$$

Letting $F(S) = \log S$, we apply Ito's Lemma to find

$$\begin{aligned} dF &= F_S S \sigma dZ + \left(F_S \mu S + F_{SS} \frac{\sigma^2 S^2}{2} + F_t \right) dt \\ &= \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dZ, \end{aligned} \quad (1.7.24)$$

where we have used that $F_S = 1/S$, $F_{SS} = -1/S^2$, and $F_t = 0$. We can integrate this to get

$$F(t) = F(0) + \left(\mu - \frac{\sigma^2}{2} \right) t + \sigma (Z(t) - Z(0)) \quad (1.7.25)$$

or, since $S = e^F$,

$$S(t) = S(0) \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma (Z(t) - Z(0)) \right]. \quad (1.7.26)$$

Unfortunately, these cases (i.e., constant σ, μ) are about the only situations where we can exactly integrate the SDE.

1.7.3 Some more uses of Ito's Lemma

We can often use Ito's Lemma and some algebraic tricks to determine some properties of distributions. Let

$$dX = a(X, t) dt + b(X, t) dZ; \quad (1.7.27)$$

then if $G = G(X)$, we have

$$dG = \left[a G_X + G_t + \frac{b^2}{2} G_{XX} \right] dt + G_X b dZ. \quad (1.7.28)$$

If $E[X] = \bar{X}$, then (since $b(X, t)$ and dZ are independent)

$$\begin{aligned} E[dX] &= d E[X] = d\bar{X} \\ &= E[a dt] + E[b] E[dZ] \\ &= E[a dt], \end{aligned} \quad (1.7.29)$$

so that

$$\begin{aligned} \frac{d\bar{X}}{dt} &= E[a] = \bar{a} \\ \bar{X} &= E \left[\int_0^t a dt \right]. \end{aligned} \quad (1.7.30)$$

Let $\bar{G} = E[(X - \bar{X})^2] = \text{Var}(X)$, then

$$\begin{aligned} d\bar{G} &= E[dG] \\ &= E[2(X - \bar{X})a - 2(X - \bar{X})\bar{a} + b^2] dt + E[2b(X - \bar{X})]E[dZ] \\ &= E[b^2 dt] + E[2(X - \bar{X})(a - \bar{a}) dt], \end{aligned} \quad (1.7.31)$$

which means that

$$\bar{G} = Var(X) = E \left[\int_0^t b^2 dt \right] + E \left[\int_0^t 2(a - \bar{a})(X - \bar{X}) dt \right] . \quad (1.7.32)$$

In a particular case, we can sometimes get more useful expressions. If

$$dS = \mu S dt + \sigma S dZ \quad (1.7.33)$$

with μ, σ constant, then

$$\begin{aligned} E[dS] &= d\bar{S} = E[\mu S] dt \\ &= \mu \bar{S} dt , \end{aligned} \quad (1.7.34)$$

so that

$$\begin{aligned} d\bar{S} &= \mu \bar{S} dt \\ \bar{S} &= S_0 e^{\mu t} . \end{aligned} \quad (1.7.35)$$

Now, let $G(S) = S^2$, so that $E[G] = \bar{G} = E[S^2]$, then (from Ito's Lemma)

$$\begin{aligned} d\bar{G} &= E[2\mu S^2 + \sigma^2 S^2] dt + E[2S^2 \sigma] E[dZ] \\ &= E[2\mu S^2 + \sigma^2 S^2] dt \\ &= (2\mu + \sigma^2) \bar{G} dt , \end{aligned} \quad (1.7.36)$$

so that

$$\begin{aligned} \bar{G} &= \bar{G}_0 e^{(2\mu + \sigma^2)t} \\ E[S^2] &= S_0^2 e^{(2\mu + \sigma^2)t} . \end{aligned} \quad (1.7.37)$$

From equations (1.7.35) and (1.7.37) we then have

$$\begin{aligned} Var(S) &= E[S^2] - (E[S])^2 \\ &= E[S^2] - \bar{S}^2 \\ &= S_0^2 e^{2\mu t} (e^{\sigma^2 t} - 1) \\ &= \bar{S}^2 (e^{\sigma^2 t} - 1) . \end{aligned} \quad (1.7.38)$$

One can use the same ideas to compute the skewness, $E[(S - \bar{S})^3]$. If $G(S) = S^3$ and $\bar{G} = E[G(S)] = E[S^3]$, then

$$\begin{aligned} d\bar{G} &= E[\mu S \cdot 3S^2 + \sigma^2 S^2 / 2 \cdot 3 \cdot 2S] dt + E[3S^2 \sigma S] E[dZ] \\ &= E[3\mu S^3 + 3\sigma^2 S^3] \\ &= 3(\mu + \sigma^2) \bar{G} , \end{aligned} \quad (1.7.39)$$

so that

$$\begin{aligned} \bar{G} &= E[S^3] \\ &= S_0^3 e^{3(\mu + \sigma^2)t} . \end{aligned} \quad (1.7.40)$$

We can then obtain the skewness from

$$\begin{aligned} E[(S - \bar{S})^3] &= E[S^3 - 2S^2 \bar{S} - 2S \bar{S}^2 + \bar{S}^3] \\ &= E[S^3] - 2\bar{S} E[S^2] - \bar{S}^3 . \end{aligned} \quad (1.7.41)$$

Equations (1.7.35, 1.7.37, 1.7.40) can then be substituted into equation (1.7.41) to get the desired result.

1.7.4 Integration by Parts

Let $X(t)$, $Y(t)$ be two stochastic variables, with $X(t_i) = X_i$ and $Y(t_i) = Y_i$, then

$$(X_{i+1} - X_i)(Y_{i+1} - Y_i) = X_{i+1}Y_{i+1} - X_iY_i - X_i(Y_{i+1} - Y_i) - Y_i(X_{i+1} - X_i) . \quad (1.7.42)$$

Hence

$$\sum_{i=1}^{i=N} (X_{i+1} - X_i)(Y_{i+1} - Y_i) = X_{N+1}Y_{N+1} - X_1Y_1 - \sum_{i=1}^{i=N} X_i(Y_{i+1} - Y_i) - \sum_{i=1}^{i=N} Y_i(X_{i+1} - X_i) . \quad (1.7.43)$$

Letting $\Delta t \rightarrow 0$, the sums in equation (1.7.43) become Ito stochastic integrals

$$\int_0^T dX(t')dY(t') = [XY]_0^T - \int_0^T X(t')dY(t') - \int_0^T Y(t')dX(t') , \quad (1.7.44)$$

which we can write as the Ito integration by parts rule

$$d(XY) = Y dX + X dY + dX dY . \quad (1.7.45)$$

Note the extra term $dX dY$ in equation (1.7.45) compared with the non-stochastic integration by parts rule.

1.8 The Black-Scholes Analysis

Assume:

- The stock price follows geometric Brownian motion, equation (1.7.1).
- The risk-free rate of return is a constant r .
- There are no arbitrage opportunities, i.e. all risk-free portfolios must earn the risk-free rate of return.
- Short selling is permitted (i.e., we can own negative quantities of an asset).

Suppose that we have an option whose value is given by $V = V(S, t)$. Construct an imaginary portfolio, consisting of one option, and a number of $(-\alpha^h)$ of the underlying asset. (If $(\alpha^h) > 0$, then we have sold the asset short, i.e., we have borrowed an asset, sold it, and are obligated to give it back at some future date).

The value of this portfolio P is

$$P = V - (\alpha^h)S \quad (1.8.1)$$

In a small time dt , $P \rightarrow P + dP$,

$$dP = dV - (\alpha^h)dS \quad (1.8.2)$$

Note that in equation (1.8.2) we have not included a term $(\alpha^h)_S S$. This is actually a rather subtle point, since we shall see (later on) that (α^h) actually depends on S . However, if we think of a real situation, at any instant in time, we must choose (α^h) , and then we hold the portfolio while the asset moves randomly. So, equation (1.8.2) is actually the change in the value of the portfolio, not a differential. If we were taking a true differential then equation (1.8.2) would be

$$dP = dV - (\alpha^h)dS - Sd(\alpha^h)$$

but we have to remember that (α^h) does not change over a small time interval, since we pick (α^h) , and then S changes randomly. We are not allowed to *peek into the future*, (otherwise, we could get rich without risk, which is not permitted by the no-arbitrage condition) and hence (α^h) is not allowed to contain any information about future asset price movements. The principle of *no peeking into the future* is why Ito

stochastic calculus is used. Other forms of stochastic calculus are used in Physics applications (i.e., turbulent flow).

Substituting equations (1.7.1) and (1.7.7) into equation (1.8.2) gives

$$dP = \sigma S (V_S - (\alpha^h)) dZ + \left(\mu S V_S + \frac{\sigma^2 S^2}{2} V_{SS} + V_t - \mu(\alpha^h) S \right) dt \quad (1.8.3)$$

We can make this portfolio riskless over the time interval dt , by choosing $(\alpha^h) = V_S$ in equation (1.8.3), since this eliminates the dZ term in equation (1.8.3). (This is the analogue of our choice of the amount of stock in the riskless portfolio for the two state tree model.) So, letting

$$(\alpha^h) = V_S \quad (1.8.4)$$

and then substituting equation (1.8.4) into equation (1.8.3) gives

$$dP = \left(V_t + \frac{\sigma^2 S^2}{2} V_{SS} \right) dt \quad (1.8.5)$$

Since P is now risk-free in the interval $t \rightarrow t + dt$, no-arbitrage says that

$$dP = rPdt \quad (1.8.6)$$

Therefore, equations (1.8.5) and (1.8.6) give

$$rPdt = \left(V_t + \frac{\sigma^2 S^2}{2} V_{SS} \right) dt \quad (1.8.7)$$

Since

$$P = V - (\alpha^h)S = V - V_S S \quad (1.8.8)$$

then substituting equation (1.8.8) into equation (1.8.7) gives

$$V_t + \frac{\sigma^2 S^2}{2} V_{SS} + rSV_S - rV = 0 \quad (1.8.9)$$

which is the Black-Scholes equation. Note the rather remarkable fact that equation (1.8.9) is independent of the drift rate μ .

Equation (1.8.9) is solved backwards in time from the option expiry time $t = T$ to the present $t = 0$.

1.9 Hedging in Continuous Time

We can construct a hedging strategy based on the solution to the above equation. Suppose we sell an option at price V at $t = 0$. Then we carry out the following:

- We sell one option worth V . (This gives us V in cash initially).
- We borrow $(S \frac{\partial V}{\partial S} - V)$ from the bank.
- We buy $\frac{\partial V}{\partial S}$ shares at price S .

At every instant in time, we adjust the amount of stock we own so that we always have $\frac{\partial V}{\partial S}$ shares. Note that this is a *dynamic* hedge, since we have to continually rebalance the portfolio. Cash will flow into and out of the bank account, in response to changes in S . If the amount in the bank is positive, we receive the risk-free rate of return. If negative, then we borrow at the risk-free rate.

So, our hedging portfolio will be:

- Short one option worth V .
- Long $\frac{\partial V}{\partial S}$ shares at price S .
- $V - S\frac{\partial V}{\partial S}$ cash in the bank account.

At any instant in time (including the terminal time), this portfolio can be liquidated and any obligations implied by the short position in the option can be covered, at *zero gain or loss*, regardless of the value of S . Note that given the receipt of the cash for the option, this strategy is *self-financing*.

1.10 The option price

So, we can see that the price of the option valued by the Black-Scholes equation is the market price of the option at any time. If the price was higher than the Black-Scholes price, we could construct the hedging portfolio, dynamically adjust the hedge, and end up with a positive amount at the end. Similarly, if the price was lower than the Black-Scholes price, we could short the hedging portfolio, and end up with a positive gain. By the *no-arbitrage* condition, this should not be possible.

Note that we are *not* trying to predict the price movements of the underlying asset, which is (assumed to be) a random process. The value of the option is based on a hedging strategy which is *dynamic*, and must be continuously rebalanced. The price is the cost of setting up the hedging portfolio. The Black-Scholes price is *not* the expected payoff.

The price given by the Black-Scholes price is *not* the value of the option to a speculator, who buys and holds the option. A speculator is making bets about the underlying drift rate of the stock (note that the drift rate does not appear in the Black-Scholes equation). For a speculator, the value of the option is given by an equation similar to the Black-Scholes equation, except that the drift rate appears. In this case, the price can be interpreted as the *expected payoff* based on the guess for the drift rate. But this is art, not science!

1.11 American early exercise

Actually, most options traded are *American* options, which have the feature that they can be exercised at any time (this contrasts with the European options introduced earlier, which can only be exercised at the expiry time T). Consequently, an investor acting optimally, will always exercise the option if the value falls below the payoff or exercise value. So, the value of an American option is given by the solution to equation (1.8.9) with the additional constraint

$$V(S, t) \geq \begin{cases} \max(S - K, 0) & \text{for a call;} \\ \max(K - S, 0) & \text{for a put.} \end{cases} \quad (1.11.1)$$

Note that since we are working backwards in time, we *know* what the option is worth in future, and therefore we can determine the optimal course of action.

In order to write equation (1.8.9) in more conventional form, define $\tau = T - t$, so that equation (1.8.9) becomes

$$\begin{aligned} V_\tau &= \frac{\sigma^2 S^2}{2} V_{SS} + rSV_S - rV \\ V(S, \tau = 0) &= \begin{cases} \max(S - K, 0) & \text{for a call} \\ \max(K - S, 0) & \text{for a put} \end{cases} \\ V(0, \tau) &\rightarrow V_\tau = -rV \\ V(S = \infty, \tau) &\rightarrow \begin{cases} \simeq S & \text{for a call} \\ \simeq 0 & \text{for a put} \end{cases} \end{aligned} \quad (1.11.2)$$

If the option is American, then we also have the additional constraints

$$V(S, \tau) \geq \begin{cases} \max(S - K, 0) & \text{for a call} \\ \max(K - S, 0) & \text{for a put} \end{cases} \quad (1.11.3)$$

Define the operator

$$LV \equiv V_\tau - \left(\frac{\sigma^2 S^2}{2} V_{SS} + rSV_S - rV \right) \quad (1.11.4)$$

and let $V(S, 0) = V^*$. More formally, the American option pricing problem can be stated as

$$\begin{aligned} LV &\geq 0 \\ V - V^* &\geq 0 \\ (V - V^*)LV &= 0 \end{aligned} \quad (1.11.5)$$

Chapter 2

More on Ito's Lemma

In Section 1.7.1, we mysteriously made the infamous comment

...it can be shown that $dZ^2 \rightarrow dt$ as $dt \rightarrow 0$

In this Section, we will give some justification for this remark. For a lot more details on this subject, we refer the reader to *Stochastic Differential Equations*, by Bernt Oksendal, Springer, 1998.

We have to go back here, and decide what the statement

$$dX = \alpha dt + c dZ \quad (2.0.1)$$

really means. The only sensible interpretation of this is

$$X(t) - X(0) = \int_0^t \alpha(X(s), s) ds + \int_0^t c(X(s), s) dZ(s) . \quad (2.0.2)$$

where we can interpret the integrals as the limit, as $\Delta t \rightarrow 0$ of a discrete sum. For example,

$$\begin{aligned} \int_0^t c(X(s), s) dZ(s) &= \lim_{\Delta t \rightarrow 0} \sum_{j=0}^{j=N-1} c_j \Delta Z_j \\ c_j &= c(X(t_j), t_j) \\ Z_j &= Z(t_j) \\ \Delta Z_j &= Z(t_{j+1}) - Z(t_j) \\ \Delta t &= t_{j+1} - t_j \\ N &= t/(\Delta t) \end{aligned} \quad (2.0.3)$$

In particular, in order to derive Ito's Lemma, we have to decide what

$$\int_0^t c(X(s), s) dZ(s)^2 \quad (2.0.4)$$

means. Replace the integral by a sum,

$$\int_0^t c(X(s), s) dZ(s)^2 = \lim_{\Delta t \rightarrow 0} \sum_{j=0}^{j=N-1} c(X_j, t_j) \Delta Z_j^2 . \quad (2.0.5)$$

Note that we have evaluated the integral using the left hand end point of each subinterval (the *no peeking into the future* principle).

From now on, we will use the notation

$$\sum_j \equiv \sum_{j=0}^{j=N-1} . \quad (2.0.6)$$

Now, we claim that

$$\int_0^t c(X(s), s) dZ^2(s) = \int_0^t c(X(s), s) ds \quad (2.0.7)$$

or

$$\lim_{\Delta t \rightarrow 0} \left[\sum_j c_j \Delta Z_j^2 \right] = \lim_{\Delta t \rightarrow 0} \sum_j c_j \Delta t \quad (2.0.8)$$

which is what we mean by equation (2.0.7). i.e. we can say that $dZ^2 \rightarrow dt$ as $dt \rightarrow 0$.

Now, let's consider a finite Δt , and consider the expression

$$E \left[\left(\sum_j c_j \Delta Z_j^2 - \sum_j c_j \Delta t \right)^2 \right] \quad (2.0.9)$$

If equation (2.0.9) tends to zero as $\Delta t \rightarrow 0$, then we can say that (in the mean square limit)

$$\begin{aligned} \lim_{\Delta t \rightarrow 0} \left[\sum_j c_j \Delta Z_j^2 \right] &= \lim_{\Delta t \rightarrow 0} \sum_j c_j \Delta t \\ &= \int_0^t c(X(s), s) ds \end{aligned} \quad (2.0.10)$$

so that in this sense

$$\int_0^t c(X, s) dZ^2 = \int_0^t c(X, s) ds \quad (2.0.11)$$

and hence we can say that

$$dZ^2 \rightarrow dt \quad (2.0.12)$$

with probability one as $\Delta t \rightarrow 0$.

Now, expanding equation (2.0.9) gives

$$E \left[\left(\sum_j c_j \Delta Z_j^2 - \sum_j c_j \Delta t \right)^2 \right] = \sum_{ij} E \left[c_j (\Delta Z_j^2 - \Delta t) c_i (\Delta Z_i^2 - \Delta t) \right] . \quad (2.0.13)$$

Now, note the following

- The increments of Brownian motion are uncorrelated, i.e. $Cov [\Delta Z_i \Delta Z_j] = 0$, $i \neq j$, which means that $Cov [\Delta Z_i^2 \Delta Z_j^2] = 0$, or $E [(\Delta Z_j^2 - \Delta t)(\Delta Z_i^2 - \Delta t)] = 0$, $i \neq j$.
- $c_i = c(t_i, X(Z_i))$, and ΔZ_i are independent.

It then follows that for $i < j$

$$\begin{aligned} E [c_j(\Delta Z_j^2 - \Delta t)c_i(\Delta Z_i^2 - \Delta t)] &= E[c_i c_j(\Delta Z_i^2 - \Delta t)]E[(\Delta Z_j^2 - \Delta t)] \\ &= 0 . \end{aligned} \quad (2.0.14)$$

Similarly, if $i > j$

$$\begin{aligned} E [c_j(\Delta Z_j^2 - \Delta t)c_i(\Delta Z_i^2 - \Delta t)] &= E[c_i c_j(\Delta Z_j^2 - \Delta t)]E[(\Delta Z_i^2 - \Delta t)] \\ &= 0 . \end{aligned} \quad (2.0.15)$$

So that in all cases

$$E [c_j(\Delta Z_j^2 - \Delta t)c_i(\Delta Z_i^2 - \Delta t)] = \delta_{ij} E [c_i^2(\Delta Z_i^2 - \Delta t)^2] . \quad (2.0.16)$$

It also follows from the above properties that

$$E[c_j^2(\Delta Z_j^2 - \Delta t)^2] = E[c_j^2] E[(\Delta Z_j^2 - \Delta t)^2] \quad (2.0.17)$$

since c_j and $(\Delta Z_j^2 - \Delta t)$ are independent.

Using equations (2.0.16-2.0.17), then equation (2.0.13) becomes

$$\sum_{ij} E [c_j(\Delta Z_j^2 - \Delta t) c_i(\Delta Z_i^2 - \Delta t)] = \sum_i E[c_i^2] E[(\Delta Z_i^2 - \Delta t)^2] . \quad (2.0.18)$$

Now,

$$\sum_i E[c_i^2] E[(\Delta Z_i^2 - \Delta t)^2] = \sum_i E[c_i^2] (E[\Delta Z_i^4] - 2\Delta t E[\Delta Z_i^2] + (\Delta t)^2) . \quad (2.0.19)$$

Recall that ΔZ is $N(0, \Delta t)$ (normally distributed with mean zero and variance Δt) so that

$$\begin{aligned} E[(\Delta Z_i)^2] &= \Delta t \\ E[(\Delta Z_i)^4] &= 3(\Delta t)^2 \end{aligned} \quad (2.0.20)$$

so that equation (2.0.19) becomes

$$E[\Delta Z_i^4] - 2\Delta t E[\Delta Z_i^2] + (\Delta t)^2 = 2(\Delta t)^2 \quad (2.0.21)$$

and

$$\begin{aligned} \sum_i E[c_i^2] E[(\Delta Z_i^2 - \Delta t)^2] &= 2 \sum_i E[c_i^2](\Delta t)^2 \\ &= 2\Delta t \left(\sum_i E[c_i^2] \Delta t \right) \\ &= O(\Delta t) \end{aligned} \quad (2.0.22)$$

so that we have

$$E \left[\left(\sum c_j \Delta Z_j^2 - \sum_j c_j \Delta t \right)^2 \right] = O(\Delta t) \quad (2.0.23)$$

or

$$\lim_{\Delta t \rightarrow 0} E \left[\left(\sum c_j \Delta Z_j^2 - \int_0^t c(s, X(s)) ds \right)^2 \right] = 0 \quad (2.0.24)$$

so that in this sense we can write

$$dZ^2 \rightarrow dt ; dt \rightarrow 0 . \quad (2.0.25)$$

Chapter 3

The Binomial Model

3.1 A Binomial Model Based on the Risk-Neutral Walk

Recall that we can determine the no-arbitrage value of an option by pretending we live in a risk-neutral world, where risky assets drift at r and are discounted at r . If we let $X = \log S$, then the risk-neutral process for X is (from equation (1.7.24))

$$dX = \left(r - \frac{\sigma^2}{2}\right)dt + \sigma dZ . \quad (3.1.1)$$

Now, we can construct a discrete approximation to this random walk using the lattice discussed in Section 1.6. In fact, all we have to do is let $\alpha = r - \frac{\sigma^2}{2}$, so that equation (3.1.1) is formally identical to equation (1.6.1). In order to ensure that in the limit as $\Delta t \rightarrow 0$, we get the process (3.1.1), we require that the sizes of the random jumps are $\Delta X = \sigma\sqrt{\Delta t}$ and that the probabilities of up (p) and down (q) moves are

$$\begin{aligned} p^r &= \frac{1}{2} \left[1 + \frac{\alpha}{\sigma} \sqrt{\Delta t} \right] \\ &= \frac{1}{2} \left[1 + \left(\frac{r}{\sigma} - \frac{\sigma}{2} \right) \sqrt{\Delta t} \right] \\ q^r &= \frac{1}{2} \left[1 - \frac{\alpha}{\sigma} \sqrt{\Delta t} \right] \\ &= \frac{1}{2} \left[1 - \left(\frac{r}{\sigma} - \frac{\sigma}{2} \right) \sqrt{\Delta t} \right] , \end{aligned} \quad (3.1.2)$$

where we have denoted the risk neutral probabilities by p^r and q^r to distinguish them from the real probabilities p, q .

Now, we will switch to a more common notation. If we are at node j , timestep n , we will denote this node location by X_j^n . Recall that $X = \log S$, so that in terms of asset price, this is $S_j^n = e^{X_j^n}$.

Now, consider that at node (j, n) , the asset can move up with probability p^r and down with probability q^r . In other words

$$\begin{aligned} S_j^n &\rightarrow S_{j+1}^{n+1} ; && \text{with probability } p^r \\ S_j^n &\rightarrow S_j^{n+1} ; && \text{with probability } q^r \end{aligned} \quad (3.1.3)$$

Now, since in Section 1.6 we showed that $\Delta X = \sigma\sqrt{\Delta t}$, so that ($S = e^X$)

$$\begin{aligned} S_{j+1}^{n+1} &= S_j^n e^{\sigma\sqrt{\Delta t}} \\ S_j^{n+1} &= S_j^n e^{-\sigma\sqrt{\Delta t}} \end{aligned} \quad (3.1.4)$$

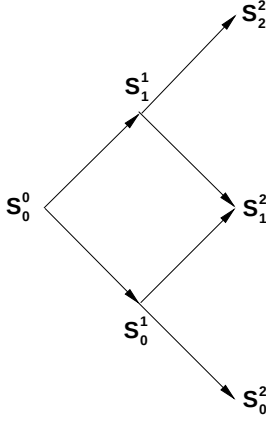


Figure 3.1.1: Lattice of stock price values

or

$$S_j^n = S_0^0 e^{(2j-n)\sigma\sqrt{\Delta t}} ; j = 0, \dots, n \quad (3.1.5)$$

So, the first step in the process is to construct a tree of stock price values, as shown on Figure 3.1.

Associated with each stock price on the lattice is the option value V_j^n . We first set the value of the option at $T = N\Delta t$ to the payoff. For example, if we are valuing a put option, then

$$V_j^N = \max(K - S_j^N, 0) ; j = 0, \dots, N \quad (3.1.6)$$

Then, we can use the risk neutral world idea to determine the no-arbitrage value of the option (it is the expected value in the risk neutral world). We can do this by working backward through the lattice. The value today is the discounted expected future value

European Lattice Algorithm

$$\begin{aligned} V_j^n &= e^{-r\Delta t} (p^r V_{j+1}^{n+1} + q^r V_j^{n+1}) \\ n &= N-1, \dots, 0 \\ j &= 0, \dots, n \end{aligned} \quad (3.1.7)$$

Rolling back through the tree, we obtain the value at S_0^0 today, which is V_0^0 .

If the option is an American put, we can determine if it is optimal to hold or exercise, since we know the continuation value. In this case the rollback (3.1.7) becomes

American Lattice Algorithm

$$\begin{aligned} (V_j^n)^c &= e^{-r\Delta t} (p^r V_{j+1}^{n+1} + q^r V_j^{n+1}) \\ V_j^n &= \max((V_j^n)^c, \max(K - S_j^n, 0)) \\ n &= N-1, \dots, 0 \\ j &= 0, \dots, n \end{aligned} \quad (3.1.8)$$

which is illustrated in Figure 3.1.2.

The binomial lattice method has the following advantages

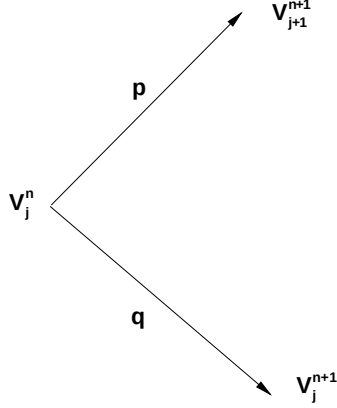


Figure 3.1.2: Backward recursion step.

- It is very easy to code for simple cases.
- It is easy to explain to managers.
- American options are easy to handle.

However, the binomial lattice method has the following disadvantages

- Except for simple cases, coding becomes complex. For example, if we want to handle simple barrier options, things become nightmarish.
- This method is algebraically identical to an explicit finite difference solution of the Black-Scholes equation. Consequently, convergence is at an $O(\Delta t)$ rate.
- The *probabilities* p^r, q^r are not real probabilities; they are simply the coefficients in a particular discretization of a PDE. Regarding them as probabilities leads to much fuzzy thinking, and complex wrong-headed arguments.

3.2 A No-arbitrage Lattice

We can also derive the lattice method directly from the discrete lattice model in Section 1.6. Suppose we assume that

$$dS = \mu S dt + \sigma S dZ \quad (3.2.1)$$

and letting $X = \log S$, we have that

$$dX = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dZ \quad (3.2.2)$$

so that $\alpha = \mu - \frac{\sigma^2}{2}$ in equation (1.6.16). Now, let's consider the usual hedging portfolio at $t = n\Delta t, S = S_j^n$,

$$P_j^n = V_j^n - (\alpha^h) S_j^n, \quad (3.2.3)$$

where V_j^n is the value of the option at $t = n\Delta t$, $S = S_j^n$. At $t = (n+1)\Delta t$,

$$\begin{aligned} S_j^n &\rightarrow S_{j+1}^{n+1} ; & \text{with probability } p \\ S_j^n &\rightarrow S_j^{n+1} ; & \text{with probability } q \\ S_{j+1}^{n+1} &= S_j^n e^{\sigma\sqrt{\Delta t}} \\ S_j^{n+1} &= S_j^n e^{-\sigma\sqrt{\Delta t}} \end{aligned}$$

so that the value of the hedging portfolio at $t = n+1$ is

$$P_{j+1}^{n+1} = V_{j+1}^{n+1} - (\alpha^h) S_{j+1}^{n+1} ; \quad \text{with probability } p \quad (3.2.4)$$

$$P_j^{n+1} = V_j^{n+1} - (\alpha^h) S_j^{n+1} ; \quad \text{with probability } q . \quad (3.2.5)$$

Now, as in Section 1.4, we can determine (α^h) so that the value of the hedging portfolio is independent of p, q . We do this by requiring that

$$P_{j+1}^{n+1} = P_j^{n+1} \quad (3.2.6)$$

so that

$$V_{j+1}^{n+1} - (\alpha^h) S_{j+1}^{n+1} = V_j^{n+1} - (\alpha^h) S_j^{n+1}$$

which gives

$$(\alpha^h) = \frac{V_{j+1}^{n+1} - V_j^{n+1}}{S_{j+1}^{n+1} - S_j^{n+1}} . \quad (3.2.7)$$

Since this portfolio is risk free, it must earn the risk free rate of return, so that

$$\begin{aligned} P_j^n &= e^{-r\Delta t} P_{j+1}^{n+1} \\ &= e^{-r\Delta t} P_j^{n+1} . \end{aligned} \quad (3.2.8)$$

Now, substitute for P_j^n from equation (3.2.3), with P_{j+1}^{n+1} from equation (3.2.5), and (α^h) from equation (3.2.7) gives

$$\begin{aligned} V_j^n &= e^{-r\Delta t} (p^{r*} V_{j+1}^{n+1} + q^{r*} V_j^{n+1}) \\ p^{r*} &= \frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \\ q^{r*} &= 1 - p^{r*} . \end{aligned} \quad (3.2.9)$$

Note that p^{r*}, q^{r*} do not depend on the real drift rate μ , which is expected. If we expand p^{r*}, q^{r*} in a Taylor series, and compare with the p^r, q^r in equations (3.1.2), we can show that

$$\begin{aligned} p^{r*} &= p^r + O((\Delta t)^{3/2}) \\ q^{r*} &= q^r + O((\Delta t)^{3/2}) . \end{aligned} \quad (3.2.10)$$

After a bit more work, one can show that the value of the option at $t = 0$, V_0^0 using either p^{r*}, q^{r*} or p^r, q^r is the same to $O(\Delta t)$, which is not surprising, since these methods can both be regarded as an explicit finite difference approximation to the Black-Scholes equation, having truncation error $O(\Delta t)$. The definition p^{r*}, q^{r*} is the common definition in finance books, since the tree has no-arbitrage.

What is the meaning of a *no-arbitrage* tree? If we are sitting at node S_j^n , and assuming that there are only two possible future states

$$\begin{aligned} S_j^n &\rightarrow S_{j+1}^{n+1} ; & \text{with probability } p \\ S_j^n &\rightarrow S_j^{n+1} ; & \text{with probability } q \end{aligned}$$

then using (α^h) from equation (3.2.7) guarantees that the hedging portfolio has the same value in both future states.

But let's be a bit more sensible here. Suppose we are hedging a portfolio of Blackberry stock. Let $\Delta t =$ one day. Suppose the price of Blackberry stocks is \$10 today. Do we actually believe that tomorrow there are only two possible prices for Blackberry stock,

$$\begin{aligned} S_{up} &= 10e^{\sigma\sqrt{\Delta t}} \\ S_{down} &= 10e^{-\sigma\sqrt{\Delta t}} ? \end{aligned} \tag{3.2.11}$$

Of course not. This is obviously a highly simplified model. The fact that there is no arbitrage in the context of the simplified model does not really have a lot of relevance to the real-world situation. The best that can be said is that if the Black-Scholes model was perfect, then we have that the portfolio hedging ratios computed using either p^r, q^r or p^{r*}, q^{r*} are both correct to $O(\Delta t)$.

3.3 A Drifting Lattice

All of the lattice methods we have discussed so far can be described in the following general way.

$$\begin{aligned} S_j^n &\rightarrow S_{j+1}^{n+1} = S_j^n u ; & \text{with probability } p \\ S_j^n &\rightarrow S_j^{n+1} = S_j^n d ; & \text{with probability } (1-p) \\ S_j^n &= S_0^0 u^j d^{n-j} , \end{aligned} \tag{3.3.1}$$

where u, d are the up and down multipliers. For example, the lattice of Section 3.1, can be described by

$$\begin{aligned} u &= e^{\sigma\sqrt{\Delta t}} \\ d &= \frac{1}{u} \\ p &= \frac{1}{2} \left[1 + \left(\frac{r}{\sigma} - \frac{\sigma}{2} \right) \sqrt{\Delta t} \right] , \end{aligned} \tag{3.3.2}$$

while the no-arbitrage lattice of Section 3.2 can be described by

$$\begin{aligned} u &= e^{\sigma\sqrt{\Delta t}} \\ d &= \frac{1}{u} \\ a &= e^{r\Delta t} \\ p &= \frac{a - d}{u - d} . \end{aligned} \tag{3.3.3}$$

A problem with parameters (3.3.2) or (3.3.3) is that if $r\Delta t > \sigma\sqrt{\Delta t}$, then $p > 1$, which is obviously undesirable. In fact, this leads to an unstable recursion. This can be avoided by using a *drifting lattice*.

Recall our random walk on a lattice in Section 1.6. Let $X_j^n = \log(S_j^n)$, so we will first determine our lattice parameters for X and then use Ito's Lemma to convert this to the GBM case. Recall that

$$\begin{aligned} X_{j+1}^{n+1} &\rightarrow X_j^n + \sigma\sqrt{\Delta t} ; \text{ with probability } p \\ X_j^{n+1} &\rightarrow X_j^n - \sigma\sqrt{\Delta t} ; \text{ with probability } (1-p) \\ p &= \frac{1}{2} \left[1 + \frac{\alpha}{\sigma} \sqrt{\Delta t} \right] . \end{aligned} \quad (3.3.4)$$

Define

$$\begin{aligned} \Delta X_j^n &= (X_{j+1}^{n+1} | X_j^n) - X_j^n \\ &= \{ \sigma\sqrt{\Delta t}, -\sigma\sqrt{\Delta t} \} . \end{aligned} \quad (3.3.5)$$

Recall that

$$\begin{aligned} E[\Delta X_j^n] &= \alpha \Delta t \\ Var[\Delta X_j^n] &= \sigma^2 \Delta t , \end{aligned} \quad (3.3.6)$$

and that the walk on the discrete lattice converges to the solution of the SDE

$$dX = \alpha dt + \sigma dZ \quad (3.3.7)$$

as $\Delta t \rightarrow 0$. Now, suppose that the drift $\alpha = 0$, then from equation (3.3.4) $p = 1/2$, and the walk converges to the SDE

$$dX = \sigma dZ ; \Delta t \rightarrow 0 . \quad (3.3.8)$$

We would like to leave $p = 1/2$, but change the lattice so that we recover the original drift rate. Consider the following

$$\begin{aligned} X_{j+1}^{n+1} &= X_j^n + \sigma\sqrt{\Delta t} + \alpha\Delta t \\ X_j^{n+1} &= X_j^n - \sigma\sqrt{\Delta t} + \alpha\Delta t \\ \Delta X_j^n &= \{ \sigma\sqrt{\Delta t} + \alpha\Delta t, -\sigma\sqrt{\Delta t} + \alpha\Delta t \} . \end{aligned} \quad (3.3.9)$$

Consequently, (noting that we leave $p = 1/2$)

$$\begin{aligned} E[\Delta X_j^n] &= p [\sigma\sqrt{\Delta t} + \alpha\Delta t] + (1-p) [-\sigma\sqrt{\Delta t} + \alpha\Delta t] \\ &= \alpha\Delta t . \end{aligned} \quad (3.3.10)$$

In addition

$$\begin{aligned} Var[\Delta X_j^n] &= E[(\Delta X_j^n)^2] - (E[\Delta X_j^n])^2 \\ &= p [(\sigma\sqrt{\Delta t} + \alpha\Delta t)^2] + (1-p) [(-\sigma\sqrt{\Delta t} + \alpha\Delta t)^2] - (\alpha\Delta t)^2 \\ &= \sigma^2 \Delta t . \end{aligned} \quad (3.3.11)$$

Consequently, the walk with $p = 1/2$ and X_j^{n+1}, X_{j+1}^{n+1} given by equation (3.3.9) converges (as $\Delta t \rightarrow 0$) to the solution of the SDE

$$dX = \alpha dt + \sigma dZ . \quad (3.3.12)$$

Now, we can price options by taking the expectation in the risk neutral world. Assuming the underlying asset follows GBM, then in the risk neutral process for the price S of the asset is given by

$$\frac{dS}{S} = r dt + \sigma dZ . \quad (3.3.13)$$

T	1.0
σ	.3
S_0 (initial price)	100
r	= .02
K	100
Option Type	Put

Table 3.3.1: Data used in lattice tests

Let $X = \log S$, then from Ito's Lemma we have

$$dX = \left(r - \frac{\sigma^2}{2} \right) dt + \sigma dZ . \quad (3.3.14)$$

This corresponds the risk neutral walk on the lattice with parameters

$$\begin{aligned} \alpha &= r - \frac{\sigma^2}{2} \\ X_j^n &= \log S_j^n . \end{aligned} \quad (3.3.15)$$

Now, substitute equation (3.3.15) into equation (3.3.9) to obtain

$$\begin{aligned} S_j^n &\rightarrow S_{j+1}^{n+1} = S_j^n u ; \text{ with probability } p \\ S_j^n &\rightarrow S_j^{n+1} = S_j^n d ; \text{ with probability } (1 - p) \\ p &= \frac{1}{2} \\ u &= \exp[\sigma\sqrt{\Delta t} + (r - \sigma^2/2)\Delta t] \\ d &= \exp[-\sigma\sqrt{\Delta t} + (r - \sigma^2/2)\Delta t] . \end{aligned} \quad (3.3.16)$$

Consequently, we can use the usual lattice backwards recursion with parameters (3.3.16), and this will converge to the correct price.

Compared to the lattice with parameters (3.3.3), the lattice with parameters (3.3.16) will be stable for any values of $r, \sigma, \Delta t$. However, the lattice constructed using parameters (3.3.16) is unbalanced, in the sense that $S_j^n u d \neq S_j^n$.

3.3.1 Numerical Comparison: No-arbitrage Lattice and Drifting Lattice

We use the test data in Table 3.3.1. Table 3.3.2 shows a convergence study for both lattice methods. We expect that the error in the lattice method is $O(\Delta t)$. In other words

$$V^{tree}(\Delta t, S, t) = V^{exact}(S, t) + C_1 \Delta t + C_2 (\Delta t)^2 \quad (3.3.17)$$

where C_1 is independent of Δt . This suggests that

$$\lim_{\Delta t \rightarrow 0} \frac{V^{tree}((\Delta t)/2, S_0^0, t=0) - V^{tree}(\Delta t, S_0^0, t=0)}{V^{tree}((\Delta t)/4, S_0^0, t=0) - V^{tree}((\Delta t)/2, S_0^0, t=0)} = 2 . \quad (3.3.18)$$

We can see from Table 3.3.2 that for sufficiently small Δt , this appears to be true for the no-arbitrage lattice parameters. However, convergence seems to be somewhat erratic for the drifting lattice parameters (3.3.16). A lattice method can be viewed as an explicit finite difference method. Due to the fact that the payoff has a discontinuous derivative at the strike, finite difference theory suggests that in order to achieve a good rate of convergence, there should always be a lattice node at the strike. If we use an even number

Timesteps	Value	Change	Ratio
American Put: No-arbitrage lattice (3.3.3)			
500	11.0093931		
1000	11.0113180	0.0037	
2000	11.0122835	9.6550e-04	3.80
4000	11.0127632	4.7970e-04	2.01
8000	11.0130028	2.3960e-04	2.00
16000	11.0131227	1.1990e-04	2.00
European Put: No arbitrage lattice (3.3.3)			
500	10.8355351		
1000	10.8384915	0.0029564	
2000	10.839970	0.001478	2.00
4000	10.8407094	7.394e-04	2.00
8000	10.8410790	3.6960e-4	2.00
16000	10.8412639	1.8489e-04	1.99
American Put: Drifting lattice (3.3.16)			
500	11.0114885		
1000	11.0157093	0.001882	
2000	11.0133918	-0.002317	-0.81225
4000	11.0139094	5.17599e-04	-4.476
8000	11.0134913	-4.1809e-04	-1.2380
16000	11.0133705	-1.20799e-04	3.46
European Put: Drifting Lattice (3.3.16)			
500	10.8383929		
1000	10.8435056	0.00511	
2000	10.8413199	-0.002185	-2.344
4000	10.8420495	7.2909e-04	-2.996
8000	10.8416523	-3.972e-04	-1.835
16000	10.8415488	-1.0349e-04	3.8

Table 3.3.2: Option value at $S = 100$, $t = 0$, data in Table 3.3.1. Change is the change from the previous solution computed using $1/2$ the timesteps. Ratio is ratio of successive changes.

Timesteps	Value	Change	Ratio
European Put: No-arbitrage lattice (3.3.3)			
500	14.5868102		
1000	14.5907463	0.00393609	
2000	14.5887935	-0.0019528	-2.01
4000	14.5885792	-2.14299e-04	9.11
8000	14.5886970	1.17799e-04	- 1.81
16000	14.5885398	-1.57200e-04	-0.749

Table 3.3.3: Option value at $S = 92$, $t = 0$, data in Table 3.3.1. Change is the change from the previous solution computed using 1/2 the timesteps. Ratio is ratio of successive changes.

of timesteps, then this is true for a no-arbitrage lattice. However, this is not true for a drifting lattice. We suspect that this explains the poor convergence for this case.

To verify the importance of having a node at the strike, let's use the no-arbitrage parameters (3.3.3), and price a European put option with the parameters in Table 3.3.1, except that we set $S_0 = 92$. In general, in this case, there will not be a node at the strike. The results are shown in Table 3.3.3. The convergence is very erratic.

3.4 Smoothing the Payoff

As we can see from the numerical results in Section 3.3.1, it is sometimes not possible to ensure that the lattice has a node at the strike, which causes erratic convergence behaviour.

From the analysis of finite difference methods for PDEs, it is known that good convergence can usually be restored if the payoff is smoothed appropriately.

For simplicity, we will derive the smoothing technique for the case of the no-arbitrage lattice (3.3.3). The same final expressions will also be valid for the drifting lattice parameters (3.3.16). For the no-arbitrage lattice, with $N = T/(\Delta t)$,

$$S_j^N = S_0^0 e^{(2j-N)\sigma\sqrt{\Delta t}} \quad (3.4.1)$$

so that, letting $X_j^N = \log(S_j^N)$, then

$$\begin{aligned} X_{j+1}^N &= X_j^N + 2\sigma\sqrt{\Delta t} \\ X_{j-1}^N &= X_j^N - 2\sigma\sqrt{\Delta t} . \end{aligned} \quad (3.4.2)$$

Let $g(X)$ represent the payoff function. For example,

$$g(X) = \begin{cases} \max(e^X - K, 0) & \text{Call} \\ \max(K - e^X, 0) & \text{Put} \end{cases} , \quad (3.4.3)$$

with strike K . Now, a simple way to smooth the payoff is take its average value around node X_j^N . Let $\hat{g}(x)$ denote the smoothed payoff. Then

$$\hat{g}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{X_j^N - \sigma\sqrt{\Delta t}}^{X_j^N + \sigma\sqrt{\Delta t}} g(x) dx . \quad (3.4.4)$$

Let $u = X - X_j^N$, then equation (3.4.4) becomes

$$\hat{g}(X_j^N) = \hat{g}_j^N = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} g(X_j^N + u) du . \quad (3.4.5)$$

For a put, the smoothed payoff $\hat{P}(X_j^N)$ would be

$$\begin{aligned}\hat{P}(X_j^N) &= \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} \max(K - e^{X_j^N} e^u, 0) du \\ &= \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} \max(K - S_j^N e^u, 0) du ,\end{aligned}\tag{3.4.6}$$

and the smoothed call payoff $\hat{C}(X_j^N)$ is

$$\hat{C}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} \max(S_j^N e^u - K, 0) du .\tag{3.4.7}$$

For a put, from equation (3.4.6),

$$\begin{aligned}\max(K - S_j^N e^u, 0) &= 0 \quad ; \quad u \in [-\sigma\sqrt{\Delta t}, +\sigma\sqrt{\Delta t}] \\ &\quad S_j^N e^{-\sigma\sqrt{\Delta t}} > K .\end{aligned}\tag{3.4.8}$$

On the other hand,

$$\begin{aligned}\max(K - S_j^N e^u, 0) &= K - S_j^N e^u \quad ; \quad u \in [-\sigma\sqrt{\Delta t}, +\sigma\sqrt{\Delta t}] \\ &\quad S_j^N e^{+\sigma\sqrt{\Delta t}} < K .\end{aligned}\tag{3.4.9}$$

In this case,

$$\begin{aligned}\frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} (K - S_j^N e^u) du &= \frac{1}{2\sigma\sqrt{\Delta t}} \left(K 2\sigma\sqrt{\Delta t} - S_j^N (e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}) \right) \\ &= K - S_j^N \left(\frac{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}{2\sigma\sqrt{\Delta t}} \right) .\end{aligned}\tag{3.4.10}$$

If $S_j^N e^{-\sigma\sqrt{\Delta t}} \leq K \leq S_j^N e^{\sigma\sqrt{\Delta t}}$ then

$$\begin{aligned}&\frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{+\sigma\sqrt{\Delta t}} \max(K - S_j^N e^u, 0) du \\ &= \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{\log(K/S_j^N)} (K - S_j^N e^u) du \\ &= \frac{1}{2\sigma\sqrt{\Delta t}} \left(K [\log(K/S_j^N) + \sigma\sqrt{\Delta t}] - S_j^N [K/S_j^N - e^{-\sigma\sqrt{\Delta t}}] \right) .\end{aligned}\tag{3.4.11}$$

Putting this all together

$$\hat{P}_j^N = \begin{cases} 0 & S_j^N e^{-\sigma\sqrt{\Delta t}} > K \\ K - S_j^N \left(\frac{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}{2\sigma\sqrt{\Delta t}} \right) & S_j^N e^{+\sigma\sqrt{\Delta t}} < K \\ \frac{1}{2\sigma\sqrt{\Delta t}} \left(K [\log(K/S_j^N) + \sigma\sqrt{\Delta t}] - S_j^N [(K/S_j^N) - e^{-\sigma\sqrt{\Delta t}}] \right) & S_j^N e^{-\sigma\sqrt{\Delta t}} \leq K \leq S_j^N e^{\sigma\sqrt{\Delta t}} \end{cases}\tag{3.4.12}$$

Timesteps	Value	Change	Ratio
European Put: No-arbitrage lattice (3.3.3)			
500	14.5921428		
1000	14.5902654	-0.0018774	
2000	14.5893256	-0.00093979	1.997
4000	14.5888575	-0.00046810	2.007
8000	14.5886207	-0.00023749	1.971
16000	14.5885031	-0.0001169	2.031
American Put: No-arbitrage lattice (3.3.3)			
500	14.8598750		
1000	14.8573440	-0.00253	
2000	14.8559674	-0.001376	1.839
4000	14.8552981	-0.0006693	2.057
8000	14.8549594	-0.0003387	1.976
16000	14.8547869	-0.0001725	1.963

Table 3.4.1: Option value at $S = 92$, $t = 0$, data in Table 3.3.1. Change is the change from the previous solution computed using 1/2 the timesteps. Ratio is ratio of successive changes. Smoothed payoff.

Timesteps	Value	Change	Ratio
American Put: Drifting lattice (3.3.16)			
500	11.0190437		
1000	11.0162151	-0.002828599	
2000	11.0147342	-0.001480900	1.910
4000	11.0140015	-0.0007327	2.021
8000	11.0136218	-0.0003797	1.929
16000	11.0134323	-0.000189499	2.003

Table 3.4.2: Option value at $S = 100$, $t = 0$, data in Table 3.3.1. Change is the change from the previous solution computed using 1/2 the timesteps. Ratio is ratio of successive changes. Smoothed payoff. Drifting lattice.

Similarly, the smoothed call payoff $\hat{C}_j^N$ is

$$\hat{C}_j^N = \begin{cases} 0 & S_j^N e^{+\sigma\sqrt{\Delta t}} < K \\ S_j^N \left(\frac{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}{2\sigma\sqrt{\Delta t}} \right) - K & S_j^N e^{-\sigma\sqrt{\Delta t}} > K \\ \frac{1}{2\sigma\sqrt{\Delta t}} \left(S_j^N \left[e^{\sigma\sqrt{\Delta t}} - (K/S_j^N) \right] - K \left[\sigma\sqrt{\Delta t} - \log(K/S_j^N) \right] \right) & S_j^N e^{-\sigma\sqrt{\Delta t}} \leq K \leq S_j^N e^{\sigma\sqrt{\Delta t}} \end{cases} \quad (3.4.13)$$

Now, let's repeat the test shown in Table 3.3.3, this time using the smoothed payoffs. The results are shown in Table 3.4.1. The convergence is much improved, and is now close to first order.

As another check, we repeat the test for the American option in Table 3.3.2, using the Drifting lattice (3.3.16) with the smoothed payoff. These results are given in Table 3.4.2, for the case $S_0 = 100$. Again, the convergence is close to first order, and is much smoother than without smoothing.

Timesteps	Value	Change	Extrapolated
American Put: Drifting lattice (3.3.16)			
500	11.0190437		
1000	11.0162151	-0.002828599	11.0133865
2000	11.0147342	-0.001480900	11.0132532
4000	11.0140015	-0.0007327	11.0132687
8000	11.0136218	- 0.0003797	11.0132421
16000	11.0134323	-0.000189499	11.0132428
32000	11.0133381	-9.419999e-5	11.0132439
64000	11.0132903	-4.78000e-5	11.0132425

Table 3.4.3: Option value at $S = 100$, $t = 0$, data in Table 3.3.1. Change is the change from the previous solution computed using $1/2$ the timesteps. Extrapolation refers to Richardson extrapolation (3.4.17). Smoothed payoff. Drifting lattice.

3.4.1 Richardson extrapolation

From Table (3.4.2) it seems we have a smoothly converging set of values for different timesteps Δt . Let $P_{\Delta t}$ be the lattice result with a timestep of size Δt . Then

$$P_{\Delta t} = P_{exact} + O(\Delta t) , \quad (3.4.14)$$

where P_{exact} is the exact solution. For Δt small, we can approximate equation (3.4.14) by

$$P_{\Delta t} = P_{exact} + C\Delta t + O((\Delta t)^2) \quad (3.4.15)$$

where C is assumed to be a constant independent of Δt . Ignoring the $O((\Delta t)^2)$ term in equation (3.4.15), we can write equation (3.4.15) for two different timestep sizes

$$\begin{aligned} P_{\Delta t} &= P_{exact} + C\Delta t \\ P_{\Delta t/2} &= P_{exact} + C\frac{\Delta t}{2} , \end{aligned} \quad (3.4.16)$$

or, eliminating C from equations (3.4.16) gives

$$P_{exact} = P_{\Delta t/2} + (P_{\Delta t/2} - P_{\Delta t}) . \quad (3.4.17)$$

Of course, we will not get the real exact solution, but we should end up with a better estimate than $P_{\Delta t/2}$. Table 3.4.3 shows the results of applying Richardson extrapolation for a sequence of runs with decreasing timesteps. Note that the extrapolated solution at $N = 2000$ appears to have about six digits correct, compared to the non-extrapolated result ($N = 2000$) which has about four digits correct.

3.5 MATLAB and Python Implementations

Below are MATLAB and Python/Numpy implementations of a European option, using the no-arbitrage lattice parameters, written using loops.

TreeSlow.m

```
% Compute Black-Scholes option value using a binomial tree
% European case
% uses loops

S0 = 100;          % S0 - current stock price
K = 100;           % K - strike
T = 1.0;           % T - expiry time
r = 0.02;          % r - interest rate
sigma = .3;        % sigma - volatility
opttype = 1;       % opttype - 0 for a call, otherwise a put
Nsteps = 10000;    % Nsteps - number of timesteps

delt = T/Nsteps;

%
% tree parameters
%
    u = exp(sigma * sqrt(delt) );
    d = 1./u;
    a = exp( r*delt );
    p = (a - d)/(u - d);
%
% payoff at t=T
%
    W= zeros(Nsteps+1,1);

    % W(j+1,1) corresponds to node  $S_{\{j\}}^N$ 
    % j=0,...,Nsteps

    for j=0:Nsteps
        W(j+1,1) = S0*u^(j)*d^(Nsteps - j);
        % offset by one since
        % matlab arrays start at 1
    end

    %vector operations

    if( opttype == 0)
        W = max( W - K, 0);
    else
        W = max( K - W, 0);
    end

% backward recursion

for n=Nsteps-1:-1:0
    for j=0:n % matlab arraya start at 1
        W(j+1,1) = exp(-r*delt)*( p*W(j+2,1) + (1-p)*W(j+1,1) );
    end
end

value = W(1);

disp(sprintf('Tree Value: %.9g \n',value));
```

TreeSlow in Python

```
# Compute Black-Scholes option value using a binomial tree - European case
# uses loops

import numpy as np

S0 = 100      # S0 - current stock price
K = 100;      # K - strike
T = 1;        # T - expiry time
r = .02;      # r - interest rate
sigma = .3;   # sigma - volatility
opttype = 1;  # opttype - 0 for a call, otherwise a put
Nsteps = 1000 # Nsteps - number of timesteps - low step count because it's slow!

delt = T/Nsteps;

# tree parameters
u = np.exp(sigma * np.sqrt(delt) );
d = 1./u;
a = np.exp( r*delt );
p = (a - d)/(u - d);

# payoff at t=T
W = np.zeros(Nsteps+1);

# W[j] corresponds to node S_{j}^N
# j=0,...,Nsteps
for j in np.arange(0,Nsteps+1):
    W[j] = S0*(u**(j))*(d**(Nsteps - j))

#vector operations - determine payoff values at N
if opttype == 0: #call
    W = np.maximum(W - K, 0)
else: #put
    W = np.maximum(K - W, 0)

# backward recursion
for n in reversed(range(Nsteps)):
    for j in np.arange(n+1):
        W[j] = np.exp(-r*delt)*( p*W[j+1] + (1-p)*W[j] )

print("Tree Value: " + str(W[0]))
```

It is, of course, more efficient to vectorize MATLAB and/or Python code, i.e., avoid using explicit loops, as in the *TreeFast* codes below.

TreeFast.m

```
%
% Compute Black-Scholes option value using a binomial tree
% European case
% vectorized code

S0 = 100;          % S0 - current stock price
K = 100;           % K - strike
T = 1.0;           % T - expiry time
r = 0.02;          % r - interest rate
sigma = 0.3;       % sigma - volatility
opttype = 1;       % opttype - 0 for a call, otherwise a put
Nsteps = 10000;    % Nsteps - number of timesteps

delt = T/Nsteps;

% tree parameters

u = exp(sigma * sqrt(delt) );
d = 1./u;
a = exp( r*delt );
p = (a - d)/(u - d);

%
% payoff at t=T
%
W = S0*d.^([Nsteps:-1:0]')*.u.^([0:Nsteps]');
% W is column vector of size Nsteps+1 X 1
if( opttype == 0)
    W = max( W - K, 0);
else
    W = max( K - W, 0);
end

% backward recursion

for i=Nsteps:-1:1
    W = exp(-r*delt)*( p*W(2:i+1) + (1-p)*W(1:i) );
end

value = W(1);

disp(sprintf('Tree Value: %.9g \n',value));
```

TreeFast in Python

```
# Compute Black-Scholes option value using a binomial tree - European case
# vectorized code - should be faster!

import numpy as np

S0 = 100          # S0 - current stock price
K = 100;          # K - strike
T = 1;            # T - expiry time
r = 0.02;         # r - interest rate
sigma = .3;       # sigma - volatility
opttype = 1;      # opttype - 0 for a call, otherwise a put
Nsteps = 10000    # Nsteps - number of timesteps

delt = T/Nsteps;

# tree parameters
u = np.exp(sigma * np.sqrt(delt) );
d = 1./u;
a = np.exp( r*delt );
p = (a - d)/(u - d);

# payoff at t=T
W = S0 * d**(np.arange(Nsteps,-1,-1)) * u**(np.arange(Nsteps+1))

# W is column vector of length Nsteps+1
if opttype == 0:
    W = np.maximum( W - K, 0);
else:
    W = np.maximum( K - W, 0);

# backward recursion
for i in np.arange(Nsteps,0,-1):
    W = np.exp(-r*delt)*( p*W[1:i+1] + (1-p)*W[0:i] )

print("Tree value: " + str(W[0]))
```

Implementing the American case requires the payoff at each node in the tree. This can be done efficiently using vector statements. For example, since $S_{j+1}^{n+1} = S_j^n u$, one could generate the asset prices at timestep n in terms of the asset prices at timestep $n + 1$, using a vector statement.

3.6 Dynamic Programming

Note that in equation (3.1.8) we were able to determine the optimal strategy for exercising an American option by working backwards through the lattice, and comparing the value of immediate exercise with the value of continuing to hold. This is an example of the Dynamic Programming Principle.

This idea was originally developed by Richard Bellman. Here is a quote from his address to the American Mathematical society in 1953:

“An optimal policy has the property that, whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy given the state resulting from that first decision.”

This can be stated more succinctly as

Principle of Optimality *From any point on an optimal control trajectory, the remaining trajectory is optimal for the corresponding problem initiated at that point.*

Here is a classic example of the application of dynamic programming. Suppose you desire to marry the richest person you meet. We make the assumptions that

- If you ask anyone to marry you, they will immediately accept.
- You are allowed to meet only N persons.
- The wealth of each person is randomly uniformly distributed in $[0, 1]$.

What is the optimal strategy to maximize the expected wealth of your marriage partner?

One way to solve this problem would be think about what happens when you meet the first person. You could then compare that person's wealth with what you might expect to get later on if you rejected the first person. But, what you expect to get later on is a complicated mixture of all the possible decisions you could make later. This becomes very difficult to work out.

This is where the Principle of Optimality comes to the rescue. Let's consider the simple problem of what to do if you encounter potential partner $N - 1$. At this stage, you only have two choices

- Ask $N - 1$ to marry you.
- Marry partner N .

What should you do? If you marry partner N , then the expected wealth of N is .5. Thus, you should marry $N - 1$ if his/her wealth is more than .5, otherwise marry N . The probability that partner $N - 1$ has wealth greater than .5 is .5. You thus marry partner $N - 1$ about 1/2 the time, and 1/2 the time you will marry partner N . The expected wealth of partner $N - 1$, given that you marry him/her is $(.5 + 1)/2 = .75$.

Let V_N be the expected value of a partner at step N . In this case $V_N = .5$. Let W_{N-1} be the wealth of a partner you marry at step $N - 1$ (assuming you stop at step $N - 1$). Let $\overline{W}_{N-1}$ be the expected wealth of your partner at step $N - 1$ assuming you act optimally. More precisely

$$\overline{W}_{N-1} = E[W_{N-1} | W_{N-1} > V_N] \quad (3.6.1)$$

where $E[\cdot]$ is the expectation operator. Let p be the probability that $W_{N-1} > V_N$. Your expected partner wealth at step $N - 1$ is thus

$$V_{N-1} = p \times \overline{W}_{N-1} + (1 - p) \times V_N \quad (3.6.2)$$

$$\begin{aligned} &= .5 \times .75 + .5 \times .5 \\ &= .625 \end{aligned} \quad (3.6.3)$$

Note that V_{N-1} is the expected partner wealth assuming you act optimally at step $N - 1$.

Now, writing down equation (3.6.2) at step $N - 2$ gives

$$\begin{aligned} V_{N-2} &= p \times \overline{W}_{N-2} + (1 - p) \times V_{N-1} \\ p &= \text{Prob}[W_{N-2} > V_{N-1}] \\ \overline{W}_{N-2} &= E[W_{N-2} | W_{N-2} > V_{N-1}] \end{aligned} \quad (3.6.4)$$

Since $V_{N-1} = .625$, we have $p = (1 - .625) = .375$, and $\overline{W}_{N-2} = (.625 + 1)/2 = .8125$. Your expected partner wealth at step $N - 2$ is then

$$\begin{aligned} V_{N-2} &= .375 \times \overline{W}_{N-2} + .625 \times V_{N-1} \\ &= .375 \times .8125 + .625 \times .625 \\ &= .6953125 \end{aligned} \quad (3.6.5)$$

Again, note that V_{N-2} is the expected partner wealth assuming you act optimally at steps $N-2, N-1$. We can continue working backwards to step $N-3, N-4$ and so on. In general

$$\begin{aligned} V_k &= p \times \bar{W}_k + (1-p) \times V_{k+1} \\ p &= \text{Prob}[W_k > V_{k+1}] \\ \bar{W}_k &= E[W_k | W_k > V_{k+1}] , \end{aligned} \tag{3.6.6}$$

where

$$\begin{aligned} V_k &= E[\text{Wealth of partner, at step } k, \text{ assuming optimal strategy} \\ &\quad \text{used at steps } k, k+1, \dots] \\ W_k &= \text{Wealth of partner } k \text{ assuming it is optimal to stop at step } k . \end{aligned} \tag{3.6.7}$$

This algorithm is shown in Figure 3.6.1. If $N = 10$, then the expected partner wealth is about .861.

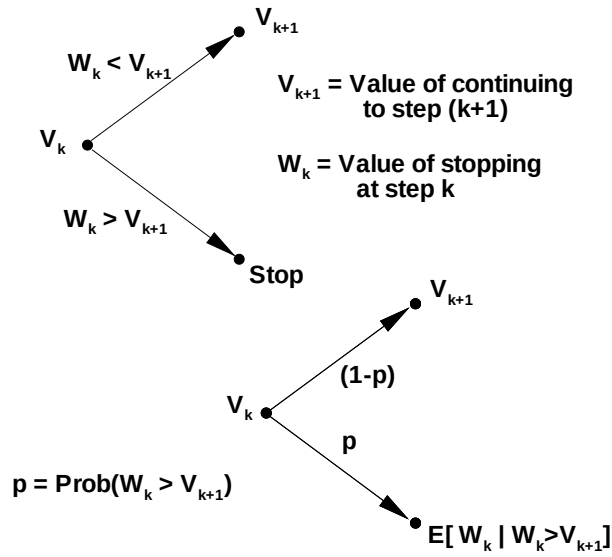


Figure 3.6.1: The dynamic programming algorithm. V_{k+1} is the value of continuing, assuming the optimal strategy is followed at steps $k+1, k+2, \dots$

Chapter 4

Introduction to Computational Mathematics

The goal of computational mathematics, put simply, is to find or develop algorithms that solve mathematical problems *computationally* (i.e., using computers). In particular, we desire that any algorithm we develop fulfills three primary properties:

- **Accuracy.** An accurate algorithm is able to return a result that is numerically very close to the correct, or analytical, result.
- **Efficiency.** An efficient algorithm is able to quickly solve the mathematical problem with reasonable computational resources.
- **Robustness.** A robust algorithm works for a wide variety of inputs $\mathbf{x}$.

In examining computational mathematics problems, we shall generally consider a problem of the following form:

Problem Consider an arbitrary problem $\mathbf{P}$ with input $\mathbf{x}$. We must compute the desired output $\mathbf{z} = f_P(\mathbf{x})$.

In general our only tools for solving such problems are primitive mathematical operations (for example, addition, subtraction, multiplication and division) combined with flow constructs (if statements and loops). As such, even simple problems such as evaluating the exponential function may be difficult computationally.

Example 4.1 Consider the problem $\mathbf{P}$ defined by the evaluation of the exponential function $z = \exp(x)$. We wish to find the approximation $\hat{z}$ for $z = \exp(x)$ computationally.

Algorithm A. Recall from calculus that the Taylor series expansion of the exponential function is given by

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \sum_{i=0}^{\infty} \frac{x^i}{i!}. \quad (4.0.1)$$

Since we obviously cannot compute an infinite sum computationally without also using infinite resources, consider the truncated series constructed by only summing the first n terms. This yields the expansion

$$\hat{z} = \sum_{i=0}^n \frac{x^i}{i!}. \quad (4.0.2)$$

As we will later see, this is actually a poor algorithm for computing the exponential - although the reason for this may not be immediately obvious.

4.1 Sources of Error

Computational error can originate from several sources, although the most prominent forms of error (and the ones that we will worry about in this text) come from two main sources:

1. Errors introduced in the input $\mathbf{x}$. In particular, this form of error can be generally classified to be from one of two sources:
 - a) Measurement error, caused by a difference between the “exact” value $\mathbf{x}$ and the “measured” value $\tilde{\mathbf{x}}$. The measurement error is computed as $\Delta\mathbf{x} = |\mathbf{x} - \tilde{\mathbf{x}}|$.
 - b) Rounding error, caused by a difference between the “exact” value $\mathbf{x}$ and the computational or floating-point representation $\hat{\mathbf{x}} = fl(\mathbf{x})$. Since infinite precision cannot be achieved with finite resources, the computational representation is a finite precision approximation of the exact value.

Consider, for example, the decimal number $x = 0.00012345876543$. In order to standardize the representation of these numbers we perform normalization (such that the number to the left of the decimal point is 0 and the first digit right of the decimal point is nonzero). The number $\hat{x}$ is thus normalized as follows:

$$x = 0.\underbrace{12345876543}_{\text{mantissa}} \times 10^{-3}. \quad (4.1.1)$$

However, a computer can only represent finite precision, so we are not guaranteed to retain all digits from the initial number. Let’s consider a hypothetical “decimal” (i.e., base-10) computer with at most 5 digits in the mantissa. The floating-point representation of x obtained on this computer, after rounding, is given by

$$\hat{x} = fl(x) = 0.12346 \times 10^{-3}. \quad (4.1.2)$$

The process of conversion to a floating point number gives us a rounding error equal to $\Delta x = x - \hat{x} = -0.00000123457$.

2. Errors as a result of the calculation, approximation, or algorithm. This form of error can again be generally classified to be from one of two sources:
 - a) Truncation error. When truncating an infinite series to provide a finite approximation, the method inherently introduces an error. In example 4.1 we first considered truncating the infinite expansion as follows:

$$z = \exp(x) = \sum_{i=0}^{\infty} \frac{x^i}{i!} = \sum_{i=0}^n \frac{x^i}{i!} + \sum_{i=n+1}^{\infty} \frac{x^i}{i!} = \hat{z} + \sum_{i=n+1}^{\infty} \frac{x^i}{i!} \quad (4.1.3)$$

In this case, the truncation error is given by

$$T = z - \hat{z} = \sum_{i=n+1}^{\infty} \frac{x^i}{i!} \quad (4.1.4)$$

- b) Rounding errors in elementary steps of the algorithm. For example, consider the addition of 1.234×10^7 and 5.678×10^3 in a floating point number system where we only maintain 4 digit accuracy (in base 10). The exact result should be 1.2345678×10^7 but due to rounding we instead calculate the result to be 1.235×10^7 .

In analyzing sources of error, it is often useful to provide a mathematical basis for the error analysis. In particular, there are two principal ways for providing a measure of the generated error.

Definition 4.1 Consider an exact result z and an approximate result $\hat{z}$ generated in a specified floating point number system. Then the **absolute error** is given by

$$\Delta z = z - \hat{z}, \quad (4.1.5)$$

and the **relative error** (assuming $z \neq 0$) is given by

$$\delta z = \frac{z - \hat{z}}{z}. \quad (4.1.6)$$

Of course, considering the mathematical hardships often involved in analyzing our algorithms for error, there must be some justification in bothering with error analysis. Although the errors may be first perceived as being negligible, some numerical algorithms are “numerically unstable” in the way they propagate errors. In doing error analysis, we will often run into one of the following situations in computational mathematics:

1. The algorithm may contain one or more “avoidable” steps that each greatly amplify errors.
2. The algorithm may propagate initially small errors such that the error is amplified without bound in many small steps.

Consider the following example of the first kind of generated error:

Example 4.2 Consider the problem **P** with input x defined by the evaluation of the exponential function $z = \exp(x)$ (as considered in (4.1)). However, in performing the calculation, assume the following conditions:

- Assume a “decimal” computer with 5 digits in the mantissa.
- Assume no measurement error or initial rounding error in x .

Consider solving this problem with input $x = -5.5$; we find a numerical approximation to the exact value $z = \exp(-5.5) \approx 0.0040868$.

In solving this problem, we first apply Algorithm A, truncating the series after the first 25 values. This yields the formula $\hat{z} = \sum_{i=0}^{24} \frac{x^i}{i!}$. Performing this calculation with our floating point system above yields the approximation $\hat{z}_A = 0.0057563$ (see Table I) which an observant reader will notice has *no significant digits* in common with the exact solution. We conclude that precision was lost in this calculation, and in fact notice that the relative error is $\delta z_A = \frac{1}{z}(z - \hat{z}_A) = -0.41 = -41\%$. This is a substantial error!

So why does this instability occur? Our answer to this question can be found in a related problem: Consider the subtraction of two numbers that are almost equal to one another.

Let $x_1 = 0.100134826$ with floating-point representation $\mathbf{fl}(x_1) = 0.10013$. The relative error in performing this approximation is $\delta x_1 = \frac{1}{x_1}(x_1 - \mathbf{fl}(x_1)) = 4.8 \times 10^{-5} = 0.0048\%$.

Let $x_2 = 0.100121111$ with floating-point representation $\mathbf{fl}(x_2) = 0.10012$. The relative error in performing this approximation is $\delta x_2 = \frac{1}{x_2}(x_2 - \mathbf{fl}(x_2)) = 1.1 \times 10^{-5} = 0.0011\%$.

So, in general, the approximation of these two numbers to their floating-point equivalents produce relatively small errors. Now consider the subtraction $z = x_1 - x_2$. The exact solution to this is $z = 0.000013715$ and the computed solution using the floating-point representations is $\hat{z} = \mathbf{fl}(x_1) - \mathbf{fl}(x_2) = 0.00001$. The relative error in this case is $\delta \hat{z} = \frac{1}{z}(z - \hat{z}) \approx 27\%$. So what was an almost negligible error when performing rounding becomes a substantial error after we have completed the subtraction.

Table I: Algorithm A applied to $x = -5.5$

i	i^{th} term in series	i^{th} truncated sum
0	1.00000000000000	1.00000000000000
1	-5.50000000000000	-4.50000000000000
2	15.12500000000000	10.62500000000000
3	-27.73000000000000	-17.10500000000000
4	38.12800000000000	21.02300000000000
5	-41.94000000000000	-20.91700000000000
6	38.44600000000000	17.52900000000000
7	-30.20600000000000	-12.67700000000000
8	20.76700000000000	8.09000000000000
9	-12.69100000000000	-4.60100000000000
10	6.98030000000000	2.37930000000000
11	-3.49000000000000	-1.11070000000000
12	1.59960000000000	0.48890000000000
13	-0.67676000000000	-0.18786000000000
14	0.26587000000000	0.07801000000000
15	-0.09748400000000	-0.01947400000000
16	0.03351000000000	0.01403600000000
17	-0.01084200000000	0.00319400000000
18	0.00331270000000	0.00650670000000
19	-0.00095890000000	0.00554780000000
20	0.00026371000000	0.00581150000000
21	-0.00006906700000	0.00574240000000
22	0.00001726700000	0.00575970000000
23	-0.00000412890000	0.00575560000000
24	0.00000094623000	0.00575650000000
25	-0.00000020817000	0.00575630000000
26	0.00000004403500	0.00575630000000

Thus, if possible, we will need to avoid these kinds of subtractions when we are developing our algorithms. Looking back at the additions we performed in calculating $\exp(-5.5)$ (see Table I), we see that there were several subtractions performed of numbers of similar magnitude. Fortunately, we have an easy way around this if we simply take advantage of a property of the exponential, namely $\exp(-x) = (\exp(x))^{-1}$. This provides us with Algorithm B:

Algorithm B. Applying the truncated Taylor expansion for $\exp(5.5)$, we get the following formula for $\exp(-5.5)$:

$$\exp(x) = \left(\sum_{i=0}^{24} \frac{(-x)^i}{i!} \right)^{-1} \quad (4.1.7)$$

This yields $\hat{z}_B = 0.0040865$, which matches 4 out of 5 digits of the exact value.

We conclude that Algorithm B is numerically stable (largely since it avoids cancellation). Similarly, Algorithm A is unstable with respect to relative error for the input $x = -5.5$. However, it is also worth noting that for any positive value of the input we are better off using Algorithm A — Algorithm B would end up with the same cancellation issues we had originally encountered with Algorithm A.

4.2 Floating Point Numbers and Operations

Floating point numbers are the standard tool for approximating real numbers on a computer. Unlike real numbers, floating point numbers only provide finite precision — effectively approximating real numbers while still attempting to provide full functionality. Consider the following definition of a floating point number system:

Definition 4.2 A *floating point number system* is defined by three components:

- The **base**, which defines the base of the number system being used in the representation. This is specified as a positive integer b_f .
- The **mantissa**, which contains the normalized value of the number being represented. Its maximal size is specified as a positive integer m_f , which represents the number of digits allowed in the mantissa.
- The **exponent**, which effectively defines the offset from normalization. Its maximal size is specified by a positive integer e_f , which represents the number of digits allowed in the exponent.

In shorthand, we write $F[b = b_f, m = m_f, e = e_f]$.

Combined, the three components allow us to approximate any real number in the following form:

$$0.\underbrace{x_1x_2\cdots x_m}_{\text{mantissa}} \times \underbrace{b}_{\text{base}}^{\underbrace{y_1y_2\cdots y_e}_{\text{exponent}}} \quad (4.2.1)$$

Example 4.3 Consider a “decimal” computer with a floating point number system defined by base 10, 5 digits in the mantissa and 3 digits in the exponent. In shorthand, we write the system as $F[b = 10, m = 5, e = 3]$.

Consider the representation of the number $x = 0.000123458765$ under this system. To find the representation, we first normalize the value and then perform rounding so both the mantissa and exponent have the correct number of digits:

input	$x = 0.000123458765$
normalize	$x = 0.123458765 \times 10^{-3}$
round	$\hat{x} = \text{fl}(x) = 0.12346 \times 10^{-003}$

Under this system, our mantissa is bounded by 99999 and the exponent is bounded by 999 (each representing the largest numbers we can display under this base in 5 and 3 digits, respectively). The largest number we can represent under this system is 0.99999×10^{999} and the smallest positive number we can represent is 0.10000×10^{-999} .

Note: Instead of rounding in the final step, we can also consider “chopping”. With chopping, we simply drop all digits that cannot be represented under our system. Thus, in our example we would get $\hat{x} = \text{fl}(x) = 0.12345 \times 10^{-003}$ since all other digits simply would be dropped.

4.2.1 A Binary Computer

Instead of working in decimal (base 10), almost all computers work in binary (base 2). We normally write binary numbers as $(x)_b$ to indicate that they are represented in binary.

Example 4.4 Consider the binary number $x = (1101.011)_b$. Similar to decimal, this notation is equivalent to writing

$$\begin{aligned} x &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} \\ &= 8 + 4 + 1 + 0.25 + 0.125 \\ &= 13.375 \end{aligned}$$

Floating-point number conversions from binary work exactly the same way as decimal conversions, except for the new base:

Example 4.5 Consider the binary number $x = (1101.011)_b$ under the floating-point system $F[b = 2, m = 4, e = 3]$.

$$\begin{aligned}
 \text{input} \quad x &= (1101.011)_b \\
 \text{normalize} \quad x &= (0.1101011)_b \times 2^4 \\
 &= (0.1101011)_b \times 2^{(100)_b} \\
 \text{round} \quad \hat{x} = \mathbf{fl}(x) &= (0.1101)_b \times 2^{(100)_b} \\
 &= (0.1101)_b \times (10)_b^{(100)_b}
 \end{aligned}$$

4.2.2 Standard floating point systems

Single Precision Numbers. Single precision is one of the standard floating point number systems, where numbers are represented on a computer in a 32-bit chunk of memory (4 bytes). Each number in this system is divided as follows:

s_m	$m = 23 \text{ bits}$	s_e	$e = 7 \text{ bits}$
-------	-----------------------	-------	----------------------

Here s_m is the sign bit of the mantissa and s_e is the sign bit for the exponent.

Recall that we normally write floating point numbers in the form given by (4.2.1). The typical convention for sign bits is to use 0 to represent positive numbers and 1 to represent negative numbers.

Example 4.6 Consider the decimal number $x = -0.3125$. We note that we may write this in binary as

$$\begin{aligned}
 x &= -0.3125 \\
 &= -(0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4}) \\
 &= -(0.0101)_b \\
 &= -(0.101)_b \times 2^{-1} \\
 &= -(0.101)_b \times 2^{-(1)_b}
 \end{aligned}$$

In the single precision floating point system $F[b = 2, m = 23, e = 7]$ we may write $\hat{x} = \mathbf{fl}(x) = |1|101000000000000000000000|1|0000001|$.

Under the single-precision floating point number system, the largest and smallest numbers that can be represented are given as follows (without consideration for normalization, in the case of the smallest number).

$$\begin{aligned}
 \hat{x}_{max} &= |0|111111111111111111111111|0|1111111| \\
 &= (1 - 2^{-23}) \cdot 2^{127} \\
 &\approx 2^{127} \\
 &\approx 1.7 \times 10^{38}
 \end{aligned}$$

$$\begin{aligned}
 \hat{x}_{min} &= |0|000000000000000000000000|1|1111111| \\
 &= 2^{-23} \cdot 2^{-127} \\
 &= 2^{-150} \\
 &\approx 7.0 \times 10^{-46}
 \end{aligned}$$

Note that there are 512 ways to represent zero under the single precision system proposed here (we only require that the mantissa is zero, meaning that the signed bits and exponent can be arbitrary). Under the IEEE standard (which is used by real computers) there are some additional optimizations that take advantage of this “lost” space:

1. use of a signed integer for the exponent
2. only allow for one way to represent zero and use the free space to represent some smaller numbers
3. do not store the first 1 in the mantissa (because it is always 1 for a normalized number)

Double Precision Numbers. Double precision is the other standard floating point number system. Numbers are represented on a computer in a 64-bit chunk of memory (8 bytes), divided as follows:

s_m	m = 52 bits	s_e	e = 10 bits
-------	-------------	-------	-------------

In this case, the maximum and minimum numbers that can be represented are $x_{max} \approx 9 \times 10^{307}$ and $x_{min} \approx 2.5 \times 10^{-324}$, respectively.

4.2.3 Machine Precision

Recall that the relative error (defined by equation (4.1.6)) when converting a real number x to a floating point number is

$$\delta x = \frac{x - \mathbf{fl}(x)}{x}. \quad (4.2.2)$$

This motivates the question: Is there a bound on the absolute value of the relative error $|\delta x|$ for a given floating point number system?

An answer to this requires some delving into the way in which floating point numbers are represented. Consider the floating point number given by

$$\pm 0.x_1x_2 \cdots x_m \times b^{\pm t} \quad (4.2.3)$$

with $1 \leq x_i < b$ for $i = 1, \dots, m$. We then define the machine epsilon:

Definition 4.3 The *machine epsilon* ϵ_{mach} is the smallest number $\epsilon > 0$ such that $\mathbf{fl}(1 + \epsilon) > 1$.

We then present the following proposition:

Proposition 4.1 The machine epsilon is given by

- a) $\epsilon_{mach} = b^{1-m}$ if chopping is used
- b) $\epsilon_{mach} = \frac{1}{2}b^{1-m}$ if rounding is used

Proof. For simplicity we only prove part (a) of the proposition. If rounding is used a more complex analysis will yield the result of part (b).

Consider the following subtraction:

$$\begin{array}{rcll}
 1 + \epsilon & = & 0. & \underbrace{1}_{b^{-1}} \underbrace{0}_{b^{-2}} \underbrace{0}_{b^{-3}} \cdots \underbrace{0}_{b^{-(m-1)}} \underbrace{1}_{b^{-m}} \times b^1 \\
 1 & = & 0. & \underbrace{1}_{x_1} \underbrace{0}_{x_2} \underbrace{0}_{x_3} \cdots \underbrace{0}_{x_{m-1}} \underbrace{0}_{x_m} \times b^1 \\
 \hline
 \epsilon & = & 0. & 0 \quad 0 \quad 0 \quad \cdots \quad 0 \quad \underbrace{1}_{b^{-m}} \times b^1 \\
 \epsilon & = & b^{1-m}
 \end{array}$$

Theorem 4.1 For any floating point system F under chopping,

$$|\delta x| = \left| \frac{x - \text{fl}(x)}{x} \right| \leq \epsilon_{mach}. \quad (4.2.4)$$

Proof. Consider the following calculation:

$$\begin{aligned} x &= 0.d_1 d_2 \cdots d_m d_{m+1} d_{m+2} \cdots \cdot b^t \\ \text{fl}(x) &= 0.d_1 d_2 \cdots d_m \cdot b^t \end{aligned}$$

Thus combining these two equations yields

$$\begin{aligned} \frac{x - \text{fl}(x)}{x} &= \frac{0.00 \cdots 0 d_{m+1} d_{m+2} \cdots}{0.d_1 d_2 \cdots d_m d_{m+1} d_{m+2} \cdots} \\ &= \frac{0.d_{m+1} d_{m+2} \cdots}{0.d_1 d_2 \cdots d_m d_{m+1} d_{m+2} \cdots} \cdot b^{-m}. \end{aligned}$$

But we know the numerator is less than or equal to 1 ($1 = 1 \cdot b^0$) and the denominator is greater than or equal to 0.1 ($0.1 = 1 \cdot b^{-1}$). So we may write:

$$\begin{aligned} (x - \text{fl}(x)) \cdot x^{-1} &\leq (1) \cdot (b^{-1})^{-1} \cdot b^{-m} \\ &= b^{-m+1} \\ &= \epsilon_{mach} \end{aligned}$$

Note. Since $\delta x = \frac{1}{x}(x - \text{fl}(x))$, we may also write $\text{fl}(x) = x(1 - \delta x)$ with $|\delta x| \leq \epsilon_{mach}$. Hence we often say

$$\text{fl}(x) = x(1 + \eta) \quad \text{with } |\eta| \leq \epsilon_{mach}. \quad (4.2.5)$$

Single Precision. Under single precision, $m = 23$ and so $\epsilon = 2^{-22} \approx 0.24 \times 10^{-6}$. Thus $|\delta x| \leq 0.24 \times 10^{-6}$, and so we expect 6 to 7 decimal digits of accuracy.

Double Precision. Under double precision, $m = 52$ and so $\epsilon = 2^{-51} \approx 0.44 \times 10^{-15}$. Thus $|\delta x| \leq 0.44 \times 10^{-15}$, and so we expect 15 to 16 decimal digits of accuracy.

4.2.4 Floating Point Operations

Definition 4.4 The symbol $\oplus$ is used to denote floating point addition, defined as

$$a \oplus b = \text{fl}(\text{fl}(a) + \text{fl}(b)) \quad (4.2.6)$$

Proposition 4.2 For any floating point number system F ,

$$a \oplus b = (\text{fl}(a) + \text{fl}(b))(1 + \eta) \quad (4.2.7)$$

with $|\eta| \leq \epsilon_{mach}$. This may also be written as

$$a \oplus b = (a(1 + \eta_1) + b(1 + \eta_2))(1 + \eta) \quad (4.2.8)$$

with $|\eta_1|, |\eta_2|$ and $|\eta| \leq \epsilon_{mach}$. In general the operation of addition under F is not associative. That is,

$$(a \oplus b) \oplus c \neq a \oplus (b \oplus c). \quad (4.2.9)$$

Note: There are analogous operations for subtraction, multiplication and division, written using the symbols $\ominus$, $\otimes$, and $\oslash$.

4.3 Conditioning of a Mathematical Problem

Consider a problem $\mathbf{P}$ with input $\tilde{\mathbf{x}}$ which requires the computation of a desired output $\mathbf{z} = f_P(\tilde{\mathbf{x}})$. In general, some mathematical problems of this form are very sensitive to small deviations in the input data, in which case there are a variety of problems (such as rounding in input data) which make accurate approximation on a computer difficult.

Definition 4.5 We say that a problem $\mathbf{P}$ is **well-conditioned** with respect to the absolute error if small changes $\Delta\tilde{\mathbf{x}}$ in $\tilde{\mathbf{x}}$ result in small changes $\Delta\tilde{\mathbf{z}}$ in $\tilde{\mathbf{z}}$. Similarly, we say $\mathbf{P}$ is **ill-conditioned** with respect to the absolute error if small changes $\Delta\tilde{\mathbf{x}}$ in $\tilde{\mathbf{x}}$ result in large changes $\Delta\tilde{\mathbf{z}}$ in $\tilde{\mathbf{z}}$.

Definition 4.6 The condition number of a problem $\mathbf{P}$ with respect to the absolute error is given by the **absolute condition number** κ_A :

$$\kappa_A = \|\Delta\tilde{\mathbf{z}}\| / \|\Delta\tilde{\mathbf{x}}\|. \quad (4.3.1)$$

The condition number with respect to the relative error is given by the **relative condition number** κ_R :

$$\kappa_R = \frac{\|\Delta\tilde{\mathbf{z}}\| / \|\tilde{\mathbf{z}}\|}{\|\Delta\tilde{\mathbf{x}}\| / \|\tilde{\mathbf{x}}\|} \quad (4.3.2)$$

If κ_A and κ_R are “small” we can generally infer that $\mathbf{P}$ is well-conditioned. As a guideline, if κ_A and κ_R are between 0.1 and 10 we can consider them to be “small.” Similarly, if κ_A and κ_R are “large” (tend to ∞ or “blow up”) then we can say that $\mathbf{P}$ is ill-conditioned.

Example 1. Consider the mathematical problem $\mathbf{P}$ defined by $z = x + y$. We wish to examine how the errors in x and y propagate to z . We define our approximations by $\Delta x = x - \hat{x}$ or $\hat{x} = x - \Delta x$, where Δx is the error in x . Similarly, we have $\Delta y = y - \hat{y}$ or $\hat{y} = y - \Delta y$, where Δy is the error in y . In solving the problem, we are given the approximations $\hat{x}$ and $\hat{y}$ and compute the approximation $\hat{z}$:

$$\hat{z} = \hat{x} + \hat{y} = (x + y) - (\Delta x + \Delta y).$$

and so we define our error by

$$\Delta z = z - \hat{z} = \Delta x + \Delta y. \quad (4.3.7)$$

a) **Conditioning with respect to the absolute error**

We attempt to find an upper bound for κ_A . Using equation (4.3.7) and the 1-norm (4.3.5), we can write

$$\kappa_A = \frac{|\Delta z|}{\|(\Delta x, \Delta y)\|_1} = \frac{|\Delta x + \Delta y|}{|\Delta x| + |\Delta y|}.$$

But by the triangle inequality we have $|\Delta x + \Delta y| \leq |\Delta x| + |\Delta y|$ and so yield

$$\kappa_A \leq \frac{|\Delta x| + |\Delta y|}{|\Delta x| + |\Delta y|} = 1. \quad (4.3.8)$$

We conclude that $\mathbf{P}$ is well-conditioned with respect to the absolute error.

b) **Conditioning with respect to the relative error**

We attempt to find an upper bound for κ_R . Consider the relative error in z , given by

$$|\delta z| = \frac{|\Delta z|}{|z|} = \frac{|\Delta x + \Delta y|}{|x + y|}.$$

Intermission: Vector Norms

Vector norms are a useful tool for providing a measure of the magnitude of a vector, and are particularly applicable to derivations for the condition number.

Definition 4.7 Suppose V is a vector space over $\mathbb{R}^n$. Then $\|\cdot\|$ is a **vector norm** on V if and only if $\|\vec{v}\| \geq 0$, and

- a) $\|\vec{v}\| = 0$ if and only if $\vec{v} = \vec{0}$
- b) $\|\lambda\vec{v}\| = |\lambda|\|\vec{v}\| \quad \forall \vec{v} \in V, \quad \forall \lambda \in \mathbb{R}$
- c) $\|\vec{u} + \vec{v}\| \leq \|\vec{u}\| + \|\vec{v}\| \quad \forall \vec{u}, \vec{v} \in V$ (triangle inequality)

There are three standard vector norms known as the 2-norm, the ∞ -norm and the 1-norm.

Definition 4.8 The **2-norm** over $\mathbb{R}^n$ is defined as

$$\|\vec{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} \quad (4.3.3)$$

Definition 4.9 The **∞ -norm** over $\mathbb{R}^n$ is defined as

$$\|\vec{x}\|_\infty = \max_{1 \leq i \leq n} (x_i) \quad (4.3.4)$$

Definition 4.10 The **1-norm** over $\mathbb{R}^n$ is defined as

$$\|\vec{x}\|_1 = \sum_{i=1}^n |x_i| \quad (4.3.5)$$

Further, all vector norms that are induced by an inner product (including the 1-norm and 2-norm - but not the ∞ -norm) satisfy the Cauchy-Schwartz Inequality, which will be of use later:

Theorem 4.2 *Cauchy-Schwartz Inequality.* Let $\|\cdot\|$ be a vector norm over a vector space V induced by an inner product. Then

$$|\vec{x} \cdot \vec{y}| \leq \|\vec{x}\| \|\vec{y}\| \quad (4.3.6)$$

We can clearly see here that if $x \approx -y$ then $|\delta z|$ can be very large, even though $|\Delta x|/|x|$ and $|\Delta y|/|y|$ may not be large. The relative condition number is thus

$$\kappa_R = \frac{|\Delta z|/|z|}{\|\Delta \vec{x}\|_1/\|\vec{x}\|_1} = \frac{|\Delta x + \Delta y|/|x + y|}{(|\Delta x| + |\Delta y|)/(|x| + |y|)} \leq \frac{|x| + |y|}{|x + y|}. \quad (4.3.9)$$

Thus we see that κ_R can grow arbitrarily large when $x \approx -y$.

We conclude that the problem $z = x + y$ is ill-conditioned with respect to the relative error only in the case $x \approx -y$.

Example 2. Consider the mathematical problem $\mathbf{P}$ defined by $z = x \cdot y$. We define our approximations by $\Delta x = x - \hat{x}$ and $\Delta y = y - \hat{y}$ as in Example 1. In solving the problem, we are given the approximations $\hat{x}$ and $\hat{y}$ and compute the approximation $\hat{z}$:

$$\begin{aligned} \hat{z} = \hat{x} \cdot \hat{y} &= (x - \Delta x)(y - \Delta y) \\ &= xy - y\Delta x - x\Delta y + \overbrace{\Delta x \Delta y}^{\text{neglect}} \\ &\approx xy - y\Delta x - x\Delta y \end{aligned}$$

and so we define our error by

$$\Delta z \approx y\Delta x + x\Delta y = (x, y) \cdot (\Delta y, \Delta x). \quad (4.3.10)$$

The relative error in z is given by

$$\delta z \approx \frac{1}{xy}(y\Delta x + x\Delta y) = \frac{\Delta x}{x} + \frac{\Delta y}{y}. \quad (4.3.11)$$

a) **Condition with respect to the absolute error**

We attempt to find an upper bound for κ_A . Using equation (4.3.10) and the Cauchy-Schwartz Inequality (4.3.6), we can write

$$\kappa_A = \frac{|\Delta z|}{\|(\Delta x, \Delta y)\|} \leq \frac{\|(x, y)\|_2 \|(\Delta x, \Delta y)\|_2}{\|(\Delta x, \Delta y)\|_2} = \|(x, y)\|_2.$$

We conclude that $\mathbf{P}$ is well-conditioned with respect to the absolute error, except when x or y are large.

b) **Condition with respect to the relative error**

From (4.3.11) we have that

$$\delta z \approx \frac{\Delta x}{x} + \frac{\Delta y}{y} = \delta x + \delta y$$

and so can immediately conclude that $\mathbf{P}$ is well-conditioned with respect to the relative error. In fact, in this particular case

$$\kappa_R = \frac{|\Delta z|/|z|}{\|\Delta \vec{x}\|/\|\vec{x}\|}$$

does not easily yield a useful bound.

Intermission: Asymptotic Behaviour of Polynomials

We wish to consider a general method of analyzing the behaviour of polynomials in two cases: when $x \rightarrow 0$ and when $x \rightarrow \infty$. In particular, if we are only interested in the asymptotic behaviour of the polynomial as opposed to the exact value of the function, we may employ the concept of Big-Oh notation.

Definition 4.11 Suppose that $f(x)$ is a polynomial in x without a constant term. Then the following are equivalent:

- a) $f(x) = O(x^n)$ as $x \rightarrow 0$.
- b) $\exists c > 0, x_0 > 0$ such that $|f(x)| < c|x|^n \forall x$ with $|x| < |x_0|$.
- c) $f(x)$ is bounded from above by $|x|^n$, up to a constant c , as $x \rightarrow 0$.

This effectively means that the dominant term in $f(x)$ is the term with x^n as $x \rightarrow 0$, or $f(x)$ goes to zero with order n .

Example 4.7 Consider the polynomial $g(x) = 3x^2 + 7x^3 + 10x^4 + 7x^{12}$. We say

$$\begin{aligned} g(x) &= O(x^2) \text{ as } x \rightarrow 0 \\ g(x) &\neq O(x^3) \text{ as } x \rightarrow 0 \\ g(x) &= 3x^2 + O(x^3) \text{ as } x \rightarrow 0 \end{aligned}$$

We note that $g(x) = O(x)$ as well, but this statement is not so useful because it is not a sharp bound.

We may also consider the behaviour of a polynomial as $x \rightarrow \infty$.

Definition 4.12 Suppose that $f(x)$ is a polynomial in x . Then the following are equivalent:

- a) $f(x) = O(x^n)$ as $x \rightarrow \infty$.
- b) $\exists c > 0, x_0 > 0$ such that $|f(x)| < c|x|^n \forall x$ with $|x| > |x_0|$.
- c) $f(x)$ is bounded from above by $|x|^n$, up to a constant c , as $x \rightarrow \infty$.

As before, this effectively means that the dominant term in $f(x)$ is the term with x^n as $x \rightarrow \infty$, or $f(x)$ goes to infinity with order n .

Example 4.8 Consider the polynomial $g(x) = 3x^2 + 7x^3 + 10x^4 + 7x^{12}$. We say

$$\begin{aligned} g(x) &= O(x^{12}) \text{ as } x \rightarrow \infty \\ g(x) &\neq O(x^8) \text{ as } x \rightarrow \infty \\ g(x) &= 7x^{12} + O(x^4) \text{ as } x \rightarrow \infty \end{aligned}$$

Addition and Multiplication of Terms Involving Big-Oh

We can also add and multiply terms using Big-Oh notation, making sure to neglect higher order terms:

$$\begin{array}{rcl}
 f(x) & = & x + O(x^2) \qquad \text{as } x \rightarrow 0 \\
 g(x) & = & 2x + O(x^3) \qquad \text{as } x \rightarrow 0 \\
 \hline
 f(x) + g(x) & = & 3x + O(x^2) + O(x^3) \qquad \text{as } x \rightarrow 0 \\
 & = & 3x + O(x^2) \qquad \text{as } x \rightarrow 0 \\
 \\
 f(x) \cdot g(x) & = & 2x^2 + O(x^3) + O(x^4) + O(x^5) \qquad \text{as } x \rightarrow 0 \\
 & = & 2x^2 + O(x^3) \qquad \text{as } x \rightarrow 0
 \end{array}$$

Applications to the Taylor Series Expansion

Recall that the Taylor series expansion for a function $f(x)$ around a point x_0 is given by

$$f(x_0 + \Delta x) = \sum_{n=0}^{\infty} \frac{1}{n!} f^{(n)}(x_0) (\Delta x)^n \quad (4.3.12)$$

We may expand this formula and write it in terms of Big-Oh notation as follows:

$$\begin{aligned}
 f(x_0 + \Delta x) &= f(x_0) \\
 &+ f'(x_0) \Delta x \\
 &+ \frac{1}{2} f''(x_0) \Delta x^2 \\
 &+ \frac{1}{6} f'''(x_0) \Delta x^3 \\
 &+ O(\Delta x^4) \text{ as } \Delta x \rightarrow 0
 \end{aligned}$$

4.4 Stability of a Numerical Algorithm

Consider a problem $\mathbf{P}$ with input $\mathbf{x}$ which requires the computation of a desired output $\mathbf{z} = f_P(\mathbf{x})$. If we assume that $\mathbf{P}$ is well-conditioned, then by definition we have that

$$\|\Delta \vec{x}\| \text{ small} \Rightarrow |\Delta z| \text{ small}$$

$$\|\Delta \vec{x}\|/\|\vec{x}\| \text{ small} \Rightarrow |\Delta z/z| \text{ small}$$

For this well-conditioned problem, some algorithms may be numerically unstable, i.e. they may produce large errors $|\Delta z|$ or $|\delta z|$, while other algorithms may be stable.

Example 4.9 *Stability with respect to the relative error. Consider the problem $\mathbf{P}$ defined by $z = \exp(x)$ with $x = 5.5$. The approximate values for x and z are denoted $\hat{x}$ and $\hat{z}$ respectively. They are related by the following formula:*

$$\hat{z} = \exp(\hat{x}) = \exp(x - \Delta x) \quad (4.4.1)$$

A) We investigate the condition of $\mathbf{P}$ as follows:

a) With respect to Δ we have

$$\kappa_A = \frac{|\Delta z|}{|\Delta x|} = \frac{|z - \hat{z}|}{|x - \hat{x}|} = \frac{|\exp(x) - \exp(x - \Delta x)|}{|x - \hat{x}|}. \quad (4.4.2)$$

We apply the Taylor series expansion given by

$$\exp(x - \Delta x) = \exp(x) - \exp(x)\Delta x + O(\Delta x^2) \quad (4.4.3)$$

to yield,

$$\begin{aligned} \kappa_A &= |\exp(x) - (\exp(x) - \exp(x)\Delta x + O(\Delta x^2))| / |x - \hat{x}| \\ &\approx |\exp(x)\Delta x| / |\Delta x| \\ &= |\exp(x)|, \end{aligned}$$

for small $|\Delta x|$. We conclude that the problem is well conditioned with respect to Δ , except for large x .

b) With respect to δ we have

$$\kappa_R = \frac{|\Delta z|/|z|}{|\Delta x|/|x|} = \left| \frac{\Delta z}{\Delta x} \right| \left| \frac{x}{z} \right| \approx |\exp(x)| \frac{|x|}{|\exp(x)|} = |x|. \quad (4.4.4)$$

We conclude that the problem is well conditioned with respect to δ , except for large x .

B) Now that we know that problem $\mathbf{P}$ is well conditioned, we can investigate the stability of our algorithms for $\mathbf{P}$.

Recall Algorithm A, given by (4.0.2). We determined that this algorithm is unstable with respect to δ for $x < 0$. This was due to a cancellation of nearly equal values in the subtraction, and so the algorithm was unstable due to ill-conditioned steps that could be avoided.

Recall Algorithm B, given by (4.1.7). As opposed to Algorithm A, this algorithm is stable with respect to δ for $x < 0$.

Example 4.10 *Instability with respect to absolute error Δ . Consider the problem $\mathbf{P}$ defined by $z = \int_0^1 \frac{x^n}{x+\alpha} dx$ where $\alpha > 0$. The approximate values for α and z are denoted $\hat{\alpha}$ and $\hat{z}$ respectively. It can be shown that $\mathbf{P}$ is well-conditioned (for instance with respect to the integration boundaries 0 and 1).*

We derive an algorithm for solving this problem using a recurrence relation. In deriving a recurrence, we need to consider the base case (a) and the recursive case (b) and then ensure that they are appropriately related.

a) Consider $n = 0$:

$$\begin{aligned} I_0 &= \int_0^1 \frac{1}{x+\alpha} dx \\ &= \log(x+\alpha) \Big|_0^1 \\ &= \log(1+\alpha) - \log(\alpha) \end{aligned}$$

and so we get

$$I_0 = \log\left(\frac{1+\alpha}{\alpha}\right). \quad (4.4.5)$$

b) For general n we can derive the following recurrence:

$$\begin{aligned} I_n &= \int_0^1 \frac{x^{n-1}x}{x+\alpha} dx \\ &= \int_0^1 \frac{x^{n-1}(x+\alpha-\alpha)}{x+\alpha} dx \\ &= \int_0^1 x^{n-1} dx - \alpha \int_0^1 \frac{x^{n-1}}{x+\alpha} dx \\ &= \frac{x^n}{n} \Big|_0^1 - \alpha I_{n-1} \end{aligned}$$

which yields the expression

$$I_n = \frac{1}{n} - \alpha I_{n-1}. \quad (4.4.6)$$

Thus we may formulate an algorithm using the base case and the recurrence:

Algorithm A.

1. Calculate I_0 from (4.4.5).
2. Calculate $I_1, I_2, \dots, I_n$ using (4.4.6).

An implementation of this algorithm on a computer provides the following results, for two sample values of α :

$$\begin{aligned} \alpha &= 0.5 \longrightarrow I_{100} \approx 6.64 \times 10^{-3} \\ \alpha &= 2.0 \longrightarrow I_{100} \approx 2.1 \times 10^{22} \end{aligned}$$

For $\alpha = 2.0$, we obtain a very large result! This may be a first indication that something is wrong. From the original equation we have that

$$I_{100} = \int_0^1 \frac{x^{100}}{x+\alpha} dx \leq \frac{1}{1+\alpha} \cdot 1 = 1/3. \quad (4.4.7)$$

Hence we conclude something is definitely amiss. We need to consider the propagation of the error in our algorithm, since we know that the algorithm is definitely correct using exact arithmetic.

Consider I_n to be the exact value at each step of the algorithm, with $\hat{I}_n$ the numerical approximation. Then the absolute error at each step of the algorithm is given by $\Delta I_n = I_n - \hat{I}_n$. Thus the initial error is $\Delta I_0 = I_0 - \hat{I}_0 = I_0 - \text{fl}(I_0)$.

The exact value satisfies

$$I_n = \frac{1}{n} - \alpha I_{n-1} \quad (4.4.8)$$

and the numerical approximation is calculated from

$$\hat{I}_n = \frac{1}{n} - \alpha \hat{I}_{n-1} + \eta_n, \quad (4.4.9)$$

where η_n is the rounding error we introduce in step n .

For a first analysis, we neglect η_n and simply investigate the propagation of the initial error ΔI_0 only. Then

$$\begin{aligned} \Delta I_n &= I_n - \hat{I}_n \\ &= \left(\frac{1}{n} - \alpha I_{n-1}\right) - \left(\frac{1}{n} - \alpha \hat{I}_{n-1}\right) \\ &= -\alpha(I_{n-1} - \hat{I}_{n-1}) \\ &= -\alpha \Delta I_{n-1}. \end{aligned}$$

Applying recursion yields an expression for the accumulated error after n steps:

$$\Delta I_n = (-\alpha)^n \Delta I_0. \quad (4.4.10)$$

Conclusion. From this expression, we note that there are potentially two very different outcomes:

- a) If $\alpha > 1$ then the initial error is propagated in such a way that blow-up occurs. Thus Algorithm A is numerically unstable with respect to Δ .
- b) If $\alpha \leq 1$ then the initial error remains constant or dies out. Thus Algorithm A is numerically stable with respect to Δ .

We note that further analysis would lead to the conclusion that the rounding error η_n is also propagated in the same way as the initial error ΔI_0 . These results confirm the experimental results observed in the implementation.

Chapter 5

Root Finding

5.1 Introduction

The root finding problem is a classical example of numerical methods in practice. The problem is stated as follows:

Problem Given any function $f(x)$, find x^* such that $f(x^*) = 0$. The value x^* is called a root of the equation $f(x) = 0$.

If $f(x) = 0$ has a root x^* there is no truly guaranteed method for finding x^* computationally for an arbitrary function, but there are several techniques which are useful for many cases. A computational limitation inherent to this problem can be fairly easily seen by an observant reader: any interval of the real line contains an infinite number of points, but computationally we can solve this problem with only a finite number of evaluations of the function f .

Additionally, since the value x^* may not be defined in our floating point number system, we will not be able to find x^* exactly. Therefore, we consider a computational version of the same problem:

Problem (Computational) Given any $f(x)$ and some error tolerance $\epsilon > 0$, find x^* such that $|f(x^*)| < \epsilon$.

We will only consider functions which are continuous in our analysis.

Example 5.1

- 1) Consider the function $f(x) = x^2 + x - 6 = (x - 2)(x + 3)$. The function $f(x) = 0$ has two roots at $x=2$, -3 .
- 2) With $f(x) = x^2 - 2x + 1 = (x - 1)^2 = 0$, $x = 1$ is a double root (i.e. $f(x^*) = 0$ and $f'(x^*) = 0$).
- 3) $f(x) = 3x^5 + 5x^4 + \frac{1}{3}x^3 + 1 = 0$. We have no general closed form solution for the roots of a polynomial with degree larger than 4. As a result, we will need to use numerical approximation by iteration.
- 4) $f(x) = x^2 - \frac{1}{2}\exp(-x) = 0$. This is naturally more difficult to solve because $\exp(-x)$ is a transcendental function. In fact, $f(x) = 0$ is called the transcendental equation and can only be solved with numerical approximation.

Definition 5.1 We say that x^* is a **double root** of $f(x) = 0$ if and only if $f(x^*) = 0$ and $f'(x^*) = 0$.

We naturally examine this computational problem from an iterative standpoint. That is, we wish to generate a sequence of iterates (x_k) such that any iterate x_{k+1} can be written as some function of $x_k, x_{k-1}, \dots, x_0$. We assume that some initial conditions are applied to the problem so that $x_p, x_{p-1}, \dots, x_0$ are either given or arbitrarily chosen. Obviously, we require that the iterates actually converge to the solution of the problem, i.e. $x^* = \lim_{k \rightarrow \infty} x_k$.

A natural question to ask might be, “how do we know where a root of a function may approximately be located?” A simple result from first year calculus will help in answering this:

Theorem 5.1 (Intermediate Value Theorem) *If $f(x)$ is continuous on a closed interval $[a, b]$ and $c \in [f(a), f(b)]$, then $\exists x^* \in [a, b]$ such that $f(x^*) = c$.*

Thus if we can find $[a, b]$ such that $f(a) \cdot f(b) < 0$ then by the Intermediate Value Theorem, $[a, b]$ will contain at least one root x^* as long as $f(x)$ is continuous.

5.2 Four Algorithms for Root Finding

5.2.1 Bisection Method

The bisection method is one of the most simple methods for locating roots, but is also very powerful as it guarantees convergence as long as its initial conditions are met. To apply this method, we require a continuous function $f(x)$ and an initial interval $[a, b]$ such that $f(a) \cdot f(b) \leq 0$. This method effectively works by bisecting the interval and recursively using the Intermediate Value Theorem to determine a new interval where the initial conditions are met.

Theorem 5.2 *If $f(x)$ is a continuous function on the interval $[a_0, b_0]$ such that $f(a_0) \cdot f(b_0) \leq 0$ then the interval $[a_k, b_k]$, defined by*

$$a_k = \begin{cases} a_{k-1} & \text{if } f((a_{k-1} + b_{k-1})/2) \cdot f(a_{k-1}) \leq 0 \\ (a_{k-1} + b_{k-1})/2 & \text{otherwise} \end{cases} \quad (5.2.1)$$

$$b_k = \begin{cases} b_{k-1} & \text{if } f((a_{k-1} + b_{k-1})/2) \cdot f(a_{k-1}) > 0 \\ (a_{k-1} + b_{k-1})/2 & \text{otherwise} \end{cases} \quad (5.2.2)$$

fulfills the property $f(a_k) \cdot f(b_k) \leq 0 \forall k$.

Algorithm: Bisection Method

in: $f(x)$, $[a, b]$, tolerance t

out: x , an approximation for x^*

```
while |b-a| > t
  c = (a+b)/2
  if f(a)*f(c) <= 0
    keep a, b = c
  else
    keep b, a = c
  end if
end while
x = (a+b)/2
```

When applying the bisection method, we only require continuity of the function $f(x)$ and an initial knowledge of two points a_0 and b_0 such that $f(a_0) \cdot f(b_0) \leq 0$. We are guaranteed the existence of a root in the interval $[a_0, b_0]$ by the Intermediate Value Theorem (5.1) and are further guaranteed that the bisection method will converge to a solution.

We consider the question of “speed of convergence,” namely given a , b and t , how many steps does it take to reach t ? If we suppose that $x^* = \lim_{k \rightarrow \infty} x_k$ then at each iteration the interval containing x^* is halved. Thus, assuming it takes n steps to fulfill $|b - a| \leq t$ we have that

$$\begin{aligned} 2^{-n}|b - a| &\leq t \\ \Rightarrow n \log 2 &\geq \log \left(\frac{|b-a|}{t} \right) \\ \Rightarrow n &\geq \frac{1}{\log 2} \log \left(\frac{|b-a|}{t} \right). \end{aligned}$$

Thus we conclude that for a given tolerance t and initial interval $[a, b]$, bisection will take

$$n \geq \frac{1}{\log 2} \log \left(\frac{|b - a|}{t} \right) \quad (5.2.3)$$

steps to converge.

Example 5.2 Given $|b - a| = 1$ and $t = 10^{-6}$, how many steps does it take to converge? From (5.2.3) we have that

$$\begin{aligned} n &\geq \frac{1}{\log_{10}(2)} \log_{10}(10^6) \\ \Rightarrow n &\geq 3.4 \cdot 6 \\ \Rightarrow n &\geq 20. \end{aligned}$$

(compare with $2^{20} \approx 1.05 \times 10^6$).

5.2.2 Fixed Point Iteration

We may rewrite the root-finding problem in an alternate way that may not be immediately obvious. Consider the real-valued function g , defined by $g(x) = x - f(x)$. We note that this function inherits the continuity of f in an interval $[a, b]$. We can also write $f(x) = x - g(x)$ in order to obtain our original function $f(x)$. The problem of root-finding for our original function is hence equivalent to the problem of finding a solution to $g(x) = x$.

Definition 5.2 We say that x^* is a **fixed point** of $g(x)$ if $g(x^*) = x^*$, i.e., if x^* is mapped to itself under g .

If our function g has certain desirable properties (in particular, as will be shown later, if $|g'(x^*)| < 1$ and x_0 is “close enough” to x^*), then repeated application of g will actually cause us to converge to this fixed point. This implies we can write our algorithm for fixed-point iteration as follows:

Algorithm: Fixed Point Iteration**in:** $g(x)$, x_0 , tolerance t **out:** x , an approximation for x^*

```
i = 0
repeat
    i = i + 1
    x[i] = g(x[i-1])
until |x[i] - x[i-1]| < t
x = x[i]
```

We note that it is not required that we limit ourselves to the choice of $g(x) = x - f(x)$ in applying this scheme. In general we can write $g(x) = x - H(f(x))$ as long as we choose H such that $H(0) = 0$. Not all choices will lead to a converging method. For convergence we must also choose H such that $|g'(x^*)| < 1$ (more on this later).

5.2.3 Newton's Method

Certain additional information may be available about our function that may assist in constructing a root-finding method. In particular, knowledge of the first derivative of $f(x)$ motivates the use of Newton's method. Consider the Taylor series expansion of $f(x^*)$ about an initial estimate x_0 :

$$f(x^*) = f(x_0) + f'(x_0)(x^* - x_0) + O((x^* - x_0)^2). \quad (5.2.4)$$

If we take this sequence to leading order, we have that

$$f(x^*) \approx f(x_0) + f'(x_0)(x^* - x_0). \quad (5.2.5)$$

But we know that $f(x^*) = 0$ and so we find a new, often better approximation x_1 from x_0 by requiring that

$$f(x_0) + f'(x_0)(x_1 - x_0) = 0. \quad (5.2.6)$$

Rearranging and taking the sequence in general yields the defining equation for Newton's method:

$$x_{i+1} = x_i - f(x_i)/f'(x_i). \quad (5.2.7)$$

We note that we will need to look out for the case where $f'(x_i) = 0$, since this will lead to a division by zero. Otherwise, this derivation allows us to provide an algorithm for applying Newton's method numerically:

Algorithm: Newton's Method**in:** $f(x)$, $f'(x)$, x_0 , tolerance t **out:** x , an approximation for x^*

```

i = 0
x[0] = x0
repeat
    i = i + 1
    if f'(x[i-1]) == 0 stop
    x[i] = x[i-1] - f(x[i-1]) / f'(x[i-1])
until |x[i] - x[i-1]| < t
x = x[i]

```

5.2.4 Secant Method

Newton's method provides very fast convergence, but relies on the knowledge of $f'(x)$. If this derivative is not known, not easily computable, or if $f(x)$ is not explicitly given but is the output of a computer program, then we will be unable to apply Newton's method. However, we can approximate the derivative using a numerical scheme that requires only evaluations of the function $f(x)$. From the definition of the derivative we know that

$$f'(x_i) = \lim_{\eta \rightarrow x_i} \frac{f(x_i) - f(\eta)}{x_i - \eta} \approx \frac{f(x_i) - f(\eta)}{x_i - \eta}. \quad (5.2.8)$$

If we choose $\eta = x_{i-1}$ then we approximate the derivative as

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}. \quad (5.2.9)$$

This result can be plugged into Newton's method to give the defining equation for the Secant method:

$$x_{i+1} = x_i - f(x_i) \left[\frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})} \right]. \quad (5.2.10)$$

Note that this method actually requires the two previous values (x_i and x_{i-1}) in order to compute x_{i+1} . Thus, we also need two initial values x_0 and x_1 in order to begin iteration. Also, as in Newton's method where we needed to check for $f'(x_i) = 0$ here we need to be wary of the case where $f(x_i) \approx f(x_{i-1})$, as this will potentially give undesirable results.

5.2.5 Stopping Criteria for Iterative Functions

For any iterative algorithm that approximates a root x^* we need to consider a stopping criterion to decide when to halt the iteration process. There are several general criteria for stopping, which can be combined if necessary.

1. **Maximum number of steps.** Using this method for stopping, we impose some maximum number of steps in the iteration i_{max} and stop when $i = i_{max}$. This provides a safeguard against infinite loops, but is not very efficient — *i.e.*, even if we are very close to the root after 1 iteration, this method will always run for the same number of iterations.

2. **Tolerance on the size of the correction.** Under this criterion, we are given some tolerance t and stop when $|x_{i+1} - x_i| \leq t$. Under the bisection method and fixed-point iteration (with “spiral-wise” convergence, see later) we actually are able to guarantee that $|x_{i+1} - x^*| \leq t$. Unfortunately, this criterion does not guarantee that $|x_{i+1} - x^*| \approx t$, in general.

This may not work very well if one desires a small function value in the approximate root for steep functions such as $f(x) = a(x - x^*)$ with $a = 10^{11}$. Even a small error in x will mean a large value for $f(x)$.

3. **Tolerance on the size of the function value.** Under this criterion, we are given some tolerance t and stop when $|f(x_i)| < t$. This may not work well for a flat function such as $f(x) = a(x - x^*)$ with $a = 10^{-9}$. In this case, for x_i far from x^* , $|f(x_i)|$ may be smaller than t .

In conclusion, choosing a good stopping criterion is difficult and dependent on the problem. Often trial and error is used to determine a good criterion which may combine any of the aforementioned options.

5.3 Rate of Convergence

We wish to examine how quickly each of the four methods discussed here converges to the root x^* , assuming that convergence occurs. In section 5.4 we will discuss the criteria necessary for convergence of each method.

In order to lay a foundation for this discussion, we must consider how rate of convergence is measured. We first consider the error at each step of the iteration:

Definition 5.3 For a sequence $\{x_i\}_{i=0}^{\infty}$ and point x^* , the error at iteration i is

$$e_i = x_i - x^*. \quad (5.3.1)$$

We will define the rate of convergence by how quickly the error converges to zero (and hence how quickly $\{x_i\}_{i=0}^{\infty}$ converges to x^*). If $\{x_i\}_{i=0}^{\infty}$ diverges from x^* , then we note that $\lim_{i \rightarrow \infty} e_i = \pm\infty$.

Definition 5.4 The sequence $\{x_i\}_{i=1}^{\infty}$ converges to x^* with order q if and only if $\{x_i\}_{i=1}^{\infty}$ converges to x^* , $\lim_{i \rightarrow \infty} c_i = N$ when $N \in [0, \infty)$ and

$$|e_{i+1}| = c_i |e_i|^q. \quad (5.3.2)$$

With these definitions in mind, we may consider the rate of convergence of each of our iteration methods. Consider the example given in Table 2.1 and Table 2.2. We wish to determine the positive root of $f(x) = x^2 - \frac{1}{2}\exp(-x)$ which has the exact value $x^* = 0.53983527690282$. We measure the value of the iterate x_i in Table 2.1 and the value of the error e_i in Table 2.2.

Bisection Method. From the derivation (5.2.3) we note that, on average, $|e_{i+1}| \approx \frac{1}{2}|e_i|$. But the error may increase for certain iterations, depending on the initial interval. Thus, we cannot directly apply the definition for convergence to the bisection method, but we nonetheless say that the method behaves like a linearly convergent method, with the following justification:

Consider the sequence defined by $\{L_i\}_{i=1}^{\infty}$ with $L_i = |b_i - a_i|$ the length of the interval at step i . We know that $L_{i+1} = \frac{1}{2}L_i$ and so the sequence $\{L_i\}$ converges to 0 linearly. We also know that $|e_i| \leq L_i$, and so we say that $\{e_i\}$ converges to 0 at least linearly.

Fixed Point Iteration. The rate of convergence for this method is highly variable and depends greatly on the actual problem being solved. In the example at hand, we find that if we define c_i by $|e_{i+1}| = c_i |e_i|$ then, on average, it appears that $\lim_{i \rightarrow \infty} c_i = 0.37$. Thus we note that fixed point iteration converges linearly as well.

Newton's Method. A thorough analysis of Newton's method indicates that Newton actually converges much faster than the other methods. In the example at hand, if we consider $|e_{i+1}| = c_i |e_i|^2$ we will find that we get $\lim_{i \rightarrow \infty} c_i = 0.62$. Thus Newton's method converges quadratically in this example.

Secant Method. With a thorough analysis of the Secant method, we find that the Secant method converges faster than fixed point iteration, but slower than Newton's method. If we consider $|e_{i+1}| = c_i |e_i|^q$ we actually find that $q = \frac{1}{2}(1 + \sqrt{5}) \approx 1.618$. In the example at hand, we actually get that $\lim_{i \rightarrow \infty} c_i \approx 0.74$.

5.4 Convergence Theory

5.4.1 Fixed Point Iteration

For fixed point iteration, we construct the iteration sequence $x_{i+1} = g(x_i)$ and iterate to approximate the fixed point $x^* = g(x^*)$. We demonstrated previously that this is equivalent to root-finding if we use $g(x) = x - H(f(x))$ for any function $H(x) = H(f(x))$ where $H(0) = 0$. Since $f(x^*) = 0$ by the definition of a root, we also have that $H(x^*) = H(f(x^*)) = H(0) = 0$ and so $g(x^*) = x^*$. We will show now that this method converges when $|g'(x^*)| < 1$.

The theory for the fixed point iteration method goes hand in hand with the theory behind contractions and contraction mapping from real analysis:

Definition 5.5 Suppose that g is a real-valued function, defined and continuous on a bounded closed interval $[a, b]$ of the real line. Then, g is said to be a **contraction** on $[a, b]$ if there exists a constant $L \in (0, 1)$ such that

$$|g(x) - g(y)| \leq L|x - y| \quad \forall x, y \in [a, b] \quad (5.4.1)$$

The definition of a contraction has two very important graphical interpretations:

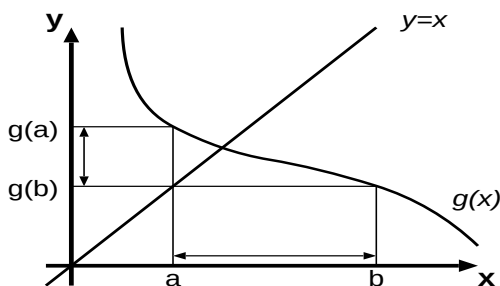


Table 2.1: Root-Finding Iteration x_i ($f(x) = x^2 - \frac{1}{2} \exp(-x)$)

Bisection	Fixed Point Iteration	Newton	Secant
1.00000000000000	1.00000000000000	2.00000000000000	2.00000000000000
0.50000000000000	0.18393972058572	1.03327097864435	0.00000000000000
0.75000000000000	0.56609887674714	0.63686054270010	0.22561484995794
0.62500000000000	0.52949890451207	0.54511924037555	0.74269471761919
0.56250000000000	0.54357981007437	0.53985256974508	0.49760551690233
0.53125000000000	0.53843372706828	0.53983527708914	0.53494633480233
0.54687500000000	0.54035370380524	0.53983527690282	0.53996743772003
0.53906250000000	0.53964266286324	0.53983527690282	0.53983487323262
0.54296875000000	0.53990672286905	0.53983527690282	0.53983527686958
0.54101562500000	0.53980875946698	0.53983527690282	0.53983527690282
0.54003906250000	0.53984511672844	0.53983527690282	0.53983527690282
0.53955078125000	0.53983162533285	0.53983527690282	0.53983527690282
0.53979492187500	0.53983663196232	0.53983527690282	0.53983527690282
0.53991699218750	0.53983477404859	0.53983527690282	0.53983527690282
0.53985595703125	0.53983546350813	0.53983527690282	0.53983527690282
0.53982543945312	0.53983520765493	0.53983527690282	0.53983527690282
0.53984069824219	0.53983530260020	0.53983527690282	0.53983527690282
0.53983306884766	0.53983526736671	0.53983527690282	0.53983527690282
0.53983688354492	0.53983528044160	0.53983527690282	0.53983527690282
0.53983497619629	0.53983527558960	0.53983527690282	0.53983527690282
0.53983592987061	0.53983527739014	0.53983527690282	0.53983527690282
0.53983545303345	0.53983527672198	0.53983527690282	0.53983527690282
0.53983521461487	0.53983527696993	0.53983527690282	0.53983527690282
0.53983533382416	0.53983527687792	0.53983527690282	0.53983527690282
0.53983527421951	0.53983527691206	0.53983527690282	0.53983527690282
0.53983530402184	0.53983527689939	0.53983527690282	0.53983527690282
0.53983528912067	0.53983527690409	0.53983527690282	0.53983527690282
0.53983528167009	0.53983527690235	0.53983527690282	0.53983527690282
0.53983527794480	0.53983527690300	0.53983527690282	0.53983527690282
0.53983527608216	0.53983527690276	0.53983527690282	0.53983527690282
0.53983527701348	0.53983527690284	0.53983527690282	0.53983527690282
0.53983527654782	0.53983527690281	0.53983527690282	0.53983527690282
0.53983527678065	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527689707	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527695527	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527692617	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527691162	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690434	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690070	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690252	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690343	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690298	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690275	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690286	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690281	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690283	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690282	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690281	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690282	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690282	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690282	0.53983527690282	0.53983527690282	0.53983527690282
0.53983527690282	0.53983527690282	0.53983527690282	0.53983527690282

Table 2.2: Root-Finding Iteration e_i ($f(x) = x^2 - \frac{1}{2} \exp(-x)$)

Bisection	Fixed Point Iteration	Newton	Secant
0.46016472309718	0.46016472309718	1.46016472309718	1.46016472309718
-0.03983527690282	-0.35589555631710	0.49343570174153	-0.53983527690282
0.21016472309718	0.02626359984432	0.09702526579728	-0.31422042694488
0.08516472309718	-0.01033637239075	0.00528396347273	0.20285944071637
0.02266472309718	0.00374453317155	0.00001729284226	-0.04222976000049
-0.00858527690282	-0.00140154983454	0.00000000018632	-0.00488894210049
0.00703972309718	0.00051842690242	0.00000000000000	0.00013216081721
-0.00077277690282	-0.00019261403958	0.00000000000000	-0.00000040367020
0.00313347309718	0.00007144596623	0.00000000000000	-0.00000000003324
0.00118034809718	-0.00002651743584	0.00000000000000	0.00000000000000
0.00020378559718	0.00000983982562	0.00000000000000	0.00000000000000
-0.00028449565282	-0.00000365156997	0.00000000000000	0.00000000000000
-0.00004035502782	0.00000135505950	0.00000000000000	0.00000000000000
0.00008171528468	-0.00000050285423	0.00000000000000	0.00000000000000
0.00002068012843	0.00000018660531	0.00000000000000	0.00000000000000
-0.00000983744969	-0.00000006924789	0.00000000000000	0.00000000000000
0.00000542133937	0.00000002569738	0.00000000000000	0.00000000000000
-0.00000220805516	-0.00000000953611	0.00000000000000	0.00000000000000
0.00000160664210	0.00000000353878	0.00000000000000	0.00000000000000
-0.00000030070653	-0.00000000131322	0.00000000000000	0.00000000000000
0.00000065296779	0.00000000048732	0.00000000000000	0.00000000000000
0.00000017613063	-0.00000000018084	0.00000000000000	0.00000000000000
-0.00000006228795	0.00000000006711	0.00000000000000	0.00000000000000
0.00000005692134	-0.00000000002490	0.00000000000000	0.00000000000000
-0.00000000268331	0.00000000000924	0.00000000000000	0.00000000000000
0.000000002711902	-0.00000000000343	0.00000000000000	0.00000000000000
0.00000001221785	0.000000000000127	0.00000000000000	0.00000000000000
0.00000000476727	-0.00000000000047	0.00000000000000	0.00000000000000
0.00000000104198	0.00000000000018	0.00000000000000	0.00000000000000
-0.00000000082066	-0.00000000000006	0.00000000000000	0.00000000000000
0.00000000011066	0.00000000000002	0.00000000000000	0.00000000000000
-0.00000000035500	-0.00000000000001	0.00000000000000	0.00000000000000
0.00000000012217	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000575	-0.00000000000000	0.00000000000000	0.00000000000000
0.00000000005245	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000002335	-0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000880	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000152	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000212	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000030	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000061	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000016	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000007	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000004	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000001	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000001	0.00000000000000	0.00000000000000	0.00000000000000
0.00000000000000	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000001	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000000	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000000	0.00000000000000	0.00000000000000	0.00000000000000
-0.00000000000000	0.00000000000000	0.00000000000000	0.00000000000000

Intermission: The Golden Ratio

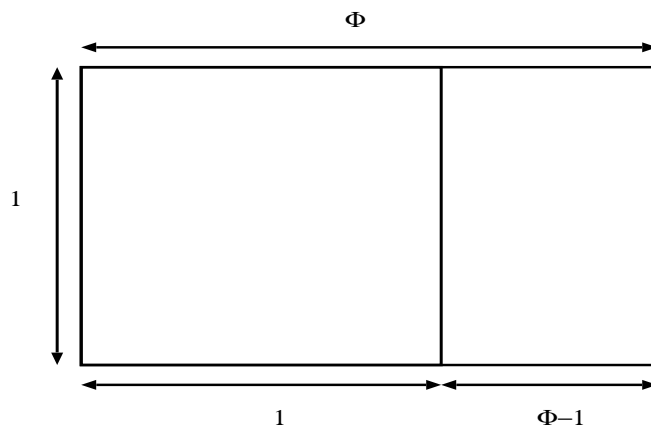


Figure 5.3.1: Two nested rectangles with equivalent aspect ratios

The golden ratio appears often in nature and in mathematics. It can be defined using two nested rectangles that have equivalent aspect ratios, as depicted in figure 5.3.1. The aspect ratio of the larger rectangle is given by $\frac{1}{\phi}$ and the smaller rectangle has the aspect ratio $\frac{\phi-1}{1}$. Equating these gives

$$\frac{1}{\phi} = \frac{1-\phi}{1}, \quad (5.3.3)$$

or equivalently

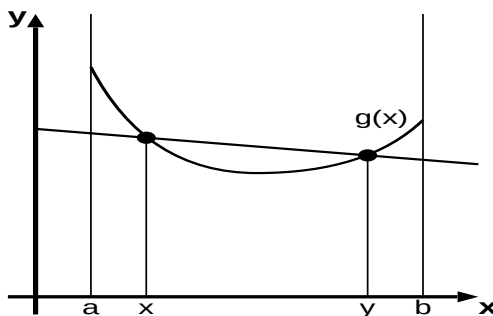
$$\phi^2 - \phi - 1 = 0. \quad (5.3.4)$$

Finally, we apply the quadratic formula and take the positive root to get

$$\phi_{1,2} = \frac{1 + \sqrt{5}}{2}, \quad (5.3.5)$$

the golden ratio.

For one, we notice $|g(a) - g(b)|$ is smaller than $|a - b|$ and so we may say that the interval $[a, b]$ has been contracted to a smaller interval $[g(a), g(b)]$.



Also, we note that for any $x, y \in [a, b]$ such that $x \neq y$ a simple manipulation indicates that a contraction fulfills

$$\frac{|g(x) - g(y)|}{|x - y|} \leq L < 1. \quad (5.4.2)$$

Thus we notice that the slope of any secant line within the interval $[a, b]$ cannot exceed L in absolute value.

An observant reader might notice that this definition of a contraction appears very similar to the definition for a derivative. In fact, if we have $g(x)$ differentiable on $[a, b]$ with $|g'(x)| < 1 \forall x \in [a, b]$, then $g(x)$ is a contraction on $[a, b]$ with

$$L = \max_{x \in [a, b]} |g'(x)|.$$

The proof of this fact is left as an exercise for the reader.

The definition of a contraction leads to a very important theorem that governs the behaviour of the contraction and, in fact, gives us the result we require for fixed point iteration.

Theorem 5.3 (Contraction Mapping Theorem) *Let g be a real-valued function, defined and continuous on a bounded closed interval $[a, b]$ of the real line, and assume that $g(x) \in [a, b]$ for all $x \in [a, b]$. Suppose, further, that g is a contraction on $[a, b]$. Then,*

1. g has a unique fixed point x^* in the interval $[a, b]$.
2. The sequence $\{x_k\}$ defined by $x_{k+1} = g(x_k)$ converges to x^* as $k \rightarrow \infty$ for any starting value x_0 in $[a, b]$.

Proof: Existence of the fixed point. The existence of a fixed point x^* for g is a consequence of the Intermediate Value Theorem. Define $u(x) = x - g(x)$. Then

$$u(a) = a - g(a) \leq 0 \quad \text{and} \quad u(b) = b - g(b) \geq 0.$$

Then by the Intermediate Value Theorem, there exists $x^* \in [a, b]$ such that $u(x^*) = 0$. Thus $x^* - g(x^*) = 0$, or equivalently $x^* = g(x^*)$ and so x^* is a fixed point of g .

Uniqueness of the fixed point. The uniqueness of this fixed point follows from (5.4.1) by contradiction. Suppose that g has a second fixed point, x_2^* , in $[a, b]$ such that $g(x^*) = x^*$ and $g(x_2^*) = x_2^*$. Then,

$$|g(x^*) - g(x_2^*)| \leq L|x^* - x_2^*|, \quad (\text{contraction property}).$$

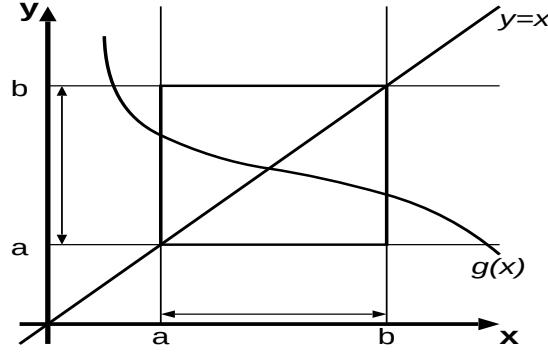


Figure 5.4.1: An illustration of the contraction mapping theorem.

Using the definition of a fixed point, we have

$$|x^* - x_2^*| \leq L|x^* - x_2^*|,$$

or equivalently, $L \geq 1$. However, from the contraction property we know $L \in (0, 1)$. Thus we have a contradiction and so there is no second fixed point.

Convergence property. Let x_0 be any element of $[a, b]$. Consider the sequence $\{x_i\}$ defined by $x_{i+1} = g(x_i)$, where $x_i \in [a, b]$ implies $x_{i+1} \in [a, b]$. We note for any x_{i-1} in the interval we have, by the contraction property

$$|g(x_{i-1}) - g(x^*)| \leq L|x_{i-1} - x^*|,$$

or equivalently

$$|x_i - x^*| \leq L|x_{i-1} - x^*|.$$

Using the fact that this applies for all i , we may use recursion to get

$$|x_i - x^*| \leq L^i|x_0 - x^*|.$$

We take the limit as $i \rightarrow \infty$ to get

$$\lim_{i \rightarrow \infty} |x_i - x^*| \leq |x_0 - x^*| \lim_{i \rightarrow \infty} L^i.$$

But since we also know that $L \in (0, 1)$ then $\lim_{i \rightarrow \infty} L^i = 0$. Thus our equation reduces to

$$\lim_{i \rightarrow \infty} |x_i - x^*| = 0,$$

or

$$\lim_{i \rightarrow \infty} x_i = x^*. \quad \square$$

From the contraction mapping theorem, we note that it appears that convergence to the fixed point x^* appears to be linear; in particular, we get from the contraction property that $e_i \leq Le_{i-1}$.

Intuitively, only one fixed point is allowed since we require a slope greater than 1 to get multiple fixed points (see Figure 5.4.2).

We can determine when a sequence converges from the following corollary to the Contraction Mapping Theorem:

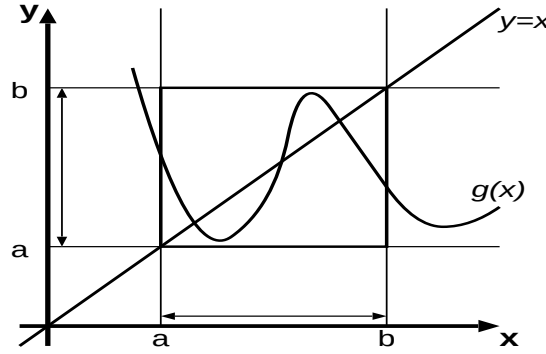


Figure 5.4.2: In order to get multiple fixed points, we need the slope of $g(x)$ to be greater than 1.

Corollary 5.1 *Let g be a real-valued function, defined and continuous on a bounded closed interval $[a, b]$ of the real line, and assume that $g(x) \in [a, b]$ for all $x \in [a, b]$. Let $x^* = g(x^*)$ be a fixed point of $g(x)$ with $x^* \in [a, b]$. Assume there exists δ such that $g'(x)$ is continuous in $I_\delta = [x^* - \delta, x^* + \delta]$. Define the sequence $\{x_i\}_{i=0}^\infty$ by $x_{i+1} = g(x_i)$. Then:*

- I. *If $|g'(x^*)| < 1$ then there exists ϵ such that $\{x_i\}$ converges to x^* for $|x_0 - x^*| < \epsilon$. Further, convergence is linear with $\lim_{i \rightarrow \infty} c_i = |g'(x^*)|$.*
- II. *If $|g'(x^*)| > 1$ then $\{x_i\}$ diverges for any starting value x_0 .*

Using this corollary, we can come up with a method of choosing our form for $g(x)$ in terms of $f(x)$ depending on the derivative at the point x^* .

Example 5.3 *Suppose that we somehow know that $f'(x^*) = 3/2$ where we wish to solve for the root using $f(x) = 0$. Then if we add and subtract x from the equivalent equation $-f(x) = 0$ we get that $x - x - f(x) = 0$. We define $g(x) = x + f(x)$ so we can apply fixed point iteration on $g(x)$ to solve $x = g(x)$. Using this definition of $g(x)$ we get that*

$$|g'(x^*)| = |1 + f'(x^*)| = |1 + 3/2| = 5/2 > 1$$

and so from the corollary we note that we will not have convergence.

If we instead add and subtract x from $f(x) = 0$ we get that $x - x + f(x) = 0$. We define $g(x) = x - f(x)$ so we can apply fixed point iteration on $g(x)$ as before. However, with this definition of $g(x)$ we get that

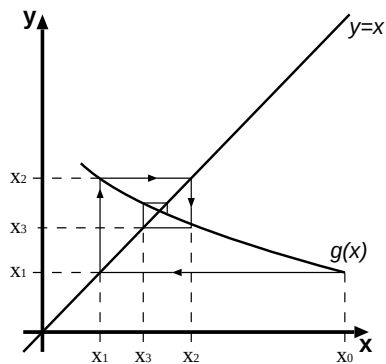
$$|g'(x^*)| = |1 - f'(x^*)| = |1 - 3/2| = 1/2 < 1$$

and so from the corollary we can choose some x_0 close enough to x^ to get convergence.*

Convergence Behaviour of the Fixed Point Method

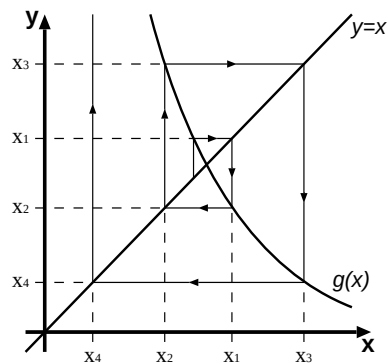
There exist four different types of behaviour when considering convergence in fixed point iteration.

Spiral convergence



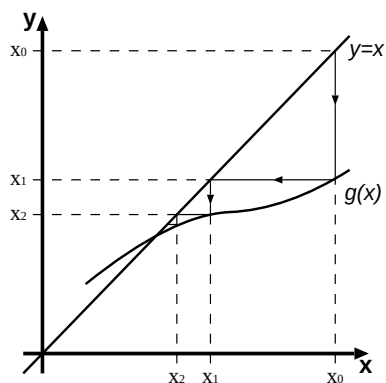
$$-1 < g'(x^*) < 0$$

Spiral divergence



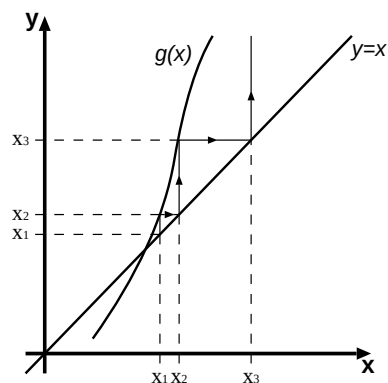
$$g'(x^*) < -1$$

Staircase convergence



$$0 < g'(x^*) < 1$$

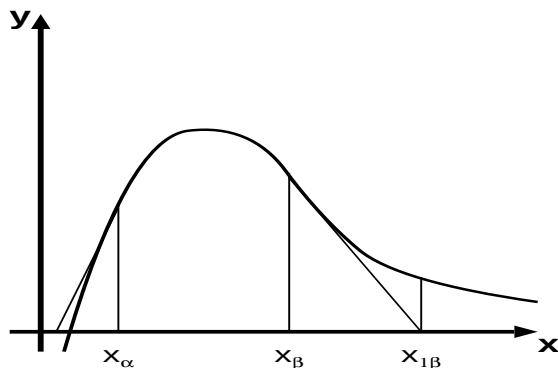
Staircase divergence



$$g'(x^*) > 1$$

5.4.2 Newton's Method

We need to use special care when applying Newton's method. In particular, if x_0 is too far away from x^* we may not get convergence. Consider the following example:



If $x_0 = x_\alpha$ then we will achieve convergence. However, if $x_0 = x_\beta$ we will diverge since the function is monotone decreasing for $x \rightarrow \infty$ but remains positive.

The convergence theorem for Newton's method is not immediately obvious, but allows us to predict cases where we will achieve convergence:

Theorem 5.4 Convergence Theorem for Newton's Method (Atkinson p.60) If $f(x^*) = 0$, $f'(x^*) \neq 0$ and f , f' and f'' are all continuous in $I_\delta = [x^* - \delta, x^* + \delta]$ with x_0 sufficiently close to x^* then the sequence $\{x_i\}_{i=0}^\infty$ converges quadratically to x^* with

$$\lim_{i \rightarrow \infty} c_i = \frac{|f''(x^*)|}{|2f'(x^*)|}. \quad (5.4.3)$$

Note that in this case, the defining equation for c_i is $|x_{i-1} - x^*| = c_i |x_i - x^*|^2$. We note that if $f'(x^*) = 0$ then the rate of convergence degrades to linear convergence. If the conditions for Newton's method are not met, we may not be able to achieve convergence for any starting value x_0 .

5.4.3 Secant Method

The secant method has a very similar convergence theorem to that of Newton's method. In fact, all that changes is the order of convergence of the method. Similar to Newton's method, it can be shown that if $f'(x^*) = 0$ then the rate of convergence degrades to linear.

Theorem 5.5 Convergence Theorem for Secant Method (Atkinson p.67) If $f(x^*) = 0$, $f'(x^*) \neq 0$ and f , f' and f'' are all continuous in $I_\delta = [x^* - \delta, x^* + \delta]$ with x_0 sufficiently close to x^* then the sequence $\{x_i\}_{i=0}^\infty$ converges with order $q = \frac{1}{2}(1 + \sqrt{5})$.

5.4.4 Overview

Each of the root-finding methods has their own strengths and weaknesses that make them particularly useful for some situations and inapplicable in others. The following two tables provide a summary of the functionality of each root-finding method:

Method	Does the method converge?
Bisection	yes, guaranteed
Fixed-point	not always; depending on $g(x)$ and x_0
Newton	not always; depending on $f(x)$ and x_0
Secant	not always; depending on $f(x)$, x_0 and x_1

Method	Speed of convergence	Require knowledge of f'
Bisection	slow (linear)	no
Fixed-point	slow (linear)	no
Newton	fast (quadratic)	yes
Secant	fast ($q \approx \frac{1}{2}(1 + \sqrt{5})$)	no

In practice, MATLAB uses a combination of the Secant method, Bisection method and a method not discussed here called Inverse Quadratic Interpolation (the combined method is accessible through the **fzero** command). It uses the method that converges fastest if possible, defaulting to the guaranteed convergence of Bisection if necessary. Further, it requires no knowledge of the derivative. This approach allows MATLAB's general root-finding function **fzero** to be well-suited to a variety of applications.

Chapter 6

The Risk Neutral World

Suppose instead of valuing an option using the above no-arbitrage argument, we wanted to know the expected value of the option. We can imagine that we are buying and holding the option, and not hedging. If we are considering the value of risky cash flows in the future, then these cash flows should be discounted at an appropriate discount rate, which we will call ρ (i.e. the riskier the cash flows, the higher ρ).

Consequently the value of an option today can be considered to be the discounted future value. This is simply the old idea of net present value. Regard S today as known, and let $V(S + dS, t + dt)$ be the value of the option at some future time $t + dt$, which is uncertain, since S evolves randomly. Thus

$$V(S, t) = \frac{1}{1 + \rho dt} E(V(S + dS, t + dt)) \quad (6.0.1)$$

where $E(\dots)$ is the expectation operator, i.e. the expected value of $V(S + dS, t + dt)$ given that $V = V(S, t)$ at $t = t$. We can rewrite equation (6.0.1) as (ignoring terms of $o(dt)$, where $o(dt)$ represents terms that go to zero faster than dt)

$$\rho dt V(S, t) = E(V(S, t) + dV) - V(S, t) . \quad (6.0.2)$$

Since we regard V as the expected value, so that $E[V(S, t)] = V(S, t)$, and then

$$E(V(S, t) + dV) - V(S, t) = E(dV) , \quad (6.0.3)$$

so that equation (6.0.2) becomes

$$\rho dt V(S, t) = E(dV) . \quad (6.0.4)$$

Assume that

$$\frac{dS}{S} = \mu dt + \sigma dZ . \quad (6.0.5)$$

From Ito's Lemma (1.7.7) we have that

$$dV = \left(V_t + \frac{\sigma^2 S^2}{2} V_{SS} + \mu S V_S \right) dt + \sigma S V_S dZ . \quad (6.0.6)$$

Noting that

$$E(dZ) = 0 \quad (6.0.7)$$

then

$$E(dV) = \left(V_t + \frac{\sigma^2 S^2}{2} V_{SS} + \mu S V_S \right) dt . \quad (6.0.8)$$

Combining equations (6.0.4-6.0.8) gives

$$V_t + \frac{\sigma^2 S^2}{2} V_{SS} + \mu S V_S - \rho V = 0 . \quad (6.0.9)$$

Equation (6.0.9) is the PDE for the *expected value* of an option. If we are not hedging, maybe this is the value that we are interested in, not the no-arbitrage value. However, if this is the case, we have to estimate the drift rate μ , and the discount rate ρ . Estimating the appropriate discount rate is always a thorny issue.

Now, note the interesting fact, if we set $\rho = r$ and $\mu = r$ in equation (6.0.9) then we simply get the Black-Scholes equation (1.8.9).

This means that the no-arbitrage price of an option is identical to the expected value if $\rho = r$ and $\mu = r$. In other words, we can determine the no-arbitrage price by pretending we are living in a world where all assets drift at rate r , and all investments are discounted at rate r . This is the so-called *risk neutral* world.

This result is the source of endless confusion. It is best to think of this as simply a mathematical fluke. This does not have any reality. Investors would be very stupid to think that the drift rate of risky investments is r . I'd rather just buy risk-free bonds in this case. There is in reality no such thing as a risk-neutral world. Nevertheless, this result is useful for determining the no-arbitrage value of an option using a Monte Carlo approach. Using this numerical method, we simply assume that

$$dS = rSdt + \sigma SdZ \quad (6.0.10)$$

and simulate a large number of random paths. If we know the option payoff as a function of S at $t = T$, then we compute

$$V(S, 0) = e^{-rT} E_Q(V(S, T)) \quad (6.0.11)$$

which should be the no-arbitrage value.

Note the E_Q in the above equation. This makes it clear that we are taking the expectation in the *risk neutral* world (the expectation in the Q measure). This contrasts with the real-world expectation (the P measure).

Suppose we want to know the expected value (in the real world) of an asset which pays $V(S, t = T)$ at $t = T$ in the future. Then, the expected value (today) is given by solving

$$V_t + \frac{\sigma^2 S^2}{2} V_{SS} + \mu S V_S = 0 . \quad (6.0.12)$$

where we have dropped the discounting term. In particular, suppose we are going to receive $V = S(t = T)$, i.e. just the asset at $t = T$. Assume that the solution to equation (6.0.12) is $V = \text{Const. } A(t)S$, and we find that

$$V = \text{Const. } S e^{\mu(T-t)} . \quad (6.0.13)$$

Noting that we receive $V = S$ at $t = T$ means that

$$V = S e^{\mu(T-t)} . \quad (6.0.14)$$

Today, we can acquire the asset for price $S(t = 0)$. At $t = T$, the asset is worth $S(t = T)$. Equation (6.0.14) then says that

$$E[V(S(t = 0), t = 0)] = E[S(t = 0)] = S(t = 0) e^{\mu(T)} \quad (6.0.15)$$

In other words, if

$$dS = S\mu dt + S\sigma dZ \quad (6.0.16)$$

then (setting $t = T$)

$$E[S] = S e^{\mu T} . \quad (6.0.17)$$

Recall that the exact solution to equation (6.0.16) is (see equation (1.7.26))

$$S(t) = S(0) \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma (Z(t) - Z(0)) \right] . \quad (6.0.18)$$

So that we have just shown that $E[S] = Se^{\mu t}$ by using a simple PDE argument and Ito's Lemma. Isn't this easier than using brute force statistics? PDEs are often much more elegant.

Chapter 7

Monte Carlo Methods

This brings us to the simplest numerical method for computing the no-arbitrage value of an option. Suppose that we assume that the underlying process is

$$\frac{dS}{S} = rdt + \sigma dZ \quad (7.0.1)$$

then we can simulate a path forward in time, starting at some price today S^0 , using a forward Euler timestepping method ($S^i = S(t_i)$)

$$S^{i+1} = S^i + S^i(r\Delta t + \sigma\phi^i\sqrt{\Delta t}) \quad (7.0.2)$$

where Δt is the finite timestep, and ϕ^i is a random number which is $N(0, 1)$. Note that at each timestep, we generate a new random number. After N steps, with $T = N\Delta t$, we have a single realized path. Given the payoff function of the option, the value for this path would be

$$Value = Payoff(S^N) . \quad (7.0.3)$$

For example, if the option was a European call, then

$$\begin{aligned} Value &= \max(S^N - K, 0) \\ K &= \text{Strike Price} \end{aligned} \quad (7.0.4)$$

Suppose we run a series of trials, $m = 1, \dots, M$, and denote the payoff after the m' th trial as $payoff(m)$. Then, the no-arbitrage value of the option is

$$\begin{aligned} Option Value &= e^{-rT} E(payoff) \\ &\simeq e^{-rT} \frac{1}{M} \sum_{m=1}^{m=M} payoff(m) . \end{aligned} \quad (7.0.5)$$

Recall that these paths are not the real paths, but are the risk neutral paths.

Now, we should remember that we are

1. approximating the solution to the SDE by forward Euler, which has $O(\Delta t)$ truncation error.
2. approximating the expectation by the mean of many random paths. This Monte Carlo error is of size $O(1/\sqrt{M})$, which is slowly converging.

There are thus two sources of error in the Monte Carlo approach: timestepping error and sampling error.

The slow rate of convergence of Monte Carlo methods makes these techniques unattractive except when the option is written on several (i.e. more than three) underlying assets. As well, since we are simulating

forward in time, we cannot know at a given point in the forward path if it is optimal to exercise or hold an American style option. This is easy if we use a PDE method, since we solve the PDE backwards in time, so we always know the *continuation value* and hence can act optimally. However, if we have more than three factors, PDE methods become very expensive computationally. As well, if we want to determine the effects of discrete hedging, for example, a Monte Carlo approach is very easy to implement.

The error in the Monte Carlo method is then

$$\begin{aligned} \text{Error} &= O\left(\max(\Delta t, \frac{1}{\sqrt{M}})\right) \\ \Delta t &= \text{timestep} \\ M &= \text{number of Monte Carlo paths} \end{aligned} \quad (7.0.6)$$

Now, it doesn't make sense to drive the Monte Carlo error down to zero if there is $O(\Delta t)$ timestepping error. We should seek to balance the timestepping error and the sampling error. In order to make these two errors the same order, we should choose $M = O(\frac{1}{(\Delta t)^2})$. This makes the total error $O(\Delta t)$. We also have that

$$\begin{aligned} \text{Complexity} &= O\left(\frac{M}{\Delta t}\right) \\ &= O\left(\frac{1}{(\Delta t)^3}\right) \\ \Delta t &= O\left((\text{Complexity})^{-1/3}\right) \end{aligned} \quad (7.0.7)$$

and hence

$$\text{Error} = O\left(\frac{1}{(\text{Complexity})^{1/3}}\right). \quad (7.0.8)$$

In practice, the convergence in terms of timestep error is often not done. People just pick a timestep, i.e. one day, and increase the number of Monte Carlo samples until they achieve convergence in terms of sampling error, and ignore the timestep error. Sometimes this gives bad results!

Note that the exact solution to Geometric Brownian motion (1.7.26) has the property that the asset value S can never reach $S = 0$ if $S(0) > 0$, in any finite time. However, due to the approximate nature of our Forward Euler method for solving the SDE, it is possible that a negative or zero S^i can show up. We can do one of three things here, in this case

- Cut back the timestep at this point in the simulation so that S is positive.
- Set $S = 0$ and continue. In this case, S remains zero for the rest of this particular simulation.
- Use Ito's Lemma, and determine the SDE for $\log S$, i.e. if $F = \log S$, then, from equation (1.7.24), we obtain (with $\mu = r$)

$$dF = \left(r - \frac{\sigma^2}{2}\right)dt + \sigma dZ, \quad (7.0.9)$$

so that now, if $F < 0$, there is no problem, since $S = e^F$, and if $F < 0$, this just means that S is very small. We can use this idea for any stochastic process where the variable should not go negative.

Usually, most people set $S = 0$ and continue. As long as the timestep is not too large, this situation is probably due to an event of low probability, hence any errors incurred will not affect the expected value very much. If negative S values show up many times, this is a signal that the timestep is too large.

In the case of simple Geometric Brownian motion, where r, σ are constants, then the SDE can be solved exactly, and we can avoid timestepping errors (see Section 1.7.2). In this case

$$S(T) = S(0) \exp\left[\left(r - \frac{\sigma^2}{2}\right)T + \sigma\phi\sqrt{T}\right] \quad (7.0.10)$$

where $\phi \sim N(0, 1)$. I'll remind you that equation (7.0.10) is exact. For these simple cases, we should always use equation (7.0.10). Unfortunately, this does not work in more realistic situations.

Monte Carlo is popular because

- It is simple to code. Easily handles complex path dependence.
- Easily handles multiple assets.

The disadvantages of Monte Carlo methods are

- It is difficult to apply this idea to problems involving optimal decision making (e.g. American options).
- It is hard to compute the Greeks (V_S, V_{SS}), which are the hedging parameters, very accurately.
- MC converges slowly.

7.1 Monte Carlo Error Estimators

The sampling error can be estimated via a statistical approach. If the estimated mean of the sample is

$$\hat{\mu} = \frac{e^{-rT}}{M} \sum_{m=1}^{m=M} \text{payoff}(m) \quad (7.1.1)$$

and the standard deviation of the estimate is

$$\omega = \left(\frac{1}{M-1} \sum_{m=1}^{m=M} (e^{-rT} \text{payoff}(m) - \hat{\mu})^2 \right)^{1/2} \quad (7.1.2)$$

then the 95% confidence interval for the actual value V of the option is

$$\hat{\mu} - \frac{1.96\omega}{\sqrt{M}} < V < \hat{\mu} + \frac{1.96\omega}{\sqrt{M}} \quad (7.1.3)$$

Note that in order to reduce this error by a factor of 10, the number of simulations must be increased by 100.

The timestep error can be estimated by running the problem with different size timesteps, comparing the solutions.

7.2 Random Numbers and Monte Carlo

There are many good algorithms for generating random sequences which are uniformly distributed in $[0, 1]$. See for example, (*Numerical Recipes in C++*, Press et al, Cambridge University Press, 2002). As pointed out in this book, often the system supplied random number generators, such as *rand* in the standard C library, and the infamous *RANDU* IBM function, are extremely bad. The MATLAB functions appear to be quite good. For more details, please look at (*Park and Miller*, ACM Transactions on Mathematical Software, 31 (1988) 1192-1201). Another good generator is described in (Matsumoto and Nishimura, “*The Mersenne Twister: a 623 dimensionally equidistributed uniform pseudorandom number generator*,” ACM Transactions on Modelling and Computer Simulation, 8 (1998) 3-30.) Code can be downloaded from the authors Web site.

However, we need random numbers which are normally distributed on $[-\infty, +\infty]$, with mean zero and variance one ($N(0, 1)$).

Suppose we have uniformly distributed numbers on $[0, 1]$, i.e., the probability of obtaining a number between x and $x + dx$ is

$$\begin{aligned} p(x)dx &= dx \quad ; \quad 0 \leq x \leq 1 \\ &= 0 \quad ; \quad \text{otherwise} \end{aligned} \quad (7.2.1)$$

Let's take a function of this random variable $y(x)$. How is $y(x)$ distributed? Let $\hat{p}(y)$ be the probability distribution of obtaining y in $[y, y + dy]$. Consequently, we must have (recall the law of transformation of probabilities)

$$p(x)|dx| = \hat{p}(y)|dy|$$

or

$$\hat{p}(y) = p(x) \left| \frac{dx}{dy} \right|. \quad (7.2.2)$$

Suppose we want $\hat{p}(y)$ to be normal,

$$\hat{p}(y) = \frac{e^{-y^2/2}}{\sqrt{2\pi}}. \quad (7.2.3)$$

If we start with a uniform distribution, $p(x) = 1$ on $[0, 1]$, then from equations (7.2.2-7.2.3) we obtain

$$\frac{dx}{dy} = \frac{e^{-y^2/2}}{\sqrt{2\pi}}. \quad (7.2.4)$$

Now, for $x \in [0, 1]$, we have that the probability of obtaining a number in $[0, x]$ is

$$\int_0^x dx' = x, \quad (7.2.5)$$

but from equation (7.2.4) we have

$$dx' = \frac{e^{-(y')^2/2}}{\sqrt{2\pi}} dy'. \quad (7.2.6)$$

So, there exists a y such that the probability of getting a y' in $[-\infty, y]$ is equal to the probability of getting x' in $[0, x]$,

$$\int_0^x dx' = \int_{-\infty}^y \frac{e^{-(y')^2/2}}{\sqrt{2\pi}} dy', \quad (7.2.7)$$

or

$$x = \int_{-\infty}^y \frac{e^{-(y')^2/2}}{\sqrt{2\pi}} dy'. \quad (7.2.8)$$

So, if we generate uniformly distributed numbers x on $[0, 1]$, then to determine y which are $N(0, 1)$, we do the following

- Generate x
- Find y such that

$$x = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^y e^{-(y')^2/2} dy'. \quad (7.2.9)$$

We can write this last step as

$$y = F(x) \quad (7.2.10)$$

where $F(x)$ is the inverse cumulative normal distribution.

7.3 The Box-Muller Algorithm

Starting from random numbers which are uniformly distributed on $[0, 1]$, there is actually a simpler method for obtaining random numbers which are normally distributed.

If $p(x)$ is the probability of finding $x \in [x, x + dx]$ and if $y = y(x)$, and $\hat{p}(y)$ is the probability of finding $y \in [y, y + dy]$, then, from equation (7.2.2) we have

$$|p(x)dx| = |\hat{p}(y)dy| \quad (7.3.1)$$

or

$$\hat{p}(y) = p(x) \left| \frac{dx}{dy} \right|. \quad (7.3.2)$$

Now, suppose we have two original random variables x_1, x_2 , and let $p(x_1, x_2)$ be the probability of obtaining (x_1, x_2) in $[x_1, x_1 + dx_1] \times [x_2, x_2 + dx_2]$. Then, if

$$\begin{aligned} y_1 &= y_1(x_1, x_2) \\ y_2 &= y_2(x_1, x_2) \end{aligned} \quad (7.3.3)$$

and we have that

$$\hat{p}(y_1, y_2) = p(x_1, x_2) \left| \frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} \right| \quad (7.3.4)$$

where the Jacobian of the transformation is defined as

$$\frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} = \det \begin{vmatrix} \frac{\partial x_1}{\partial y_1} & \frac{\partial x_1}{\partial y_2} \\ \frac{\partial x_2}{\partial y_1} & \frac{\partial x_2}{\partial y_2} \end{vmatrix} \quad (7.3.5)$$

Recall that the Jacobian of the transformation can be regarded as the scaling factor which transforms $dx_1 dx_2$ to $dy_1 dy_2$, i.e.

$$dx_1 dx_2 = \left| \frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} \right| dy_1 dy_2. \quad (7.3.6)$$

Now, suppose that we have x_1, x_2 uniformly distributed on $[0, 1] \times [0, 1]$, i.e.

$$p(x_1, x_2) = U(x_1)U(x_2) \quad (7.3.7)$$

where

$$\begin{aligned} U(x) &= 1 \quad ; \quad 0 \leq x \leq 1 \\ &= 0 \quad ; \quad \text{otherwise} . \end{aligned} \quad (7.3.8)$$

We denote this distribution as $x_1 \sim U[0, 1]$ and $x_2 \sim U[0, 1]$.

If $p(x_1, x_2)$ is given by equation (7.3.7), then we have from equation (7.3.4) that

$$\hat{p}(y_1, y_2) = \left| \frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} \right| \quad (7.3.9)$$

Now, we want to find a transformation $y_1 = y_1(x_1, x_2), y_2 = y_2(x_1, x_2)$ which results in normal distributions for y_1, y_2 . Consider

$$\begin{aligned} y_1 &= \sqrt{-2 \log x_1} \cos 2\pi x_2 \\ y_2 &= \sqrt{-2 \log x_1} \sin 2\pi x_2 \end{aligned} \quad (7.3.10)$$

or solving for (x_1, x_2)

$$\begin{aligned} x_1 &= \exp\left(\frac{-1}{2}(y_1^2 + y_2^2)\right) \\ x_2 &= \frac{1}{2\pi} \tan^{-1}\left[\frac{y_2}{y_1}\right]. \end{aligned} \quad (7.3.11)$$

After some tedious algebra, we can see that (using equation (7.3.11))

$$\left| \frac{\partial(x_1, x_2)}{\partial(y_1, y_2)} \right| = \frac{1}{\sqrt{2\pi}} e^{-y_1^2/2} \frac{1}{\sqrt{2\pi}} e^{-y_2^2/2} \quad (7.3.12)$$

Now, assuming that equation (7.3.7) holds, then from equations (7.3.9-7.3.12) we have

$$\hat{p}(y_1, y_2) = \frac{1}{\sqrt{2\pi}} e^{-y_1^2/2} \frac{1}{\sqrt{2\pi}} e^{-y_2^2/2} \quad (7.3.13)$$

so that (y_1, y_2) are independent, normally distributed random variables, with mean zero and variance one, or

$$y_1 \sim N(0, 1) \quad ; \quad y_2 \sim N(0, 1). \quad (7.3.14)$$

This gives the following algorithm for generating normally distributed random numbers (given uniformly distributed numbers):

Box Muller Algorithm

$$\begin{aligned} &\text{Repeat} \\ &\quad \text{Generate } u_1 \sim U(0, 1), u_2 \sim U(0, 1) \\ &\quad \theta = 2\pi u_2, \rho = \sqrt{-2 \log u_1} \\ &\quad z_1 = \rho \cos \theta; z_2 = \rho \sin \theta \\ &\text{End Repeat} \end{aligned} \quad (7.3.15)$$

This has the effect that $Z_1 \sim N(0, 1)$ and $Z_2 \sim N(0, 1)$.

Note that we generate two draws from a normal distribution on each pass through the loop.

7.4 An improved Box Muller

The algorithm (7.3.15) can be expensive due to the trigonometric function evaluations. We can use the following method to avoid these evaluations. Let

$$\begin{aligned} U_1 &\sim U[0, 1] \quad ; \quad U_2 \sim U[0, 1] \\ V_1 &= 2U_1 - 1 \quad ; \quad V_2 = 2U_2 - 1 \end{aligned} \quad (7.4.1)$$

which means that (V_1, V_2) are uniformly distributed in $[-1, 1] \times [-1, 1]$. Now, we carry out the following procedure

Rejection Method

$$\begin{aligned} &\text{Repeat} \\ &\quad \text{If } (V_1^2 + V_2^2 < 1) \\ &\quad \quad \text{Accept} \end{aligned}$$

```

Else
    Reject
Endif
End Repeat

```

(7.4.2)

which means that if we define (V_1, V_2) as in equation (7.4.1), and then process the pairs (V_1, V_2) using algorithm (7.4.2) we have that (V_1, V_2) are uniformly distributed on the disk centered at the origin, with radius one, in the (V_1, V_2) plane. This is denoted by

$$(V_1, V_2) \sim D(0, 1) . \quad (7.4.3)$$

If $(V_1, V_2) \sim D(0, 1)$ and $R^2 = V_1^2 + V_2^2$, then the probability of finding R in $[R, R + dR]$ is

$$\begin{aligned} p(R) dR &= \frac{2\pi R dR}{\pi(1)^2} \\ &= 2R dR . \end{aligned} \quad (7.4.4)$$

From the fundamental law of transformation of probabilities, we have that

$$\begin{aligned} p(R^2)d(R^2) &= p(R)dR \\ &= 2R dR \end{aligned} \quad (7.4.5)$$

so that

$$\begin{aligned} p(R^2) &= \frac{2R}{\frac{d(R^2)}{dR}} \\ &= 1 \end{aligned} \quad (7.4.6)$$

so that R^2 is uniformly distributed on $[0, 1]$, ($R^2 \sim U[0, 1]$).

As well, if $\theta = \tan^{-1}(V_2/V_1)$, i.e. θ is the angle between a line from the origin to the point (V_1, V_2) and the V_1 axis, then $\theta \sim U[0, 2\pi]$. Note that

$$\begin{aligned} \cos \theta &= \frac{V_1}{\sqrt{V_1^2 + V_2^2}} \\ \sin \theta &= \frac{V_2}{\sqrt{V_1^2 + V_2^2}} . \end{aligned} \quad (7.4.7)$$

Now in the original Box Muller algorithm (7.3.15),

$$\begin{aligned} \rho &= \sqrt{-2 \log U_1} \quad ; \quad U_1 \sim U[0, 1] \\ \theta &= 2\pi U_2 \quad ; \quad U_2 \sim U[0, 1] , \end{aligned} \quad (7.4.8)$$

but $\theta = \tan^{-1}(V_2/V_1) \sim U[0, 2\pi]$, and $R^2 = U[0, 1]$. Therefore, if we let $W = R^2$, then we can replace θ, ρ in algorithm (7.3.15) by

$$\begin{aligned} \theta &= \tan^{-1} \left(\frac{V_2}{V_1} \right) \\ \rho &= \sqrt{-2 \log W} . \end{aligned} \quad (7.4.9)$$

Now, the last step in the Box Muller algorithm (7.3.15) is

$$\begin{aligned} Z_1 &= \rho \cos \theta \\ Z_2 &= \rho \sin \theta , \end{aligned} \quad (7.4.10)$$

but since $W = R^2 = V_1^2 + V_2^2$, then $\cos \theta = V_1/R$, $\sin \theta = V_2/R$, so that

$$\begin{aligned} Z_1 &= \rho \frac{V_1}{\sqrt{W}} \\ Z_2 &= \rho \frac{V_2}{\sqrt{W}} . \end{aligned} \tag{7.4.11}$$

This leads to the following algorithm

Polar form of Box Muller

Repeat

Generate $U_1 \sim U[0, 1]$, $U_2 \sim U[0, 1]$.

Let

$$\begin{aligned} V_1 &= 2U_1 - 1 \\ V_2 &= 2U_2 - 1 \\ W &= V_1^2 + V_2^2 \end{aligned}$$

If($W < 1$) then

$$\begin{aligned} Z_1 &= V_1 \sqrt{-2 \log W/W} \\ Z_2 &= V_2 \sqrt{-2 \log W/W} \end{aligned} \tag{7.4.12}$$

End If

End Repeat

Consequently, (Z_1, Z_2) are independent (uncorrelated), and $Z_1 \sim N(0, 1)$, and $Z_2 \sim N(0, 1)$. Because of the rejection step (7.4.2), about $(1 - \pi/4)$ of the random draws in $[-1, +1] \times [-1, +1]$ are rejected (about 21%), but this method is still generally more efficient than brute force Box Muller.

7.5 Speeding up Monte Carlo

Monte Carlo methods are slow to converge, since the error is given by

$$\text{Error} = O\left(\frac{1}{\sqrt{M}}\right)$$

where M is the number of samples. There are many methods which can be used to try to speed up convergence. These are usually termed Variance Reduction techniques.

Perhaps the simplest idea is the Antithetic Variable method. Suppose we compute a random asset path

$$S^{i+1} = S^i \mu \Delta t + S^i \sigma \phi^i \sqrt{\Delta t}$$

where ϕ^i are $N(0, 1)$. We store all the $\phi^i, i = 1, \dots$, for a given path. Call the estimate for the option price from this sample path V^+ . Then compute a second sample path where $(\phi^i)' = -\phi^i, i = 1, \dots$. Call this estimate V^- . Then compute the average

$$\bar{V} = \frac{V^+ + V^-}{2} ,$$

and continue sampling in this way. Averaging over all the $\bar{V}$, slightly faster convergence is obtained. Intuitively, we can see that this *symmetrizes* the random paths.

Let X^+ be the option values obtained from all the V^+ simulations, and X^- be the estimates obtained from all the V^- simulations. Note that $Var(X^+) = Var(X^-)$ (they have the same distribution). Then

$$\begin{aligned} Var\left(\frac{X^+ + X^-}{2}\right) &= \frac{1}{4}Var(X^+) + \frac{1}{4}Var(X^-) + \frac{1}{2}Cov(X^+, X^-) \\ &= \frac{1}{2}Var(X^+) + \frac{1}{2}Cov(X^+, X^-) \end{aligned} \quad (7.5.1)$$

which will be smaller than $Var(X^+)$ if $Cov(X^+, X^-)$ is nonpositive. Warning: this is not always the case. For example, if the payoff is not a monotonic function of S , the results may actually be worse than crude Monte Carlo. For example, if the payoff is a capped call

$$\begin{aligned} \text{payoff} &= \min(K_2, \max(S - K_1, 0)) \\ &K_2 > K_1 \end{aligned}$$

then the antithetic method performs poorly.

Note that this method can be used to estimate the mean. In the MC error estimator (7.1.3), compute the standard deviation of the estimator as $\omega = \sqrt{Var(\frac{X^+ + X^-}{2})}$.

However, if we want to estimate the distribution of option prices (i.e. a probability distribution), then we should not average each V^+ and V^- , since this changes the variance of the actual distribution.

If we want to know the actual variance of the distribution (and not just the mean), then to compute the variance of the distribution, we should just use the estimates V^+ , and compute the estimate of the variance in the usual way. This should also be used if we want to plot a histogram of the distribution, or compute the Value at Risk.

7.6 Estimating the mean and variance

An estimate of the mean $\bar{x}$ and variance s_M^2 of M numbers $x_1, x_2, \dots, x_M$ is

$$\begin{aligned} s_M^2 &= \frac{1}{M-1} \sum_{i=1}^M (x_i - \bar{x})^2 \\ \bar{x} &= \frac{1}{M} \sum_{i=1}^M x_i \end{aligned} \quad (7.6.1)$$

Alternatively, one can use

$$s_M^2 = \frac{1}{M-1} \left(\sum_{i=1}^M x_i^2 - \frac{1}{M} \left(\sum_{i=1}^M x_i \right)^2 \right) \quad (7.6.2)$$

which has the advantage that the estimate of the mean and standard deviation can be computed in one loop.

Something else to be aware of, is that doing a simple sum of many numbers is prone to accumulation of roundoff errors. A simple remedy is to do pairwise accumulation of the sum. See, for example, *Accurate floating point summation*, P. Linz, Comm. ACM, vol 13 (1970) 361-362, or *The accuracy of floating point summation*, N. Higham, SIAM J. Sci. Computing, vol 14 (1993) 783-799.

In order to avoid roundoff, the following method is suggested by Seydel (R. Seydel, Tools for Computational Finance, Springer, 2002). Set

$$\alpha_1 = x_1 \quad ; \quad \beta_1 = 0 \quad (7.6.3)$$

then compute recursively

$$\begin{aligned}\alpha_i &= \alpha_{i-1} + \frac{x_i - \alpha_{i-1}}{i} \\ \beta_i &= \beta_{i-1} + \frac{(i-1)(x_i - \alpha_{i-1})^2}{i}\end{aligned}\tag{7.6.4}$$

so that

$$\begin{aligned}\bar{x} &= \alpha_M \\ s_M^2 &= \frac{\beta_M}{M-1}\end{aligned}\tag{7.6.5}$$

7.7 Low Discrepancy Sequences

In an effort to get around the $\frac{1}{\sqrt{M}}$, (M = number of samples) behaviour of Monte Carlo methods, quasi-Monte Carlo methods have been devised.

These techniques use a deterministic sequence of numbers (low discrepancy sequences). The idea here is that a Monte Carlo method does not fill the sample space very evenly (after all, it's random). A low discrepancy sequence tends to sample the space in an orderly fashion. If d is the dimension of the space, then the worst case error bound for an LDS method is

$$\text{Error} = O\left(\frac{(\log M)^d}{M}\right)\tag{7.7.1}$$

where M is the number of samples used. Clearly, if d is small, then this error bound is (at least asymptotically) better than Monte Carlo.

LDS methods generate numbers on $[0, 1]$. We cannot use the Box-Muller method in this case to produce normally distributed numbers, since these numbers are deterministic. We have to invert the cumulative normal distribution in order to get the numbers distributed with mean zero and standard deviation one on $[-\infty, +\infty]$. So, if $F(x)$ is the inverse cumulative normal distribution, then

$$\begin{aligned}x_{LDS} &= \text{uniformly distributed on } [0, 1] \\ y_{LDS} &= F(x_{LDS}) \text{ is } N(0, 1) .\end{aligned}\tag{7.7.2}$$

Another problem has to do with the fact that if we are stepping through time, i.e.

$$\begin{aligned}S^{n+1} &= S^n + S^n(r\Delta t + \phi\sigma\sqrt{\Delta t}) \\ \phi &= N(0, 1)\end{aligned}\tag{7.7.3}$$

with, say, N steps in total, then we need to think of this as a problem in N dimensional space. In other words, the k -th timestep is sampled from the k -th coordinate in this N dimensional space. We are trying to uniformly sample from this N dimensional space.

Let $\hat{x}$ be a vector of LDS numbers on $[0, 1]$, in N dimensional space

$$\hat{x} = \begin{bmatrix} x_1 \\ x_2 \\ | \\ x_N \end{bmatrix} .\tag{7.7.4}$$

So, an LDS algorithm would proceed as follows, for the j' th trial

- Generate $\hat{x}^j$ (the j' th LDS number in an N dimensional space).

- Generate the normally distributed vector $\hat{y}^j$ by inverting the cumulative normal distribution for each component

$$\hat{y}^j = \begin{bmatrix} F(x_1^j) \\ F(x_2^j) \\ \vdots \\ F(x_N^j) \end{bmatrix} \quad (7.7.5)$$

- Generate a complete sample path $k = 0, \dots, N - 1$

$$S_j^{k+1} = S_j^k + S_j^k(r\Delta t + \hat{y}_{k+1}^j \sigma \sqrt{\Delta t}) \quad (7.7.6)$$

- Compute the payoff at $S = S_j^N$

The option value is the average of these trials.

There are a variety of LDS numbers: Halton, Sobol, Niederrieter, etc. Our tests seem to indicate that Sobol is the best.

Note that the worst case error bound for the error is given by equation (7.7.1). If we use a reasonable number of timesteps, say 50 – 100, then, $d = 50 - 100$, which gives a very bad error bound. For d large, the numerator in equation (7.7.1) dominates. The denominator only dominates when

$$M \simeq e^d \quad (7.7.7)$$

which is a very large number of trials for $d \simeq 100$. Fortunately, at least for path-dependent options, we have found that things are not quite this bad, and LDS seems to work if the number of timesteps is less than 100 – 200. However, once the dimensionality gets above a few hundred, convergence seems to slow down.

7.8 Correlated Random Numbers

In many cases involving multiple assets, we would like to generate correlated, normally distributed random numbers. Suppose we have $i = 1, \dots, d$ assets, and each asset follows the simulated path

$$S_i^{n+1} = S_i^n + S_i^n(r\Delta t + \phi_i^n \sigma_i \sqrt{\Delta t}) \quad (7.8.1)$$

where ϕ_i^n is $N(0, 1)$ and

$$E(\phi_i^n \phi_j^n) = \rho_{ij} \quad (7.8.2)$$

where ρ_{ij} is the correlation between asset i and asset j .

Now, it is easy to generate a set of d uncorrelated $N(0, 1)$ variables. Call these $\epsilon_1, \dots, \epsilon_d$. So, how do we produce correlated numbers? Let

$$[\Psi]_{ij} = \rho_{ij} \quad (7.8.3)$$

be the matrix of correlation coefficients. Assume that this matrix is SPD (if not, one of the random variables is a linear combination of the others, hence this is a degenerate case). Assuming Ψ is SPD, we can Cholesky factor $\Psi = LL^t$, so that

$$\rho_{ij} = \sum_k L_{ik} L_{kj}^t \quad (7.8.4)$$

Let $\bar{\phi}$ be the vector of correlated normally distributed random numbers (i.e. what we want to get), and let $\bar{\epsilon}$ be the vector of uncorrelated $N(0, 1)$ numbers (i.e. what we are given).

$$\bar{\phi} = \begin{bmatrix} \phi_1 \\ \phi_2 \\ | \\ \phi_d \end{bmatrix} ; \quad \bar{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ | \\ \epsilon_d \end{bmatrix} \quad (7.8.5)$$

So, given $\bar{\epsilon}$, we have

$$E(\epsilon_i \epsilon_j) = \delta_{ij}$$

where

$$\begin{aligned} \delta_{ij} &= 0 ; \text{ if } i \neq j \\ &= 1 ; \text{ if } i = j . \end{aligned}$$

since the ϵ_i are uncorrelated. Now, let

$$\phi_i = \sum_j L_{ij} \epsilon_j \quad (7.8.6)$$

which gives

$$\begin{aligned} \phi_i \phi_k &= \sum_j \sum_l L_{ij} L_{kl} \epsilon_j \epsilon_l \\ &= \sum_j \sum_l L_{ij} \epsilon_j \epsilon_l L_{lk}^t . \end{aligned} \quad (7.8.7)$$

Now,

$$\begin{aligned} E(\phi_i \phi_k) &= E \left[\sum_j \sum_l L_{ij} \epsilon_j \epsilon_l L_{lk}^t \right] \\ &= \sum_j \sum_l L_{ij} E(\epsilon_j \epsilon_l) L_{lk}^t \\ &= \sum_j \sum_l L_{ij} \delta_{jl} L_{lk}^t \\ &= \sum_l L_{il} L_{lk}^t \\ &= \rho_{ij} \end{aligned} \quad (7.8.8)$$

It is easy to show that

$$\begin{aligned} E(\phi_i) &= 0 \\ E(\phi_i^2) &= \rho_{ii} = 1 . \end{aligned} \quad (7.8.9)$$

So, in order to generate correlated $N(0, 1)$ numbers:

- Factor the correlation matrix $\Psi = LL^t$
- Generate uncorrelated $N(0, 1)$ numbers ϵ_i
- Correlated numbers ϕ_i are given from

$$\bar{\phi} = L \bar{\epsilon}$$

7.9 Integration of Stochastic Differential Equations

Up to now, we have been fairly slack about defining what we mean by convergence when we use forward Euler timestepping (7.0.2) to integrate

$$dS = \mu S dt + \sigma S dZ . \quad (7.9.1)$$

The forward Euler algorithm is simply

$$S^{i+1} = S^i + S^i(\mu h + \phi^i \sqrt{h}) \quad (7.9.2)$$

where $h = \Delta t$ is the finite timestep. For a good overview of these methods, check out (“*An algorithmic introduction to numerical simulation of stochastic differential equations*,” by D. Higham, SIAM Review vol. 43 (2001) 525-546). This article also has some good tips on solving SDEs using MATLAB, in particular, taking full advantage of the vectorization of MATLAB. Note that eliminating as many *for* loops as possible (i.e. computing all the MC realizations for each timestep in a vector) can reduce computation time by orders of magnitude.

Before we start defining what we mean by convergence, let’s consider the following situation. Recall that

$$dZ = \phi \sqrt{dt} \quad (7.9.3)$$

where ϕ is a random draw from a normal distribution with mean zero and variance one. Let’s imagine generating a set of Z values at discrete times t_i , e.g. $Z(t_i) = Z_i$, by

$$Z_{i+1} = Z_i + \phi \sqrt{\Delta t} . \quad (7.9.4)$$

Now, these are all legitimate points along a Brownian motion path, since there is no timestepping error here, in view of equation (1.7.22). So, this set of values $\{Z_0, Z_1, \dots\}$ are valid points along a Brownian path. Now, recall that the exact solution (for a given Brownian path) of equation (7.9.1) is given by equation (1.7.26)

$$S(T) = S(0) \exp[(\mu - \frac{\sigma^2}{2})t + \sigma(Z(T) - Z(0))] \quad (7.9.5)$$

where T is the stopping time of the simulation.

Now if we integrate equation (7.9.1) using forward Euler, with the discrete timesteps $\Delta t = t_{i+1} - t_i$, using the realization of the Brownian path $\{Z_0, Z_1, \dots\}$, we will not get the exact solution (7.9.5). This is because even though the Brownian path points are exact, time discretization errors are introduced in equation (7.9.2). So, how can we systematically study convergence of algorithm (7.9.2)? We can simply take smaller timesteps. However, we want to do this by *filling in* new Z values in the Brownian path, while keeping the old values (since these are perfectly legitimate values). Let $S(T)^h$ represent the forward Euler solution (7.9.2) for a fixed timestep h . Let $S(T)$ be the exact solution (7.9.5). As $h \rightarrow 0$, we would expect $S(T)^h \rightarrow S(T)$, for a given path.

7.9.1 The Brownian Bridge

So, given a set of valid Z_k , how do we refine this path, keeping the existing points along this path? In particular, suppose we have two points Z_i, Z_k , at (t_i, t_k) , and we would like to generate a point Z_j at t_j , with $t_i < t_j < t_k$. How should we pick Z_j ? What density function should we use when generating Z_j , given that Z_k is known?

Let x, y be two draws from a normal distribution with mean zero and variance one. Suppose we have the point $Z(t_i) = Z_i$ and we generate $Z(t_j) = Z_j$, $Z(t_k) = Z_k$ along the Wiener path,

$$Z_j = Z_i + x \sqrt{t_j - t_i} \quad (7.9.6)$$

$$Z_k = Z_j + y \sqrt{t_k - t_j} \quad (7.9.7)$$

$$Z_k = Z_i + x \sqrt{t_j - t_i} + y \sqrt{t_k - t_j} . \quad (7.9.8)$$

So, given (x, y) , and Z_i , we can generate Z_j, Z_k . Suppose on the other hand, we have Z_i , and we generate Z_k directly using

$$Z_k = Z_i + z\sqrt{t_k - t_i}, \quad (7.9.9)$$

where z is $N(0, 1)$. Then how do we generate Z_j using equation (7.9.6)? Since we are now specifying that we know Z_k , this means that our method for generating Z_j is constrained. For example, given z , we must have that, from equations (7.9.8) and (7.9.9)

$$y = \frac{z\sqrt{t_k - t_i} - x\sqrt{t_j - t_i}}{\sqrt{t_k - t_j}}. \quad (7.9.10)$$

Now the probability density of drawing the pair (x, y) given z , denoted by $p(x, y|z)$ is

$$p(x, y|z) = \frac{p(x)p(y)}{p(z)} \quad (7.9.11)$$

where $p(\cdot)$ is a standard normal distribution, and we have used the fact that successive increments of a Brownian process are uncorrelated.

From equation (7.9.10), we can write $y = y(x, z)$, so that $p(x, y|z) = p(x, y(x, z)|z)$

$$\begin{aligned} p(x, y(x, z)|z) &= \frac{p(x)p(y(x, z))}{p(z)} \\ &= \frac{1}{\sqrt{2\pi}} \exp \left[-\frac{1}{2}(x^2 + y^2 - z^2) \right] \end{aligned} \quad (7.9.12)$$

or (after some algebra, using equation (7.9.10))

$$\begin{aligned} p(x|z) &= \frac{1}{\sqrt{2\pi}} \exp \left[-\frac{1}{2}(x - \alpha z)^2 / \beta^2 \right] \\ \alpha &= \sqrt{\frac{t_j - t_i}{t_k - t_i}} \\ \beta &= \sqrt{\frac{t_k - t_j}{t_k - t_i}} \end{aligned} \quad (7.9.13)$$

so that x is normally distributed with mean αz and variance β^2 . Since

$$z = \frac{Z_k - Z_i}{\sqrt{t_k - t_i}} \quad (7.9.14)$$

we have that x has mean

$$E(x) = \frac{\sqrt{t_j - t_i}}{t_k - t_i} (Z_k - Z_i) \quad (7.9.15)$$

and variance

$$E[(x - E(x))^2] = \frac{t_k - t_j}{t_k - t_i} \quad (7.9.16)$$

Now, let

$$x = \frac{\sqrt{t_j - t_i}}{t_k - t_i} (Z_k - Z_i) + \phi \sqrt{\frac{t_k - t_j}{t_k - t_i}} \quad (7.9.17)$$

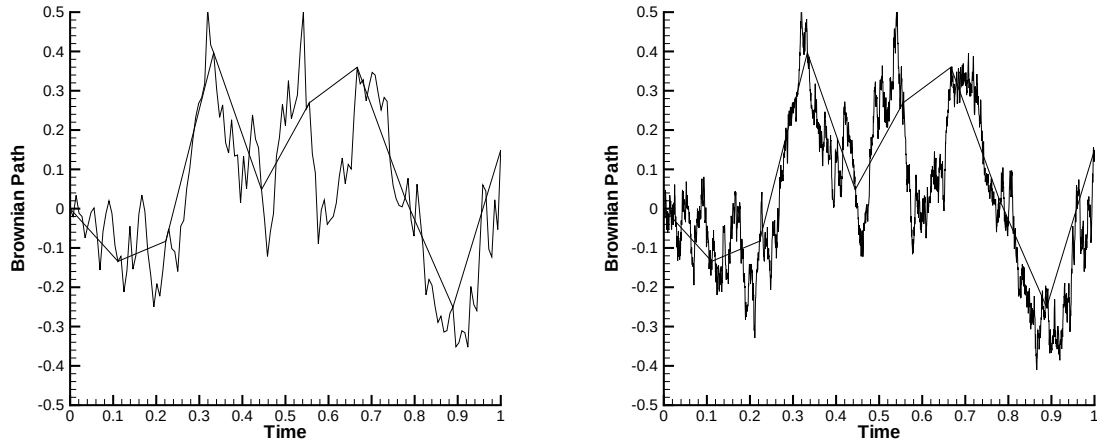


Figure 7.9.1: *Effect of adding more points to a Brownian path using a Brownian bridge. Note that the small timestep points match the coarse timestep points. Left: each coarse timestep is divided into 16 substeps. Right: each coarse timestep divided into 64 substeps.*

where ϕ is $N(0, 1)$. Clearly, x satisfies equations (7.9.15) and (7.9.17). Substituting equation (7.9.17) into (7.9.6) gives

$$Z_j = \left(\frac{t_k - t_j}{t_k - t_i} \right) Z_i + \left(\frac{t_j - t_i}{t_k - t_i} \right) Z_k + \phi \sqrt{\frac{(t_j - t_i)(t_k - t_j)}{(t_k - t_i)}} \quad (7.9.18)$$

where ϕ is $N(0, 1)$. Equation (7.9.18) is known as the Brownian Bridge.

Figure 7.9.1 shows different Brownian paths constructed for different timestep sizes. An initial coarse path is constructed, then the fine timestep path is constructed from the coarse path using a Brownian Bridge. By construction, the final timestep path will pass through the coarse timestep nodes.

Figure 7.9.2 shows the asset paths integrated using the forward Euler algorithm (7.9.2) fed with the Brownian paths in Figure 7.9.1. In this case, note that the fine timestep path does not coincide with the coarse timestep nodes, due to the timestepping error.

7.9.2 Strong and Weak Convergence

Since we are dealing with a probabilistic situation here, it is not obvious how to define convergence. Given a number of points along a Brownian path, we could imagine refining this path (using a Brownian Bridge), and then seeing if the solution converged to exact solution. For the model SDE (7.9.1), we could ask that

$$E [|S(T) - S^h(T)|] \leq \text{Const. } h^\gamma \quad (7.9.19)$$

where the expectation in equation (7.9.19) is over many Brownian paths, and h is the timestep size. Note that $S(T)$ is the exact solution along a particular Brownian path; the same path used to compute $S^h(T)$. Criterion (7.9.19) is called *strong convergence*. A less strict criterion is

$$|E [S(T)] - E [S^h(T)]| \leq \text{Const. } h^\gamma \quad (7.9.20)$$

It can be shown that using forward Euler results in weak convergence with $\gamma = 1$, and strong convergence with $\gamma = .5$.

Table 7.9.1 shows some test data used to integrate the SDE (7.9.1) using method (7.9.2). A series of Brownian paths was constructed, beginning with a coarse timestep path. These paths were systematically

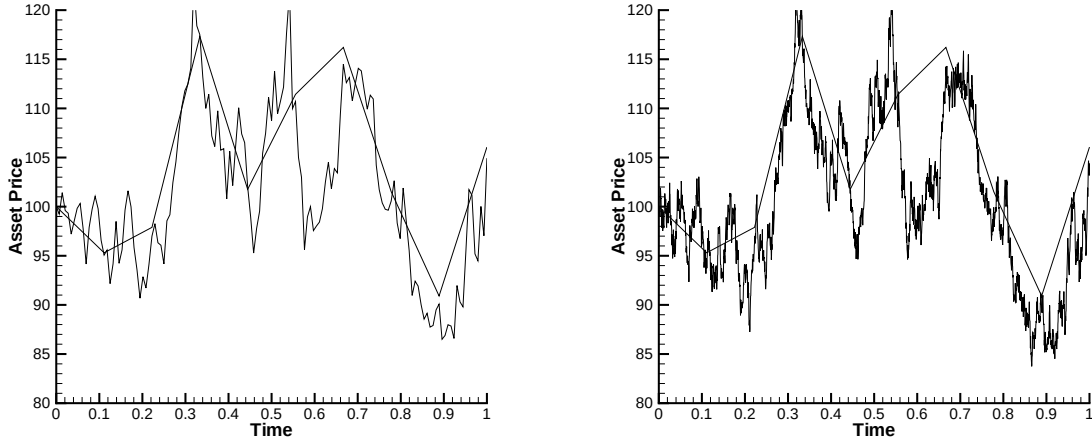


Figure 7.9.2: Brownian paths shown in Figure 7.9.1 used to determine asset price paths using forward Euler timestepping (7.9.2). In this case, note that the asset paths for fine and coarse timestepping do not agree at the final time (due to the timestepping error). Eventually, for small enough timesteps, the final asset value will converge to the exact solution to the SDE. Left: each coarse timestep is divided into 16 substeps. Right: each coarse timestep divided into 64 substeps.

refined using the Brownian Bridge construction. Table 7.9.2 shows results where the strong and weak convergence errors are estimated as

$$\text{Strong Error} = \frac{1}{N} \sum_{i=1}^N [|S(T)_i - S^h(T)_i|] \quad (7.9.21)$$

$$\text{Weak Error} = \left| \frac{1}{N} \sum_{i=1}^N [S(T)_i] - \frac{1}{N} \sum_{i=1}^N [S^h(T)_i] \right|, \quad (7.9.22)$$

where $S^h(T)_i$ is the solution obtained by forward Euler timestepping along the i 'th Brownian path, and $S(T)_i$ is the exact solution along this same path, and N is the number of samples. Note that for equation (7.9.1), we have the exact solution

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N [S(T)_i] = S_0 e^{\mu T} \quad (7.9.23)$$

but we do not replace the approximate sampled value of the limit in equation (7.9.22) by the theoretical limit (7.9.23). If we use enough Monte Carlo samples, we could replace the approximate expression

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N [S(T)_i]$$

by $S_0 e^{\mu T}$, but for normal parameters, the Monte Carlo sampling error is much larger than the timestepping error, so we would have to use an enormous number of Monte Carlo samples. Estimating the weak error using equation (7.9.22) will measure the timestepping error, as opposed to the Monte Carlo sampling error. However, for normal parameters, even using equation (7.9.22) requires a large number of Monte Carlo samples in order to ensure that the error is dominated by the timestepping error.

In Table 7.9.1, we can see that the ratio of the errors is about $\sqrt{2}$ for the strong error, and about two for the weak error. This is consistent with a convergence rate of $\gamma = .5$ for strong convergence, and $\gamma = 1.0$ for weak convergence.

T	.25
σ	.4
μ	.06
S_0	100

Table 7.9.1: *Data used in the convergence tests.*

Timesteps	Strong Error (7.9.19)	Weak Error (7.9.20)
72	.0269	.00194
144	.0190	.00174
288	.0135	.00093
576	.0095	.00047

Table 7.9.2: *Convergence results, 100,000 samples used. Data in Table 7.9.1.*

7.10 MATLAB and Monte Carlo Simulation

Straightforward Matlab and Python/Numpy implementations of Monte Carlo timestepping for solving the SDE

$$dS = \mu S dt + \sigma S dZ \quad (7.10.1)$$

are shown in Algorithm (7.10.2) and Algorithm (7.10.3). This code runs very slowly.

Alternatively, we can use vectorization to interchange the timestep loop and the simulation loop. The innermost simulation loop can be replaced by vector statements, as shown in Algorithm (7.10.4) and Algorithm (7.10.5). These run much faster.

Slow.m

```
randn('state',100);
T = 1.00;           % expiry time
sigma = 0.25;       % volatility
mu = .10;           % P measure drift
S_init = 100;       % initial value
N_sim = 10000;      % number of simulations
N = 100;            % number of timesteps
delt = T/N;         % timestep

drift = mu*delt;
sigma_sqrt_delt = sigma*sqrt(delt);
S_new = zeros(N_sim,1);

for m=1:N_sim
    S = S_init;
    for i=1:N % timestep loop

        S = S + S*( drift + sigma_sqrt_delt*randn(1,1) );

        S = max(0.0, S );
        % check to make sure that S_new cannot be < 0

    end % timestep loop
    S_new(m,1) = S;
end % simulation loop

n_bin = 200;
hist(S_new, n_bin);

stndrd_dev = std(S_new);
disp(sprintf('standard deviation:  %.5g\n',stndrd_dev));

mean_S = mean(S_new);
disp(sprintf('mean:  %.5g\n',stndrd_dev));
```

(7.10.2)

Slow MC in Python

```
import math
import numpy as np
from matplotlib import pyplot as plt

np.random.seed(100)

T = 1.00 # expiry time
sigma = 0.25 # volatility
mu = 0.10 # P measure drift
S_init = 100 # initial value
N_sim = 10000 # number of simulations
N = 100 # number of timesteps
delt = T/N # timestep

drift = mu*delt
sigma_sqrt_delt = sigma*math.sqrt(delt)

S_old = np.zeros(N_sim)
S_new = np.zeros(N_sim)

for m in np.arange(N_sim):
    S = S_init
    for i in np.arange(N): #timestep loop
        S = S + S*(drift + sigma_sqrt_delt* np.random.normal(0,1))
        S = max(0.0, S) # check to ensure S_new cannot be < 0
    S_new[m] = S

n_bin = 200
plt.hist(S_new,n_bin)
stdrd_dev = np.std(S_new)
print("Standard deviation %.5g" % (stdrd_dev))

mean_S = np.mean(S_new)
print("Mean %.5g" % (mean_S))
```

(7.10.3)

Fast.m

```

randn('state',100);
%
T = 1.00;           % expiry time
sigma = 0.25;       % volatility
mu = .10;           % P measure drift
S_init = 100;       % initial value
N_sim = 10000;      % number of simulations
N = 100;            % number of timesteps
delt = T/N;         % timestep

drift = mu*delt;
sigma_sqrt_delt = sigma*sqrt(delt);

S_old = zeros(N_sim,1);
S_new = zeros(N_sim,1);

S_old(1:N_sim,1) = S_init;

for i=1:N % timestep loop

    % now, for each timestep, generate info for
    % all simulations

    S_new(:,1) = S_old(:,1) +...
    S_old(:,1).*( drift + sigma_sqrt_delt*randn(N_sim,1) );

    S_new(:,1) = max(0.0, S_new(:,1) );
    % check to make sure that S_new cannot be < 0

    S_old(:,1) = S_new(:,1);
    %
    % end of generation of all data for all simulations
    % for this timestep

end % timestep loop

n_bin = 200;
hist(S_new, n_bin);

stndrd_dev = std(S_new);
disp(sprintf('standard deviation:  %.5g\n',stndrd_dev));

mean_S = mean(S_new);
disp(sprintf('mean:  %.5g\n',stndrd_dev));

```

(7.10.4)

Fast MC in Python

```
import math
import numpy as np
from matplotlib import pyplot as plt

np.random.seed(100)

T = 1.00 # expiry time
sigma = 0.25 # volatility
mu = 0.10 # P measure drift
S_init = 100 # initial value
N_sim = 10000 # number of simulations
N = 100 # number of timesteps
delt = T/N # timestep

drift = mu*delt
sigma_sqrt_delt = sigma*math.sqrt(delt)

S_old = S_init*np.ones(N_sim)
S_new = np.zeros(N_sim)

for i in np.arange(N): #timestep loop
    # now for each timestep generate info for all simulations
    S_new = S_old + S_old*(drift + sigma_sqrt_delt * np.random.randn(N_sim))

    # check to ensure S_new cannot be < 0
    S_new = np.maximum(S_new, 0)
    S_old = S_new

    # end of generation of all data for all simulations
    # for this timestep

n_bin = 200
plt.hist(S_new,n_bin)

stndrd_dev = np.std(S_new)
print("Standard deviation %.5g" % (stndrd_dev))

mean_S = np.mean(S_new)
print("Mean %.5g" % (mean_S))
```

(7.10.5)

Chapter 8

Discrete Hedging

In practice, we cannot hedge at infinitesimal time intervals. In fact, we would like to hedge as infrequently as possible, since in real life, there are transaction costs (something which is ignored in the basic Black-Scholes equation, but which can be taken into account and results in a nonlinear PDE).

8.1 Delta Hedging

Recall that the basic derivation of the Black-Scholes equation used a hedging portfolio where we hold V_S shares. In finance, V_S is called the option delta, hence this strategy is called delta hedging.

As an example, consider the hedging portfolio $P(t)$ which is composed of

- A short position in an option $-V(t)$.
- Long $\alpha(t)^h S(t)$ shares
- An amount in a risk-free bank account $B(t)$.

Initially, we have

$$\begin{aligned} P(0) = 0 &= -V(0) + \alpha(0)^h S(0) + B(0) \\ \alpha &= V_S \\ B(0) &= V(0) - \alpha(0)^h S(0) \end{aligned}$$

The hedge is rebalanced at discrete times t_i . Defining

$$\begin{aligned} \alpha_i^h &= V_S(S_i, t_i) \\ V_i &= V(S_i, t_i) \end{aligned}$$

then, we have to update the hedge by purchasing $\alpha_i - \alpha_{i-1}$ shares at $t = t_i$, so that updating our share position requires

$$S(t_i)(\alpha_i^h - \alpha_{i-1}^h)$$

in cash, which we borrow from the bank if $(\alpha_i^h - \alpha_{i-1}^h) > 0$. If $(\alpha_i^h - \alpha_{i-1}^h) < 0$, then we sell some shares and deposit the proceeds in the bank account. If $\Delta t = t_i - t_{i-1}$, then the bank account balance is updated by

$$B_i = e^{r\Delta t} B_{i-1} - S_i(\alpha_i^h - \alpha_{i-1}^h)$$

At the instant after the rebalancing time t_i , the value of the portfolio is

$$P(t_i) = -V(t_i) + \alpha(t_i)^h S(t_i) + B(t_i)$$

Since we are hedging at discrete time intervals, the hedge is no longer risk free (it is risk free only in the limit as the hedging interval goes to zero). We can determine the distribution of profit and loss (P & L) by carrying out a Monte Carlo simulation. Suppose we have precomputed the values of V_S for all the likely (S, t) values. Then, we simulate a series of random paths. For each random path, we determine the discounted relative hedging error

$$\text{error} = \frac{e^{-rT^*} P(T^*)}{V(S_0, t = 0)} \quad (8.1.1)$$

After computing many sample paths, we can plot a histogram of relative hedging error, i.e. fraction of Monte Carlo trials giving a hedging error between E and $E + \Delta E$. We can compute the variance of this distribution, and also the value at risk (VAR). VAR is the worst case loss with a given probability. For example, a typical VAR number reported is the maximum loss that would occur 95% of the time. In other words, find the value of E along the x-axis such that the area under the histogram plot to the right of this point is $.95 \times$ the total area.

As an example, consider the case of an American put option, $T = .25$, $\sigma = .3$, $r = .06$, $K = S_0 = 100$. At $t = 0$, $S_0 = 100$. Since there are discrete hedging errors, the results in this case will depend on the stock drift rate, which we set at $\mu = .08$. The initial value of the American put, obtained by solving the Black-Scholes linear complementarity problem, is \$5.34. Figure 8.1.1 shows the results for no hedging, and hedging once a month. The x-axis in these plots shows the relative P & L of this portfolio (i.e. P & L divided by the Black-Scholes price), and the y-axis shows the relative frequency.

$$\text{Relative P\&L} = \frac{\text{Actual P\&L}}{\text{Black-Scholes price}} \quad (8.1.2)$$

Note that the no-hedging strategy actually has a high probability of ending up with a profit (from the option writer's point of view) since the drift rate of the stock is positive. In this case, the hedger does nothing, but simply pockets the option premium. Note the sudden jump in the relative frequency at relative $P\&L = 1$. This is because the maximum the option writer stands to gain is the option premium, which we assume is the Black-Scholes value. The writer makes this premium for any path which ends up $S > K$, which is many paths, hence the sudden jump in probability. However, there is significant probability of a loss as well. Figure 8.1.1 also shows the relative frequency of the $P\&L$ of hedging once a month (only three times during the life of the option).

In fact, there is a history of Ponzi-like hedge funds which simply write put options with essentially no hedging. In this case, these funds will perform very well for several years, since markets tend to drift up on average. However, then a sudden market drop occurs, and they will *blow up*. Blowing up is a technical term for losing all your capital and being forced to get a real job. However, usually the owners of these hedge funds walk away with large bonuses, and the shareholders take all the losses.

Figure 8.1.2 shows the results for rebalancing the hedge once a week, and daily. We can see clearly here that the mean is zero, and variance is getting smaller as the hedging interval is reduced. In fact, one can show that the standard deviation of the hedge error should be proportional to $\sqrt{\Delta t}$ where Δt is the hedge rebalance frequency.

As another example, it is interesting to examine the stock, bond, and portfolio values along a single stochastic path. Figure 8.1.3 shows these values for hedging one year put, at a very high rebalancing frequency.

8.2 Gamma Hedging

In an attempt to account for some the errors in delta hedging at finite hedging intervals, we can try to use second derivative information. The second derivative of an option value V_{SS} is called the option gamma, hence this strategy is termed delta-gamma hedging.

A gamma hedge consists of

- A short option position $-V(t)$.

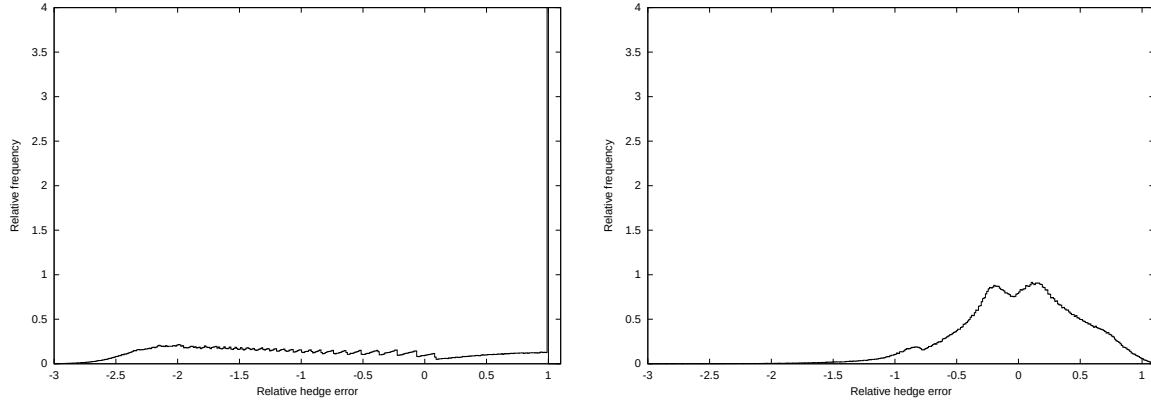


Figure 8.1.1: Relative frequency (y-axis) versus relative P&L of delta hedging strategies. Left: no hedging, right: rebalance hedge once a month. American put, $T = .25$, $\sigma = .3$, $r = .06$, $\mu = .08$, $K = S_0 = 100$. The relative P&L is computed by dividing the actual P&L by the Black-Scholes price.

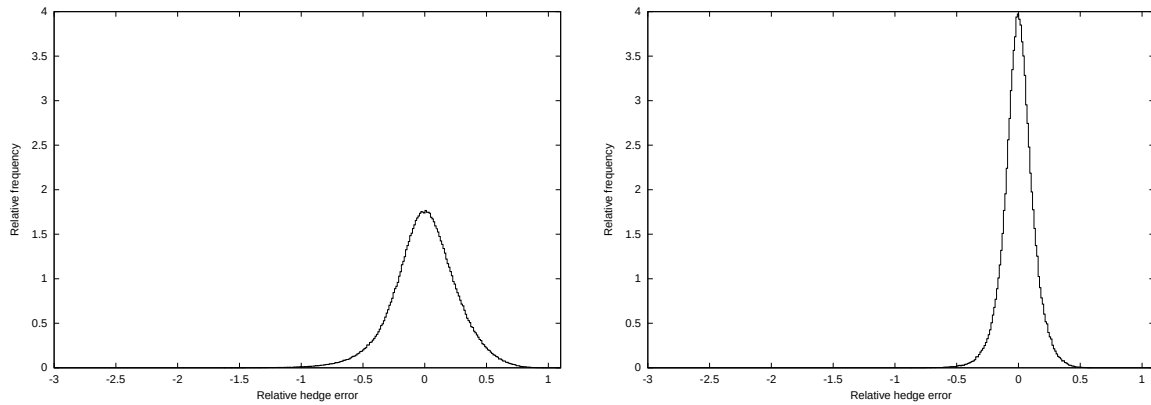


Figure 8.1.2: Relative frequency (y-axis) versus relative P&L of delta hedging strategies. Left: rebalance hedge once a week, right: rebalance hedge daily. American put, $T = .25$, $\sigma = .3$, $r = .06$, $\mu = .08$, $K = S_0 = 100$. The relative P&L is computed by dividing the actual P&L by the Black-Scholes price.

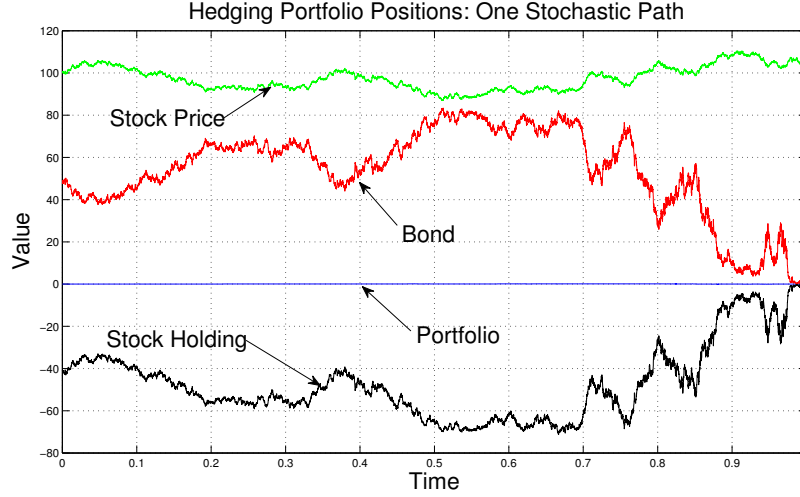


Figure 8.1.3: Values of components of the hedging portfolio along a single stochastic path. One year put. $K = 100$, $r = .02$, $\sigma = .20$, real world drift $.10$. Rebalanced 10000 times.

- Long $\alpha^h S(t)$ shares
- Long β another derivative security I .
- An amount in a risk-free bank account $B(t)$.

Now, recall that we consider α^h, β to be constant over the hedging interval (no peeking into the future), so we can regard these as constants (for the duration of the hedging interval).

The hedge portfolio $P(t)$ is then

$$P(t) = -V + \alpha^h S + \beta I + B(t)$$

Assuming that we buy and hold α^h shares and β of the secondary instrument at the beginning of each hedging interval, then we require that

$$\begin{aligned} \frac{\partial P}{\partial S} &= -\frac{\partial V}{\partial S} + \alpha^h + \beta \frac{\partial I}{\partial S} = 0 \\ \frac{\partial^2 P}{\partial S^2} &= -\frac{\partial^2 V}{\partial S^2} + \beta \frac{\partial^2 I}{\partial S^2} = 0 \end{aligned} \quad (8.2.1)$$

Note that

- If $\beta = 0$, then we get back the usual delta hedge.
- In order for the gamma hedge to work, we need an instrument which has some gamma (the asset S has second derivative zero). Hence, traders often speak of being long (positive) or short (negative) gamma, and try to buy/sell things to get gamma neutral.

So, at $t = 0$ we have

$$P(0) = 0 \Rightarrow B(0) = V(0) - \alpha_0^h S_0 - \beta_0 I_0$$

The amounts α_i^h, β_i are determined by requiring that equation (8.2.1) hold

$$\begin{aligned} -(V_S)_i + \alpha_i^h + \beta_i (I_S)_i &= 0 \\ -(V_{SS})_i + \beta_i (I_{SS})_i &= 0 \end{aligned} \quad (8.2.2)$$

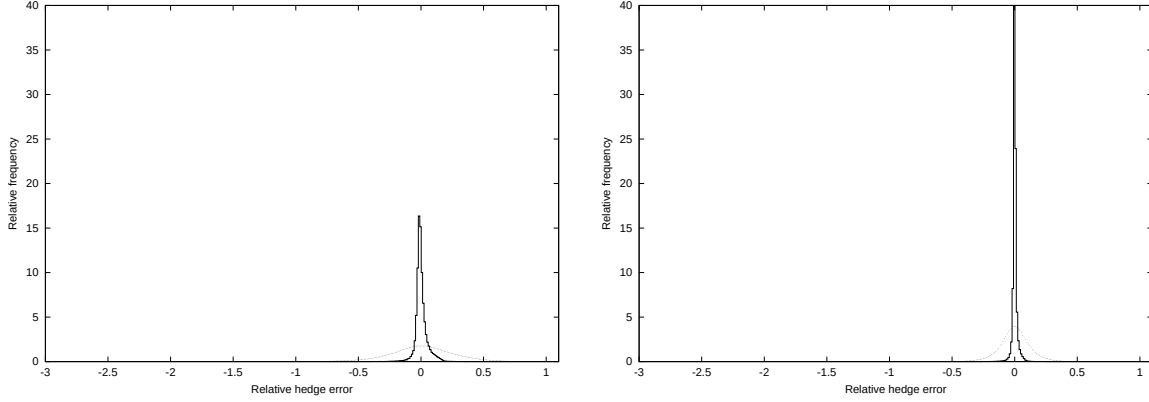


Figure 8.2.1: Relative frequency (y-axis) versus relative P&L of gamma hedging strategies. Left: rebalance hedge once a week, right: rebalance hedge daily. Dotted lines show the delta hedge for comparison. American put, $T = .25$, $\sigma = .3$, $r = .06$, $\mu = .08$, $K = 100$, $S_0 = 100$. Secondary instrument: European put option, same strike, $T = .5$ years. The relative P&L is computed by dividing the actual P&L by the Black-Scholes price.

The bank account balance is then updated at each hedging time t_i by

$$B_i = e^{r\Delta t} B_{i-1} - S_i(\alpha_i^h - \alpha_{i-1}^h) - I_i(\beta_i - \beta_{i-1})$$

We will consider the same example as we used in the delta hedge example. For an additional instrument, we will use a European put option written on the same underlying with the same strike price and a maturity of $T=.5$ years.

Figure 8.2.1 shows the results of gamma hedging, along with a comparison on delta hedging. In principle, gamma hedging produces a smaller variance with less frequent hedging. However, we are exposed to more *model error* in this case, since we need to be able to compute the second derivative of the theoretical price.

8.3 Vega Hedging

The most important parameter in the option pricing is the volatility. What if we are not sure about the value of the volatility? It is possible to assume that the volatility itself is stochastic, i.e.

$$\begin{aligned} dS &= \mu S dt + \sqrt{v} S dZ_1 \\ dv &= \kappa(\theta - v) dt + \sigma_v \sqrt{v} dZ_2 \end{aligned} \quad (8.3.1)$$

where μ is the expected growth rate of the stock price, $\sqrt{v}$ is its instantaneous volatility, κ is a parameter controlling how fast v reverts to its mean level of θ , σ_v is the “volatility of volatility” parameter, and Z_1, Z_2 are Wiener processes with correlation parameter ρ .

If we use the model in equation (8.3.1), this will result in a two factor PDE to solve for the option price and the hedging parameters. Since there are two sources of risk (dZ_1, dZ_2), we will need to hedge with the underlying asset and another option (Heston, *A closed form solution for options with stochastic volatility with applications to bond and currency options*, Rev. Fin. Studies 6 (1993) 327-343).

Another possibility is to assume that the volatility is uncertain, and to assume that

$$\sigma_{min} \leq \sigma \leq \sigma_{max},$$

and to hedge based on a worst case (from the hedger’s point of view). This results in an uncertain volatility model (Avellaneda, Levy, Paris, *Pricing and Hedging Derivative Securities in Markets with Uncertain*

Volatilities, Appl. Math. Fin. 2 (1995) 77-88). This is great if you can get someone to buy this option at this price, because the hedger is always guaranteed to end up with a non-negative balance in the hedging portfolio. But you may not be able to sell at this price, since the option price is expensive (after all, the price you get has to cover the worst case scenario).

An alternative, much simpler, approach (and therefore popular in industry), is to construct a vega hedge. We assume that we know the volatility, and price the option in the usual way. Then, as with a gamma hedge, we construct a portfolio

- A short option position $-V(t)$.
- Long $\alpha^h S(t)$ shares
- Long β another derivative security I .
- An amount in a risk-free bank account $B(t)$.

The hedge portfolio $P(t)$ is then

$$P(t) = -V + \alpha^h S + \beta I + B(t)$$

Assuming that we buy and hold α^h shares and β of the secondary instrument at the beginning of each hedging interval, then we require that

$$\begin{aligned} \frac{\partial P}{\partial S} &= -\frac{\partial V}{\partial S} + \alpha^h + \beta \frac{\partial I}{\partial S} = 0 \\ \frac{\partial P}{\partial \sigma} &= -\frac{\partial V}{\partial \sigma} + \beta \frac{\partial I}{\partial \sigma} = 0 \end{aligned} \tag{8.3.2}$$

Note that if we assume that σ is constant when pricing the option, yet do not assume σ is constant when we hedge, this is somewhat inconsistent. Nevertheless, we can determine the derivatives in equation (8.3.2) numerically (solve the pricing equation for several different values of σ , and then finite difference the solutions).

In practice, we would sell the option priced using our best estimate of σ (today). This is usually based on looking at the prices of traded options, and then backing out the volatility which gives back today's traded option price (this is the *implied volatility*). Then as time goes on, the implied volatility will likely change. We use the current implied volatility to determine the current hedge parameters in equation (8.3.2). Since this implied volatility has likely changed since we last rebalanced the hedge, there is some error in the hedge. However, taking into account the change in the hedge portfolio through equations (8.3.2) should make up for this error. This procedure is called delta-vega hedging.

In fact, even if the underlying process is a stochastic volatility, the vega hedge computed using a constant volatility model works surprisingly well (Hull and White, *The pricing of options on assets with stochastic volatilities*, J. of Finance, 42 (1987) 281-300).

8.4 A Stop-Loss Strategy

A simple minded idea for hedging options is based on the following idea. Suppose the writer has sold one call option with strike K . Assume that $S_0 < K$, i.e. the initial asset price is less than K . As soon as $S = K + \epsilon$, $\epsilon \ll 1$, the writer borrows $K + \epsilon$ and buys the stock. As soon as $S = K - \epsilon$, the writer sells the stock. If $S < K$ at expiry, the writer holds no stock, and does not owe the holder of the option anything. If $S > K$ at expiry, the writer sells the stock at price S , gives the holder $S - K$, and pays back the loan of K used to buy the stock.

This looks like a good strategy. As far as a loan to buy the stock, the worst case for the financing cost of this would be $K(e^{rT} - 1)$, which will be small if $rT \ll 1$. In addition, note that buying occurs at price $K + \epsilon$, and selling occurs at price $K - \epsilon$, resulting in a transaction cost of 2ϵ for each buy-sell trade. But if we monitor the strategy closely, we can make ϵ as small as we like. Consequently, this strategy should cost less than the Black-Scholes price. We should be able to make millions.

T	.3846
σ	.2
μ (real world drift)	.13
S_0 (initial price)	49
r	= .05
K	50
Option Type	Call
Early Exercise	No

Table 8.4.1: Data used in the stop loss hedging strategy test.

Hedging Frequency	100	200	400	800
Mean	-0.047345	-0.04373	-0.040295	-0.040762
Standard Deviation	0.78175	0.77473	0.76954	0.77247
VAR (95%)	-1.523	-1.512	-1.5059	-1.5043
CVAR (95%)	-2.0122	-1.9983	-1.9847	-1.9974

Table 8.4.2: Statistics for the profit and loss, relative to the Black-Scholes price of 2.4005. Assumes 2.4005 received in cash initially.

Consider the following example, from the Hull book, with the data in Table 8.4.2. A Monte Carlo strategy was used to simulate this strategy (80,000 simulations). Table 8.4.1 shows the results in terms of the relative hedging error (8.1.1) for various numbers of rebalancing times. It is assumed that the hedger receives the Black-Scholes price in cash initially. Hence, a positive mean would indicate a relative profit compared to the Black-Scholes strategy.

From this table, we can see that the expected cost of the strategy is actually larger than the Black-Scholes strategy (the expected relative $P\&L$ is negative), and no matter how many times the hedge is rebalanced, the standard deviation seems to be about .77. This strategy is very bad.

Why does this not work? No matter where we start, there is a finite probability of crossing the strike price. The problem can be traced to the fact that we buy at $K + \epsilon$ and sell at $K - \epsilon$. As we make the hedging interval smaller, we can make ϵ smaller, but Brownian motion is infinitely jagged on any scale. Hence the number of times we buy and sell as the asset crosses $S = K$ becomes larger as the hedging interval becomes smaller. Recall from equation (1.6.27) that the total distance travelled by a particle undergoing Brownian motion, for a finite time, is infinite.

The probability density for the relative hedging error (8.1.1) is shown in Figure 8.4.1.

8.5 Profit and Loss: probability density, VAR and CVAR

Note that Figure 8.4.1 shows the probability density instead of the relative frequency. This is a more desirable way to show the results.

Suppose P^i represents the $P\&L$ from the i^{th} simulation. Then, we can construct a histogram of the results by first specifying a series of bins, and counting the number of occurrences in each bin. Suppose the k^{th} bin lies in the interval $[a, b]$ of the x-axis. Then a histogram will show a bar indicating the number of occurrences of the P in the interval $[a, b]$. Now

$$Prob[a \leq x \leq b] \simeq \frac{\text{Number of occurrences in bin } [a, b]}{\text{Total number of simulations}}. \quad (8.5.1)$$

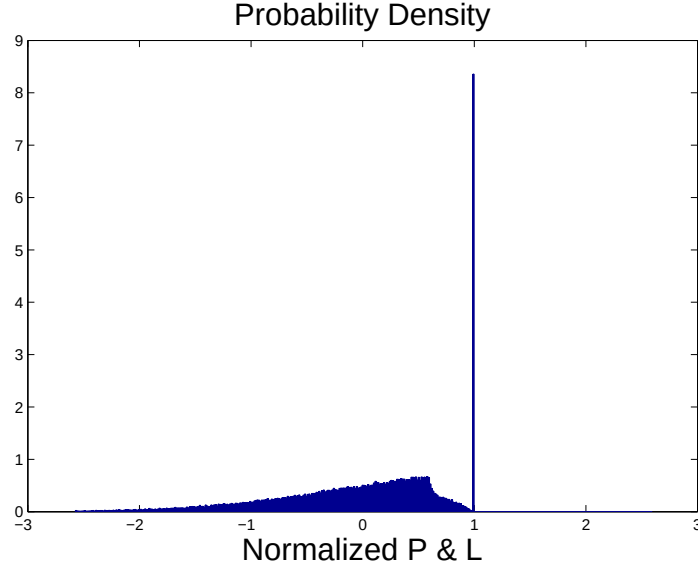


Figure 8.4.1: Probability density (y-axis) versus relative P&L of the stop loss strategy. The relative P&L is computed by dividing the actual P&L by the Black-Scholes price.

Let $p(x) dx$ be the probability of $P \in [x, x + dx]$, so

$$p(a)(b - a) \simeq \frac{\text{Number of occurrences in bin } [a, b]}{\text{Total number of simulations}}, \quad (8.5.2)$$

so that

$$p(a) \simeq \frac{1}{(b - a)} \frac{\text{Number of occurrences in bin } [a, b]}{\text{Total number of simulations}}. \quad (8.5.3)$$

Note that the MATLAB functions `histc(X, EDGES)` , `bar(X, Y, 'hist')` are useful for producing probability density plots.

A useful way to measure tail risk, is to compute the Value at Risk (VAR) and the Conditional VAR (CVAR). Given an approximate probability density $p(x)$, the $y\%$ VAR is the point on the x-axis where

$$\int_{VAR}^{\infty} p(x) dx = \frac{y}{100}. \quad (8.5.4)$$

Typically, $y = 95\%, 99\%$. VAR can be interpreted as “ $y\%$ of the time, we can lose no more than VAR”.

Another measure of tail risk is CVAR. This is defined as

$$CVAR = \frac{\int_{-\infty}^{VAR} x p(x) dx}{\int_{-\infty}^{VAR} p(x) dx}. \quad (8.5.5)$$

The $y\%$ CVAR can be interpreted as the expected loss, given that we are experiencing the worst $(100 - y)\%$ scenarios. In MATLAB, VAR can be easily computed by sorting the array of outcomes P^i . CVAR is then computed by averaging. It is interesting to note that CVAR can be computed in another way without first computing VAR.

Chapter 9

Some Investing Facts

9.1 Stocks for the Long Run?

Conventional wisdom states that investment in a diversified portfolio of equities has a low risk for a long term investor. However, in a recent article ("Irrational Optimism," Fin. Anal. J. E. Simson, P. Marsh, M. Staunton, vol 60 (January, 2004) 25-35) an extensive analysis of historical data of equity returns was carried out. Projecting this information forward, the authors conclude that the probability of a negative real return over a twenty year period, for a investor holding a diversified portfolio, is about 14 per cent. In fact, most individuals in defined contribution pension plans have poorly diversified portfolios. Making more realistic assumptions for defined contribution pension plans, the authors find that the probability of a negative real return over twenty years is about 25 per cent.

Let's see if we can explain why there is this common misconception about the riskiness of long term equity investing. Table 9.1.1 shows a typical table in a Mutual Fund advertisement. From this table, we are supposed to conclude that

- Long term equity investment is not very risky, with an annualized compound return about 3% higher than the current yield on government bonds.
- If S is the value of the mutual fund, and B is the value of the government bond, then

$$\begin{aligned} B(T) &= B(0)e^{rT} \\ r &= .03 \\ S(T) &\simeq S(0)e^{\alpha T} \\ \alpha &= .06, \end{aligned} \tag{9.1.1}$$

for T large, which gives

$$\frac{\frac{S(T=30)}{S(0)}}{\frac{B(T=30)}{B(0)}} = e^{1.8-.9} = e^{.9} \simeq 2.46, \tag{9.1.2}$$

indicating that you more than double your return by investing in equities compared to bonds (over the long term).

A convenient way to measure the relative returns on these two investments (bonds and stocks) is to compare the total compound return

$$\begin{aligned} \text{Compound return: stocks} &= \log \left[\frac{S(T)}{S(0)} \right] = \alpha T \\ \text{Compound return: bonds} &= \log \left[\frac{B(T)}{B(0)} \right] = rT, \end{aligned} \tag{9.1.3}$$

1 year	2 years	5 years	10 years	20 years	30 years	30 year bond yield
-2%	-5%	10%	8%	7%	6%	3%

Table 9.1.1: Historical annualized compound return, XYZ Mutual Equity Funds. Also shown is the current yield on a long term government bond.

or the annualized compound returns

$$\begin{aligned}\text{Annualized compound return: stocks} &= \frac{1}{T} \log \left[\frac{S(T)}{S(0)} \right] = \alpha \\ \text{Annualized compound return: bonds} &= \frac{1}{T} \log \left[\frac{B(T)}{B(0)} \right] = r .\end{aligned}\tag{9.1.4}$$

If we assume that the value of the equity portfolio S follows a Geometric Brownian Motion

$$dS = \mu S dt + \sigma S dZ\tag{9.1.5}$$

then from equation (1.7.25) we have that

$$\log \left(\frac{S(T)}{S(0)} \right) \sim N\left((\mu - \frac{\sigma^2}{2})T, \sigma^2 T\right),\tag{9.1.6}$$

i.e. the compound return in is normally distributed with mean $(\mu - \frac{\sigma^2}{2})T$ and variance $\sigma^2 T$, so that the variance of the total compound return increases as T becomes large.

Since $\text{var}(aX) = a^2 \text{var}(X)$, it follows that

$$\frac{1}{T} \log \left(\frac{S(T)}{S(0)} \right) \sim N\left((\mu - \frac{\sigma^2}{2}), \sigma^2 / T\right),\tag{9.1.7}$$

so that the the variance of the annualized return tends to zero at T becomes large.

Of course, what we really care about is the total compound return (that's how much we actually have at $t = T$, relative to what we invested at $t = 0$) at the end of the investment horizon. This is why Table 9.1.1 is misleading. There is significant risk in equities, even over the long term (30 years would be long-term for most investors).

Figure 9.1.1 shows the results of 100,000 simulations of asset prices assuming that the asset follows equation (9.1.5), with $\mu = .08, \sigma = .2$. The investment horizon is 5 years. The results are given in terms of histograms of the annualized compound return (equation (9.1.4)) and the total compound return ((equation (9.1.3)).

Figure 9.1.2 shows similar results for an investment horizon of 30 years. Note how the variance of the annualized return has decreased, while the variance of the total return has increased (verifying equations (9.1.6-9.1.7)).

Assuming long term bonds yield 3%, this gives a total compound return over 30 years of .90, for bonds. Looking at the right hand panel of Figure 9.1.2 shows that there are many possible scenarios where the return on equities will be less than risk free bonds after 30 years. The number of scenarios with return less than risk free bonds is given by the area to the left of .9 in the histogram.

9.1.1 GBM is Risky

Consider an investment which follows Geometric Brownian Motion

$$\frac{dS}{S} = \mu dt + \sigma dZ .\tag{9.1.8}$$

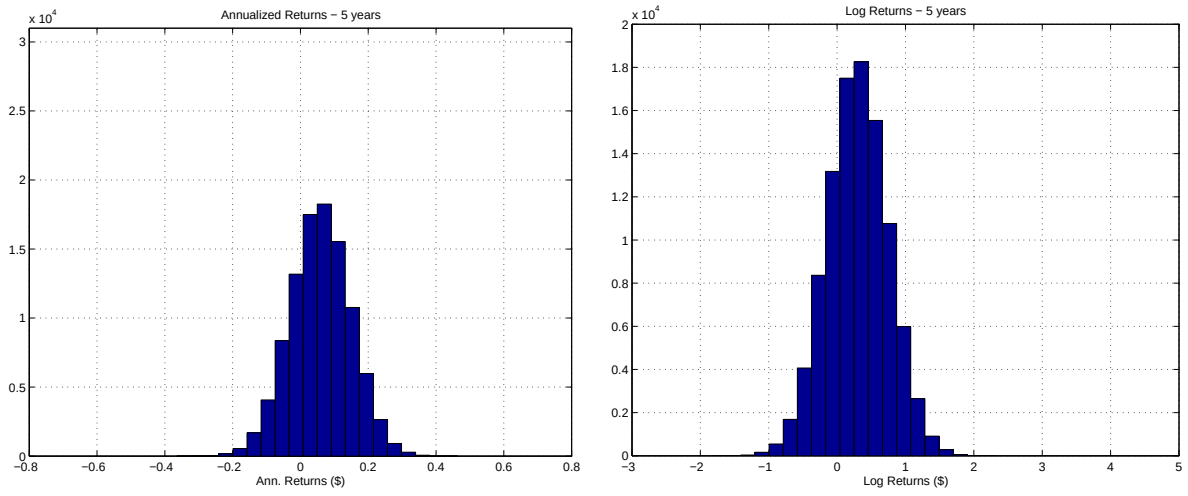


Figure 9.1.1: Histogram of distribution of returns $T = 5$ years. $\mu = .08, \sigma = .2$, 100,000 simulations. Left: annualized return $1/T \log[S(T)/S(0)]$. Right: return $\log[S(T)/S(0)]$.

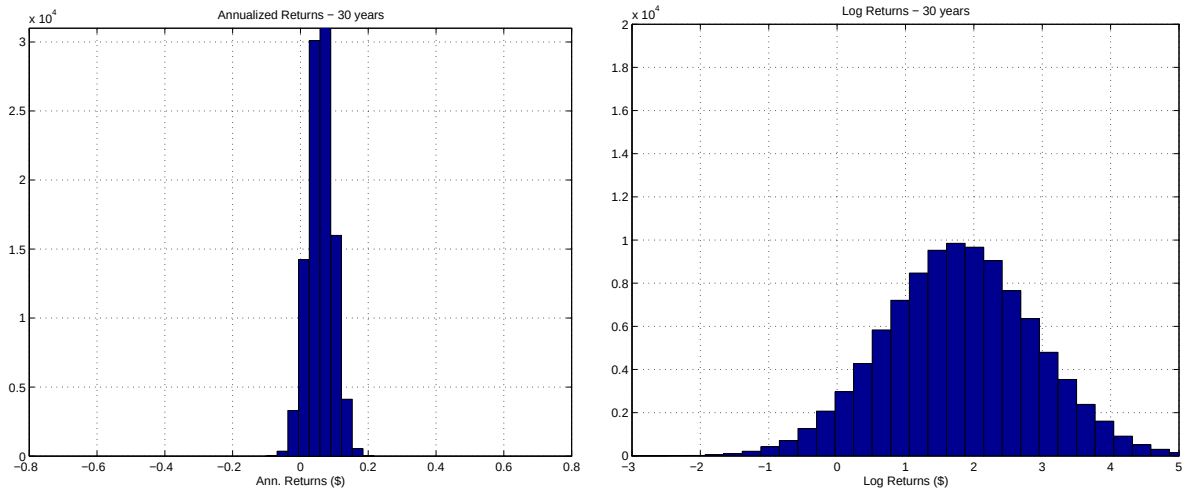


Figure 9.1.2: Histogram of distribution of returns $T = 30$ years. $\mu = .08, \sigma = .2$, 100,000 simulations. Left: annualized return $1/T \log[S(T)/S(0)]$. Right: return $\log[S(T)/S(0)]$,

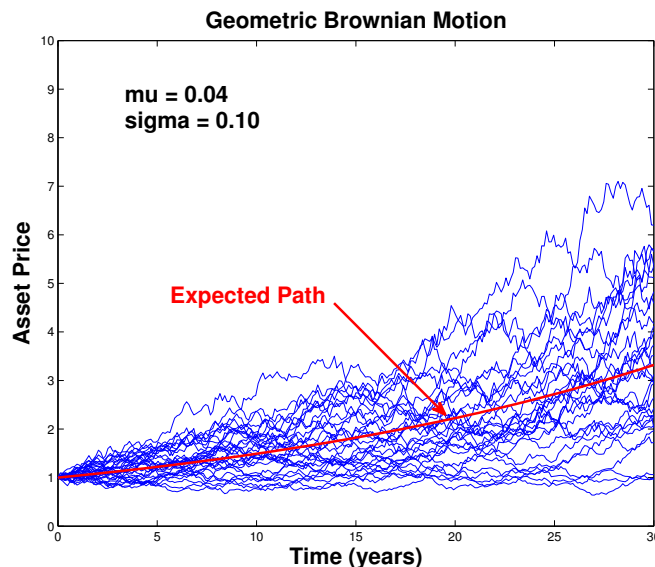


Figure 9.1.3: Thirty Monte Carlo realized Geometric Brownian Motion paths, compared with mean path. $\sigma = .10$, $\mu = .04$, $T = 30$ years.

α	$Pr(S(T)) < \alpha E[S(T)]$
1.0	0.61
0.9	0.53
0.8	0.45
0.7	0.35
0.6	0.25
0.5	0.16

Table 9.1.2: $T = 30$ years, $\sigma = 0.10$, $\mu = 0.04$. For example, there is a 25% chance that the realized value of $S(T)$ is less than 60% of the expected value.

GBM has a very large relative standard deviation for long times, even for moderate volatilities. From equations (1.7.35 - 1.7.38), we have

$$\begin{aligned} E[S] &= S_0 e^{\mu t} \\ \frac{\text{standard deviation}[S(t)]}{E[S(t)]} &= \sqrt{e^{\sigma^2 t} - 1} . \end{aligned} \quad (9.1.9)$$

Figure 9.1.3 shows some Monte Carlo simulations (30 realizations) along with the mean path. You can see that there is a large spread compared to the mean path. Figure 9.1.4 shows the probability density of $S(T)/E[S(T)]$.

Table 9.1.2 shows the probabilities of underperforming the mean path. This is something which financial planners never tell you. As well, there is standard method for determining the financial health of a pension plan called *going concern* valuation. Essentially, this method assumes that the plan investments will follow the mean path. You can see from table 9.1.2 that this is an extremely risky idea.

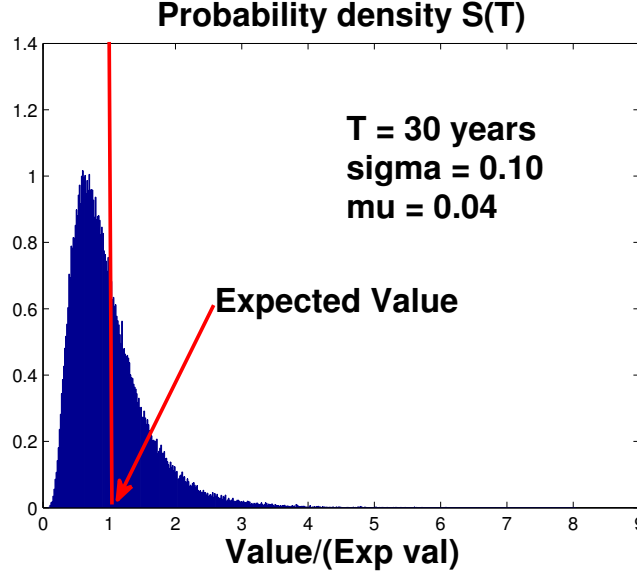


Figure 9.1.4: Probability density of $S(T)/E[S(T)]$. This is a log-normal density, which is highly skewed. $\sigma = .10$, $\mu = .04$, $T = 30$ years. The median of this distribution is always less than the mean.

9.2 Volatility Pumping

Consider an investment with value $S(t_i) = S_i$. We consider some discrete sets of times $t_1, \dots, t_n$. Let the value of the investment at t_{i+1} be $S_{i+1} = X_i S_i$. The total relative gain G^{tot} is

$$\begin{aligned} G^{tot} &= S_n / S_1 \\ &= X_1 X_2 \dots X_{n-1} . \end{aligned} \quad (9.2.1)$$

The average compound return A_{ret} is given by

$$\begin{aligned} A_{ret} &= \frac{1}{n-1} \log(S_n / S_1) \\ &= \frac{1}{n-1} \sum_{i=1}^{i=n-1} \log X_i \end{aligned} \quad (9.2.2)$$

Suppose the X_i are random, independent, and identically distributed. Then the expected average compound return is

$$\begin{aligned} E[A_{ret}] &= \frac{1}{n-1} \sum_{i=1}^{i=n-1} E[\log(X_i)] \\ &= E[\log(X)] \end{aligned} \quad (9.2.3)$$

where $E[\log(X)]$ is the expected value of $\log(X_i)$, which is the same for any period, since we have assumed that the gains X_i are independent and identically distributed. Suppose we have an investment a which has the gain distribution

$$\begin{aligned} X &= 2 ; \quad \text{with probability } \frac{1}{2} \\ &= \frac{1}{2} ; \quad \text{with probability } \frac{1}{2} \end{aligned} \quad (9.2.4)$$

The expected average compound return of a is

$$\begin{aligned} E[(A_{ret})^a] &= \frac{1}{2} \log(2) + \frac{1}{2} \log\left(\frac{1}{2}\right) \\ &= 0 \end{aligned} \quad (9.2.5)$$

Suppose we have investment b , has the gain distribution

$$X = 1 \quad ; \quad \text{with probability } 1 \quad . \quad (9.2.6)$$

Clearly b has expected average compound return of zero as well, i.e.

$$E[(A_{ret})^b] = 0 \quad (9.2.7)$$

However, consider the following strategy. At the beginning of each period, we divide our total wealth equally into investment a and investment b . This is a rebalancing strategy. Now, let's work out our expected average compound return $E[(A_{ret})^{Rebalance}]$

$$\begin{aligned} E[(A_{ret})^{Rebalance}] &= \frac{1}{2} \log\left(\frac{1}{2} + 1\right) + \frac{1}{2} \log\left(\frac{1}{2} + \frac{1}{4}\right) \\ &\simeq 6\% \quad . \end{aligned} \quad (9.2.8)$$

Let's summarize this result. We have constructed a portfolio consisting of two assets, a and b . Each of these assets has zero average compound return. However, the rebalancing strategy generates an average compound return of 6%. This is a bit surprising. This is the whole idea behind the well known strategy of frequently rebalancing a portfolio. This allows us to *buy low, sell high*, and generate a positive average compound return, even though the components of the portfolio may individually have zero expected average compound return.

We can formalize this idea in continuous time in the next section.

9.2.1 Constant Proportions Strategy

Suppose we have n assets $S_i; i = 1, \dots, n$ which follow

$$dS_i = \mu S_i dt + \sigma S_i dZ_i. \quad (9.2.9)$$

Suppose our investment strategy at time t is to invest $n_i(t)$ in each asset. Note that $n_i(t)$ is a function of time in general. As usual, we cannot peek into the future, i.e. we have to pick $n_i(t)$, and then let the asset evolve randomly in $t \rightarrow t + dt$. At $t + dt$, we can then rebalance our portfolio.

Let

$$P(t) = \sum_i n_i(t) S_i(t) \quad (9.2.10)$$

be the value of our portfolio at t . Since $P = P(S_1, \dots, S_n)$, we will need the multidimensional version of Ito's Lemma. To keep things general here, suppose that

$$\begin{aligned} dS_i &= a_i dt + b_i dZ_i \\ G &= G(t, S_1, \dots, S_n) \\ E[dZ_i dZ_k] &= \rho_{ik} dt \end{aligned} \quad (9.2.11)$$

where ρ_{ik} is the correlation between dZ_i and dZ_k . Then

$$dG = \left[G_t + \sum_k \frac{\partial G}{\partial S_k} a_k + \frac{1}{2} \sum_{k,m} \frac{\partial^2 G}{\partial S_k \partial S_m} b_k b_m \rho_{km} \right] dt + \sum_k \frac{\partial G}{\partial S_k} b_k dZ_k \quad (9.2.12)$$

We will be interested in the log return on our portfolio. Consider $G = \log P(t)$. Using Ito's Lemma (9.2.12) for the case where $a_i = \mu_i S_i$ and $b_i = \sigma_i S_i$, then

$$\begin{aligned} G_t &= 0 \\ G_{S_k} &= \frac{n_k}{P} \\ G_{S_k S_m} &= -\frac{n_k n_m}{P^2} . \end{aligned} \quad (9.2.13)$$

Combining equations (9.2.12-9.2.13), gives

$$dG = \left[\sum_k \frac{n_k \mu_k S_k}{P} - \frac{1}{2} \sum_{k,m} \frac{n_k n_m S_k S_m \sigma_k \sigma_m \rho_{km}}{P^2} \right] dt + \sum_k \frac{n_k n_k S_k \sigma_k}{P} dZ_k . \quad (9.2.14)$$

Now, suppose we choose a constant proportions strategy, i.e we rebalance at every time so that we have a constant weight w_i in each asset,

$$\begin{aligned} w_i &= \frac{n_i S_i}{P(t)} \\ \sum_i w_i &= 1 . \end{aligned} \quad (9.2.15)$$

Note that w_i is independent of t , since we rebalance at every opportunity. We can then write equation (9.2.14) as (using equation (9.2.15))

$$dG = \left[\sum_k w_k \mu_k - \frac{1}{2} \sum_{k,m} w_k w_m \sigma_k \sigma_m \rho_{km} \right] dt + \sum_k w_k \sigma_k dZ_k . \quad (9.2.16)$$

Equation (9.2.16) has the exact solution

$$\begin{aligned} G(t) &= G(0) + \\ &\quad \left[\sum_k w_k \mu_k - \frac{1}{2} \sum_{k,m} w_k w_m \sigma_k \sigma_m \rho_{km} \right] t + \sum_k w_k \sigma_k (Z_k(t) - Z_k(0)) . \end{aligned} \quad (9.2.17)$$

Now, let's look at what happens for the special case

$$\begin{aligned} \mu_i &= \mu ; \forall i \\ \sigma_i &= \sigma ; \forall i \\ w_i &= \frac{1}{n} ; \forall i . \end{aligned} \quad (9.2.18)$$

Recall that $G(t) = \log(P(t))$, so that, using equation (9.2.18), equation (9.2.17) becomes

$$\log\left(\frac{P(t)}{P(0)}\right) = \left[\mu - \sigma^2 \frac{1}{2n^2} \sum_{k,m} \rho_{km} \right] t + \frac{\sigma}{n} \sum_k (Z_k(t) - Z_k(0)) \quad (9.2.19)$$

so that

$$E\left[\log\left(\frac{P(t)}{P(0)}\right)\right] = \left[\mu - \frac{\sigma^2}{2} \left(\frac{1}{n^2} \sum_{k,m} \rho_{km} \right) \right] t . \quad (9.2.20)$$

As well,

$$\text{var}\left[\log\left(\frac{P(t)}{P(0)}\right)\right] = \left[\sigma^2 \left(\frac{1}{n^2} \sum_{k,m} \rho_{km} \right) \right] t . \quad (9.2.21)$$

Recall that for a single asset which follows Geometric Brownian Motion

$$dS = \mu S dt + \sigma S dZ \quad (9.2.22)$$

with constant μ, σ , then (from equation (1.7.26))

$$\log\left(\frac{S(t)}{S(0)}\right) = \left(\mu - \frac{\sigma^2}{2}\right)t + \sigma(Z(t) - Z(0)) \quad (9.2.23)$$

so that the mean and variance of the log return are

$$\begin{aligned} E\left[\log\left(\frac{S(t)}{S(0)}\right)\right] &= \left(\mu - \frac{\sigma^2}{2}\right)t \\ \text{var}\left[\log\left(\frac{S(t)}{S(0)}\right)\right] &= \sigma^2 t. \end{aligned} \quad (9.2.24)$$

Since

$$\frac{1}{n^2} \sum_{k,m} \rho_{km} \leq 1 \quad (9.2.25)$$

then, in this special case,

$$\begin{aligned} E\left[\log\left(\frac{P(t)}{P(0)}\right)\right] &\geq E\left[\log\left(\frac{S(t)}{S(0)}\right)\right] \\ \text{var}\left[\log\left(\frac{P(t)}{P(0)}\right)\right] &\leq \text{var}\left[\log\left(\frac{S(t)}{S(0)}\right)\right] \end{aligned} \quad (9.2.26)$$

so that in this case, using a constant weight portfolio, even if all the assets in the portfolio have the same drift, then the portfolio has a larger drift. Note that the variance of the portfolio is also usually reduced as well (the usual diversification result).

This enhanced drift is sometimes called *volatility pumping*, since the effect is large if the volatility is large. Note that for a single asset, the drift of the log return is reduced to $\mu - \frac{\sigma^2}{2}$ by the volatility. The constant weight portfolio reduces the effect of the volatility on the drift term.

9.2.2 Leveraged Two Times Bull/Bear ETFs

Recently, new ETFs (exchange traded fund) have been marketed which allow investors to receive

- Twice the daily return of the index (Bull version).
- Twice the daily negative return of the index (Bear version).

Are these ETFs a good idea? Here is an interesting fact. Suppose you hold equal amounts of both Bull and Bear versions. Then, after a finite period of time (say one year), what is your expected gain? You might think it would be zero. However, if the market is very volatile, you can end up losing on both ETFs.

We can use the analysis of Section 9.2.1 to examine these ETFs. The ETFs are actually an investment strategy which works as follows. Suppose you invest \$100 in the Bull version. The manager of the fund then borrows another \$100, and invests \$200 in the index, at the start of the trading day. The total position of the manager is $(200 - 100) = 100$. At the end the day, suppose the market goes up by 10%, this gives the manager a total position of $(220 - 100) = 120$. Assume you leave your money in the ETF for the following day. At the start of the next trading day, the manager rebalances, so that he has \$240 in the index, and a total borrowing of \$120, so that his total position is $(240 - 120) = 120$. Let's approximate this daily rebalancing by a continuous rebalancing, so we can use equation (9.2.17).

In this example, we have two assets, the risky index and a risk free bank account, which we assume grows at rate r . (This is unrealistic, no bank is going to loan the fund manager at the risk-free rate).

In this case

$$\begin{aligned}\mu_1 &= \mu \\ \mu_2 &= r \\ \sigma_1 &= \sigma \\ \sigma_2 &= 0 \\ \rho_{12} = \rho_{21} &= 0 .\end{aligned}\tag{9.2.27}$$

From equations (9.2.17) and (9.2.27) we have

$$\begin{aligned}E[\log(\text{Return})] &= (w_1\mu + w_2r - \frac{|w_1|^2\sigma^2}{2})t \\ std[\log(\text{Return})] &= (|w_1|\sigma)\sqrt{t} ,\end{aligned}\tag{9.2.28}$$

where w_1, w_2 are the weights in each asset, and we have

$$w_1 + w_2 = 1 .\tag{9.2.29}$$

For simplicity, let's let $t = 1$ in equation (9.2.28).

Now, let's suppose that

$$\mu = r + \lambda\sigma\tag{9.2.30}$$

where λ is the market price of risk. Let's examine the *Sharpe ratio* of this investment strategy. We will look at the excess return here, i.e. expected return in excess of the risk-free rate. Our Sharpe ratio in terms of excess return is (using equations (9.2.29 and 9.2.30))

$$\begin{aligned}S_{ratio} &= \frac{w_1\mu + w_2r - \frac{|w_1|^2\sigma^2}{2} - r}{|w_1|\sigma} \\ &= \frac{w_1\lambda\sigma - \frac{|w_1|^2\sigma^2}{2}}{|w_1|\sigma} \\ &= \text{sgn}(w_1)\lambda - \frac{|w_1|\sigma}{2} .\end{aligned}\tag{9.2.31}$$

Note that the Bear ETF ($w_1 = -2$) only makes sense if we assume that the drift is less than the risk-free rate, i.e. $\lambda < 0$. Now, suppose $w_1 = 2$, which is the Bull ETF strategy. This gives us

$$S_{ratio}^{Bull} = \lambda - \sigma .\tag{9.2.32}$$

Now, suppose we had an ETF (the Forsyth fund) which rebalanced daily with $w_1 = 1/2$. In this case, we have

$$S_{ratio}^{Forsyth} = \lambda - \frac{\sigma}{4}\tag{9.2.33}$$

which is always better than equation (9.2.32), especially if the volatility is high. In other words, you could achieve the same expected gain with a smaller standard deviation by borrowing to invest in the Forsyth Fund, compared to the Bull ETF.

Why to these Bull ETFs exist? Good question. I am sure it generates lots of fees for the fund managers.

9.3 More on Volatility Pumping

In the last section we considered optimality in terms of the continuously compounded return, i.e. $\log(P(t)/P(0))$. This is really a type of log utility function. This section is based on the paper “*A dual approach to portfolio evaluation: a comparison of the static, myopic and generalized buy and hold strategies,*” Quantitative Finance Vol 11 (2011) pages 81-99.

You can get further insights here by looking at the actual values of $P(t)/P(0)$. Let’s look at a simple case for equation (9.2.17), with

$$\mu_1 = \mu ; \mu_2 = r ; \sigma_1 = \sigma ; \sigma_2 = 0 ; \rho_{12} = \rho_{21} = 0 . \quad (9.3.1)$$

From equations (9.2.17) and (9.3.1) we get $(G(t) = \log(P(t)))$

$$G(t) = G(0) + (1 - w_1)rt + w_1\mu t - \frac{1}{2}w_1^2\sigma^2 t + w_1\sigma(Z_1(t) - Z_1(0)) . \quad (9.3.2)$$

Some Ito calculus also shows that

$$\frac{dP}{P} = \left((1 - w_1)r + w_1\mu \right) dt + w_1\sigma dZ_1 . \quad (9.3.3)$$

The exact solution to equation (9.3.2) is

$$\frac{P(t)}{P(0)} = e^{(1-w_1)rt+w_1\mu t} e^{-w_1^2\sigma^2 t/2} e^{w_1\sigma(Z_1(t)-Z_1(0))} . \quad (9.3.4)$$

Now

$$S_1(t) = S_1(0)e^{\mu t - \sigma^2/2t} e^{\sigma(Z_1(t)-Z_1(0))} . \quad (9.3.5)$$

or

$$\left(\frac{S_1(t)}{S_1(0)} \right)^{w_1} = e^{w_1\mu t - w_1\sigma^2/2t} e^{w_1\sigma(Z_1(t)-Z_1(0))} . \quad (9.3.6)$$

Substituting equation (9.3.6) into equation (9.3.4) gives

$$\frac{P(t)}{P(0)} = e^{(1-w_1)rt+w_1(1-w_1)\sigma^2 t/2} \left(\frac{S_1(t)}{S_1(0)} \right)^{w_1} . \quad (9.3.7)$$

Suppose $S_1(t) = S_1(0)$ (i.e. a sideways market), and $r = 0$, then equation (9.3.7) gives

$$\frac{P(t)}{P(0)} = e^{\sigma^2 t/2w_1(1-w_1)} . \quad (9.3.8)$$

This means that in a sideways market, we will get a positive return if $0 < w_1 < 1$ (the pumping effect). On the other hand, if $w_1 < 0$ or $w_1 > 1$, we will get a negative return (e.g. leveraged ETFs).

It is also interesting to compare equation (9.3.7) with a static portfolio, where we take our initial wealth $P(0)$ and buy $w_1P(0)/S_1(0)$ shares of asset 1, and put $(1 - w_1)P(0)$ into the risk free bond (asset 2). In this case, the static or buy and hold portfolio will yield

$$\frac{P(t)}{P(0)} = (1 - w_1)e^{rt} + w_1 \frac{S_1(t)}{S_1(0)} . \quad (9.3.9)$$

T	1.0
σ	.4
μ (real world drift)	.10
P_0 (initial wealth)	100
r	.02
rebalancing frequency	250
initial cash	100
initial risky asset position α_0	0

Table 9.4.1: Data used in the CPPI strategy experiments.

F	M	Mean	Standard Deviation	VAR (95%)	CVAR (95%)
0	1	0.021876	0.39973	-0.63697	-0.80533
0	.5	0.04094	0.19986	-0.28857	-0.3724
0	2	-0.13662	0.80046	-1.4576	-1.7951
80	2	.037598	0.1809	-0.1563	-0.17154
80	4	0.0083732	.33117	-0.21038	-0.2142

Table 9.4.2: Results in terms of the log return (9.4.3), data in Table 9.4.1. 80,000 Monte Carlo simulations.

9.4 Constant Proportion Portfolio Insurance

Another idea for providing some sort of insurance on a portfolio is Constant Proportion Portfolio Insurance (CPPI). In its simplest form, we can consider the possibility of investing in a risky asset (with price S) and a risk free bond (price B). If $B < 0$, this represents borrowing. Consider a set of rebalancing times t_i , with $\Delta t = t_{i+1} - t_i$. Let

$$\begin{aligned}
B(t_i) = B_i &= \text{Amount in risk free asset at } t_i \\
S(t_i) = S_i &= \text{Price of risky asset at } t_i \\
\alpha(t_i) = \alpha_i &= \text{Number of units of risky asset at } t_i \\
P(t_i) = P_i &= \text{total portfolio at } t_i \\
M(t_i) = M_i &= \text{CPPI multiplier at } t_i \\
F(t_i) = F_i &= \text{CPPI floor at } t_i .
\end{aligned} \tag{9.4.1}$$

A CPPI strategy is determined by specifying a floor F_i and a multiplier M_i . The state of the strategy is given by α_i and B_i . At each rebalancing date t_{i+1} the following reallocation takes place

$$\begin{aligned}
\alpha_{i+1} &= M_{i+1} \left[\frac{\max(0, B_i e^{r\Delta t} + \alpha_i S_{i+1} - F_{i+1})}{S_{i+1}} \right] \\
B_{i+1} &= B_i e^{r\Delta t} - (\alpha_{i+1} - \alpha_i) S_{i+1}
\end{aligned} \tag{9.4.2}$$

Usually $M_i > 1$, so that the strategy becomes more invested in the risky asset as the total wealth ($B_i e^{r\Delta t} + \alpha_i S_{i+1}$) increases above the floor. As the wealth decreases, more assets are diverted to the risk free bond, hence the floor is the lowest possible value of the portfolio (if the rebalancing interval is infinitesimal).

Consider the case where $M_i = M = \text{const.}$, $F_i = F = \text{const.}$. Then $M = 2, F = 0$ corresponds to the Bull ETF. A more conservative constant proportions strategy might be $M = .5, F = 0$. A typical CPPI strategy would be $F = .8S_0, M = 2$. Consider the data in Table 9.4.1.

We will measure the performance of these strategies by examining the statistics of the log return

$$R = \log(P(T)/P(0)) . \tag{9.4.3}$$

Table 9.4.2 gives the results for different choices of M, F . Table 9.4.2 indicates that $F = 80, M = 2$ is a good strategy. The leveraged Bull ETF is very bad.

Figure 9.4.1 shows the density of the log return for the strategy with $F = 80, M = 2$. Note the highly skewed distribution.

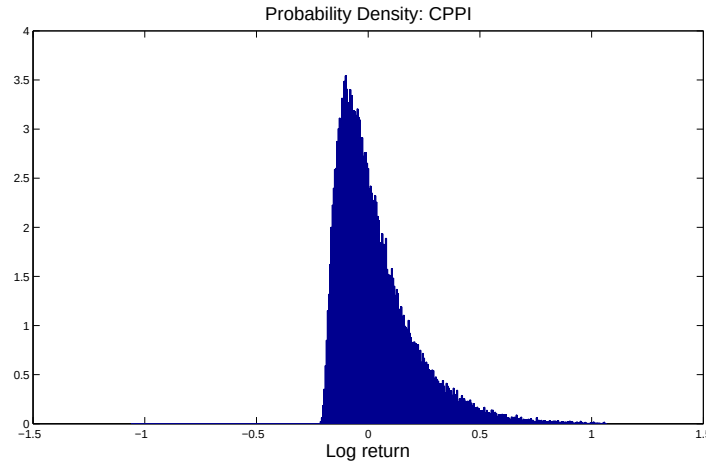


Figure 9.4.1: Probability density (y-axis) of the log return (9.4.3), $F = 80, M = 2$.

9.4.1 Covered Call Writing

Another popular strategy is covered call writing. There are now some ETFs which follow this strategy.

Basically, you own the stock and sell a call on the stock. The optimal strategy turns out to be to sell a call with a strike just above the current stock price. Using the parameters in Table 9.4.1, this strategy was simulated with a strike of $K = 100.1$. The results are shown in Table 9.4.3. The probability density of the log return is shown in Figure 9.4.2. Note the highly skewed distribution.

If $C(t)$ is the call price, and $P(t)$ is the put price, then Put-Call parity for European options is

$$C(t) - P(t) + Ke^{-r(T-t)} = S. \quad (9.4.4)$$

A covered call is $S - C(t)$, which is equivalent to

$$S - C(t) = Ke^{-r(T-t)} - P(t), \quad (9.4.5)$$

which is long the risk free account $Ke^{-r(T-t)}$ and short a put. In other words, owning stock worth K and selling a call with strike K is equivalent to investing K in a risk free account and selling a put. This is a bet that the market won't go down, and hence the investor gains the insurance premium (the put). This results in the highly skewed return distribution.

Covered call writing is often justified by *back testing*, i.e. seeing what would have happened by applying the strategy to historical market data. This amounts to examining a single stochastic price path. From the probability density shown in Figure 9.4.2, we can see that the most frequently observed outcome (the highest value on the graph) is quite good. This is, of course, not the mean outcome, and does not take into account the tail of the plot. This explains why backtesting strategies which are very skewed often suggests that these strategies work well (since the historically observed single stochastic path is most likely one where nothing bad happens).

In "*Derivatives in portfolio management: Why beating the market is easy*" (F-S Lhabitant, EDHEC Business School, 2000), the author suggests that use of covered call writing allows fund managers to pretend they are providing superior returns in terms of Sharpe ratios. This paper suggests that this is simply due to the inadequacy of the Sharpe ratio in this situation, since the return distribution is far from normal.

Mean	Standard Deviation	VAR (95%)	CVAR (95%)
0.034853	0.18239	-0.3574	-0.48052

Table 9.4.3: Results in terms of the log return (9.4.3), data in Table 9.4.1. 80,000 Monte Carlo simulations. Covered call, $K = 100.1$.

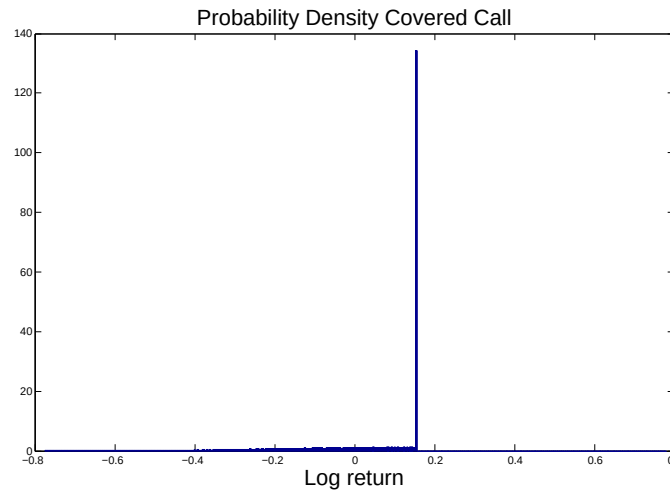


Figure 9.4.2: Probability density (y-axis) of the log return (9.4.3), covered call strategy. Data in Table 9.4.1. 80,000 Monte Carlo simulations. Covered call, $K = 100.1$.

T	1.0
P_0 (initial wealth)	100
r	.02
rebalancing frequency	Every 3 hours
initial cash	100
initial risky asset position	0
Stop loss price trigger K	90

Table 9.4.4: Data used in the stop-loss start-gain strategy experiments. K is the stock price stop-loss trigger. K is the strike price of the European put if the Buy-Put strategy is used.

σ	μ	Mean	Standard Deviation	VAR (95%)	CVAR (95%)
Pure Stop-loss					
.2	.10	0.062856	0.18242	-0.10453	-0.10807
.15	.10	0.077945	0.14964	-0.10252	-0.10585
Stop-loss start-gain					
.2	.10	0.073376	0.19646	-0.21016	-0.26076
.15	.10	0.085093	0.15126	-0.15222	-0.18997
Buy put					
.2	.10	0.068787	0.16958	-0.13479	-0.13545
.15	.10	0.079694	0.13723	-0.12048	-0.12095

Table 9.4.5: Comparison of stop-loss start-gain with buy put strategies. Results in terms of the log return of the portfolio. Data in Table 9.4.4. 80,000 Monte Carlo simulations. GBM with volatility σ , real world drift μ . Buy-Put strategy includes the cost of buying the put in the portfolio return.

9.4.2 Stop Loss, Start Gain

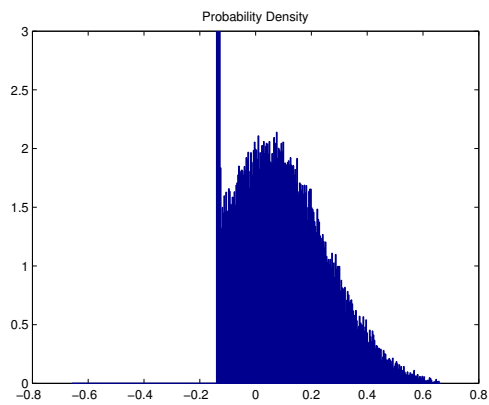
A simple idea for providing some sort of portfolio insurance is to use *stop loss* triggers. This can be set automatically through an online broker. The idea is to set a floor price K , and sell the asset if the stock price S falls below K . Of course, there is no guarantee that the order will be executed at K . What happens is that a *market order* is submitted if the price drops below K and the stock is then sold at whatever bid price appears on the order book. If the stop-loss is triggered, we assume that the investor simply holds the cash in a risk free account until the end of the investment period. We call this the pure stop-loss strategy.

Similarly, if one believes that this is a good investment, one could couple this with a *start-gain*. Once the stock has dropped below K and has been sold, the investor could then set a start-gain order. If the stock rises above K , then a market order is submitted, and the stock is purchased at whatever ask price appears on the order book.

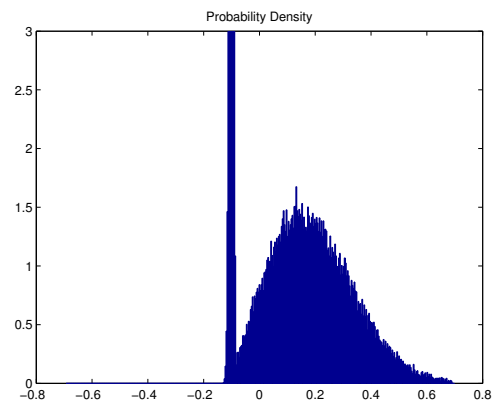
Alternatively, one could simply buy a European put option with strike K . This would guarantee that the terminal value of the stock holding would never drop below K . In other words, we take out initial investment, buy a put, and then invest whatever is left over in the stock. However, this strategy is psychologically unappealing, since we have to pay for the put, which represents an immediately realized cost, and this would reduce the return on the portfolio.

Of course, the stop-loss start-gain strategy accumulates costs since we always sell low ($K - \epsilon$) and buy high ($K + \epsilon$).

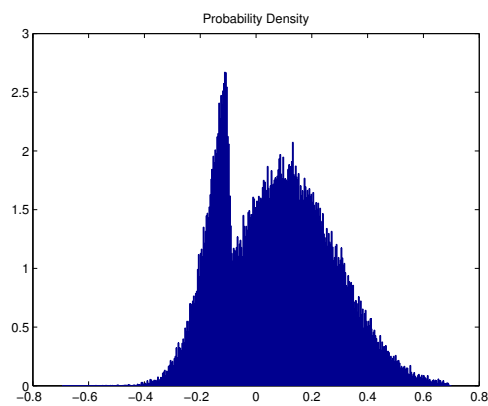
As an example, we will assume GBM, with the parameters given in Table 9.4.4. The stop-loss start-gain strategy is compared with a buy put strategy in Table 9.4.5. The density of this strategy is highly skewed, but the buy-put strategy definitely protects better against extreme losses. Stop-loss start-gain strategies are negatively affected by volatility.



(a) Buy Put



(b) Pure Stop-Loss



(c) Stop-Loss Start-Gain

Figure 9.4.3: Densities of the log return, Buy Put, Pure Stop-Loss and Stop-Loss Start-Gain. Data in Table 9.4.4. $\sigma = .20$.

Chapter 10

Jump Diffusion

Recall that if

$$dS = \mu S dt + \sigma S dZ \quad (10.0.1)$$

then from Ito's Lemma we have

$$d[\log S] = \left[\mu - \frac{\sigma^2}{2}\right] dt + \sigma dZ. \quad (10.0.2)$$

Now, suppose that we observe asset prices at discrete times t_i , i.e. $S(t_i) = S_i$, with $\Delta t = t_{i+1} - t_i$. Then from equation (10.0.2) we have

$$\begin{aligned} \log S_{i+1} - \log S_i &= \log\left(\frac{S_{i+1}}{S_i}\right) \\ &\simeq \left[\mu - \frac{\sigma^2}{2}\right] \Delta t + \sigma \phi \sqrt{\Delta t} \end{aligned} \quad (10.0.3)$$

where ϕ is $N(0,1)$. Now, if Δt is sufficiently small, then Δt is much smaller than $\sqrt{\Delta t}$, so that equation (10.0.3) can be approximated by

$$\begin{aligned} \log\left(\frac{S_{i+1} - S_i + S_i}{S_i}\right) &= \log\left(1 + \frac{S_{i+1} - S_i}{S_i}\right) \\ &\simeq \sigma \phi \sqrt{\Delta t}. \end{aligned} \quad (10.0.4)$$

Define the return R_i in the period $t_{i+1} - t_i$ as

$$R_i = \frac{S_{i+1} - S_i}{S_i} \quad (10.0.5)$$

so that equation (10.0.4) becomes

$$\log(1 + R_i) \simeq R_i = \sigma \phi \sqrt{\Delta t}.$$

Consequently, a plot of the discretely observed returns of S should be normally distributed, if the assumption (10.0.1) is true. In Figure 10.0.1 we can see a histogram of monthly returns from the S&P500 for the period 1982 – 2002. The histogram has been scaled to zero mean and unit standard deviation. A standard normal distribution is also shown. Note that for real data, there is a higher peak, and fatter tails than the normal distribution. This means that there is higher probability of zero return, or a large gain or loss compared to a normal distribution.

As $\Delta t \rightarrow 0$, Geometric Brownian Motion (equation (10.0.1)) assumes that the probability of a large return also tends to zero. The amplitude of the return is proportional to $\sqrt{\Delta t}$, so that the tails of the distribution become unimportant.

But, in real life, we can sometimes see very large returns (positive or negative) in small time increments. It therefore appears that Geometric Brownian Motion (GBM) is missing something important.

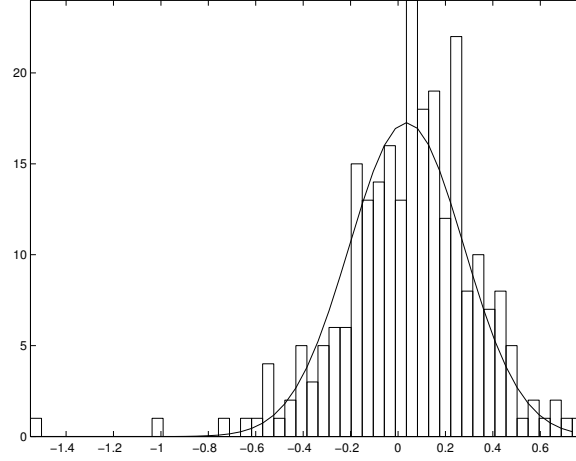


Figure 10.0.1: Probability density functions for the S&P 500 monthly returns 1982 – 2002, scaled to zero mean and unit standard deviation and the standardized Normal distribution.

10.1 The Poisson Process

Consider a process where most of the time nothing happens (contrast this with Brownian motion, where some changes occur at any time scale we look at), but on rare occasions, a jump occurs. The jump size does not depend on the time interval, but the probability of the jump occurring does depend on the interval.

More formally, consider the process dq where, in the interval $[t, t + dt]$,

$$\begin{aligned} dq &= 1 ; && \text{with probability } \lambda dt \\ &= 0 ; && \text{with probability } 1 - \lambda dt. \end{aligned} \quad (10.1.1)$$

Note, once again, that size of the Poisson outcome does not depend on dt . Also, the probability of a jump occurring in $[t, t + dt]$ goes to zero as $dt \rightarrow 0$, in contrast to Brownian motion, where some movement always takes place (the probability of movement is constant as $dt \rightarrow 0$), but the size of the movement tends to zero as $dt \rightarrow 0$. For future reference, note that

$$\begin{aligned} E[dq] &= \lambda dt \cdot 1 + (1 - \lambda dt) \cdot 0 \\ &= \lambda dt \end{aligned} \quad (10.1.2)$$

and

$$\begin{aligned} \text{Var}(dq) &= E[(dq - E[dq])^2] \\ &= E[(dq - \lambda dt)^2] \\ &= (1 - \lambda dt)^2 \cdot \lambda dt + (0 - \lambda dt)^2 \cdot (1 - \lambda dt) \\ &= \lambda dt + O((dt)^2). \end{aligned} \quad (10.1.3)$$

Now, suppose we assume that, along with the usual GBM, occasionally the asset jumps, i.e. $S \rightarrow JS$, where J is the size of a (proportional) jump. We will restrict J to be non-negative.

Suppose a jump occurs in $[t, t + dt]$, with probability λdt . Let's write this jump process as an SDE, i.e.

$$[dS]_{\text{jump}} = (J - 1)S dq$$

since, if a jump occurs

$$\begin{aligned} S_{\text{after jump}} &= S_{\text{before jump}} + [dS]_{\text{jump}} \\ &= S_{\text{before jump}} + (J - 1)S_{\text{before jump}} \\ &= JS_{\text{before jump}} \end{aligned} \quad (10.1.4)$$

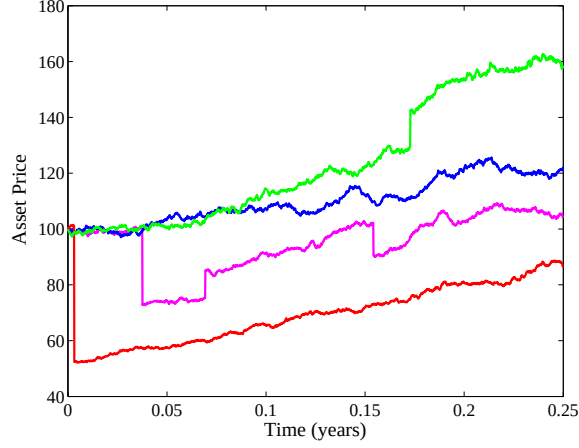


Figure 10.1.1: Some realizations of a jump diffusion process which follows equation (10.1.5).



Figure 10.1.2: Actual price of a drug company stock. Compare with simulation of a jump diffusion in Figure 10.1.1.

which is what we want to model. So, if we have a combination of GBM and a rare jump event, then

$$dS = \mu S dt + \sigma S dZ + (J - 1)S dq \quad (10.1.5)$$

Assume that the jump size has some known probability density $g(J)$, i.e. given that a jump occurs, then the probability of a jump in $[J, J + dJ]$ is $g(J) dJ$, and

$$\int_{-\infty}^{+\infty} g(J) dJ = \int_0^{\infty} g(J) dJ = 1 \quad (10.1.6)$$

since we assume that $g(J) = 0$ if $J < 0$. For future reference, if $f = f(J)$, then the expected value of f is

$$E[f] = \int_0^{\infty} f(J)g(J) dJ. \quad (10.1.7)$$

The process (10.1.5) is basically geometric Brownian motion (a continuous process) with rare discontinuous jumps. Some example realizations of jump diffusion paths are shown in Figure 10.1.1.

Figure 10.1.2 shows the price followed by a listed drug company. Note the extreme price changes over very small periods of time.

10.2 The Jump Diffusion Pricing Equation

Now, form the usual hedging portfolio

$$P = V - \alpha S. \quad (10.2.1)$$

Now, consider

$$[dP]_{total} = [dP]_{Brownian} + [dP]_{jump} \quad (10.2.2)$$

where, from Ito's Lemma

$$[dP]_{Brownian} = [V_t + \frac{\sigma^2 S^2}{2} V_{SS}]dt + [V_S - \alpha](\mu S dt + \sigma S dZ) \quad (10.2.3)$$

and, noting that the jump is of finite size,

$$[dP]_{jump} = [V(JS, t) - V(S, t)] dq - \alpha(J - 1)S dq . \quad (10.2.4)$$

If we hedge the Brownian motion risk, by setting $\alpha = V_S$, then equations (10.2.2-10.2.4) give us

$$dP = [V_t + \frac{\sigma^2 S^2}{2} V_{SS}]dt + [V(JS, t) - V(S, t)]dq - V_S(J - 1)S dq . \quad (10.2.5)$$

So, we still have a random component (dq) which we have not hedged away. Let's take the expected value of this change in the portfolio, e.g.

$$E(dP) = [V_t + \frac{\sigma^2 S^2}{2} V_{SS}]dt + E[V(JS, t) - V(S, t)]E[dq] - V_S S E[J - 1]E[dq] \quad (10.2.6)$$

where we have assumed that probability of the jump and the probability of the size of the jump are independent. Defining $E(J - 1) = \kappa$, then we have that equation (10.2.6) becomes

$$E(dP) = [V_t + \frac{\sigma^2 S^2}{2} V_{SS}]dt + E[V(JS, t) - V(S, t)]\lambda dt - V_S S \kappa \lambda dt . \quad (10.2.7)$$

Now, we make a rather interesting assumption. Assume that an investor holds a diversified portfolio of these hedging portfolios, for many different stocks. If we make the rather dubious assumption that these jumps for different stocks are uncorrelated, then the variance of this portfolio of portfolios is small, hence there is little risk in this portfolio. Hence, the expected return should be

$$E[dP] = rP dt . \quad (10.2.8)$$

Now, equating equations (10.2.7 and (10.2.8) gives

$$V_t + \frac{\sigma^2 S^2}{2} V_{SS} + V_S[rS - S\kappa\lambda] - (r + \lambda)V + E[V(JS, t)]\lambda = 0 . \quad (10.2.9)$$

Using equation (10.1.7) in equation (10.2.9) gives

$$V_t + \frac{\sigma^2 S^2}{2} V_{SS} + V_S[rS - S\kappa\lambda] - (r + \lambda)V + \lambda \int_0^\infty g(J)V(JS, t) dJ = 0 . \quad (10.2.10)$$

Equation (10.2.10) is a Partial Integral Differential Equation (PIDE).

A common assumption is to assume that $g(J)$ is log normal,

$$g(J) = \frac{\exp\left(-\frac{(\log(J) - \hat{\mu})^2}{2\gamma^2}\right)}{\sqrt{2\pi}\gamma J} . \quad (10.2.11)$$

where, some algebra shows that

$$E(J - 1) = \kappa = \exp(\hat{\mu} + \gamma^2/2) - 1 . \quad (10.2.12)$$

Now, what about our dubious assumption that jump risk was diversifiable? In practice, we can regard $\sigma, \hat{\mu}, \gamma, \lambda$ as parameters, and fit them to observed option prices. If we do this, (see L. Andersen and J. Andreasen, *Jump-Diffusion processes: Volatility smile fitting and numerical methods*, Review of Derivatives Research (2002), vol 4, pages 231-262), then we find that σ is close to historical volatility, but that the fitted values of $\lambda, \hat{\mu}$ are at odds with the historical values. The fitted values seem to indicate that investors are pricing in larger more frequent jumps than has been historically observed. In other words, actual prices seem to indicate that investors do require some compensation for jump risk, which makes sense. In other words, these parameters contain a market price of risk.

Consequently, our assumption about jump risk being diversifiable is not really a problem if we fit the jump parameters from market (as opposed to historical) data, since the market-fit parameters will contain some effect due to risk preferences of investors.

One can be more rigorous about this if you assume some utility function for investors. See (Alan Lewis, *Fear of jumps*, Wilmott Magazine, December, 2002, pages 60-67) or (V. Naik, M. Lee, *General equilibrium pricing of options on the market portfolio with discontinuous returns*, The Review of Financial Studies, vol 3 (1990) pages 493-521.)

10.3 Alternate Derivation of the Jump Diffusion Pricing Equation

We will give a pure hedging argument in this section, in order to derive the PIDE for jump diffusion. Initially, we suppose that there is only one possible jump size J , i.e. after a jump, $S \rightarrow JS$, where J is a known constant. Suppose

$$dS = a(S, t)dt + b(S, t)dZ + (J - 1)S dq, \quad (10.3.1)$$

where dq is the Poisson process. Consider a contract on S , $F(S, t)$, then

$$dF = \left[aF_S + \frac{b^2}{2}F_{SS} + F_t \right] dt + bF_S dZ + [F(JS, t) - F(S, t)] dq, \quad (10.3.2)$$

or, in more compact notation

$$\begin{aligned} dF &= \mu dt + \sigma^* dZ + \Delta F dq \\ \mu &= aF_S + \frac{b^2}{2}F_{SS} + F_t \\ \sigma^* &= bF_S \\ \Delta F &= [F(JS, t) - F(S, t)] . \end{aligned} \quad (10.3.3)$$

Now, instead of hedging with the underlying asset, we will hedge one contract with another. Suppose we have three contracts F_1, F_2, F_3 (they could have different maturities for example).

Consider the portfolio Π

$$\Pi = n_1 F_1 + n_2 F_2 + n_3 F_3 \quad (10.3.4)$$

so that

$$\begin{aligned} d\Pi &= n_1 dF_1 + n_2 dF_2 + n_3 dF_3 \\ &= n_1(\mu_1 dt + \sigma_1^* dZ + \Delta F_1 dq) \\ &\quad + n_2(\mu_2 dt + \sigma_2^* dZ + \Delta F_2 dq) \\ &\quad + n_3(\mu_3 dt + \sigma_3^* dZ + \Delta F_3 dq) \\ &= (n_1\mu_1 + n_2\mu_2 + n_3\mu_3) dt \\ &\quad + (n_1\sigma_1^* + n_2\sigma_2^* + n_3\sigma_3^*) dZ \\ &\quad + (n_1\Delta F_1 + n_2\Delta F_2 + n_3\Delta F_3) dq . \end{aligned} \quad (10.3.5)$$

Eliminate the random terms by setting

$$\begin{aligned}(n_1\Delta F_1 + n_2\Delta F_2 + n_3\Delta F_3) &= 0 \\ (n_1\sigma_1^* + n_2\sigma_2^* + n_3\sigma_3^*) &= 0 .\end{aligned}\tag{10.3.6}$$

This means that the portfolio is riskless, hence

$$d\Pi = r\Pi dt ,\tag{10.3.7}$$

hence (using equations (10.3.5-10.3.7))

$$(n_1\mu_1 + n_2\mu_2 + n_3\mu_3) = (n_1F_1 + n_2F_2 + n_3F_3)r .\tag{10.3.8}$$

Putting together equations (10.3.6) and (10.3.8), we obtain

$$\begin{bmatrix} \sigma_1^* & \sigma_2^* & \sigma_3^* \\ \Delta F_1 & \Delta F_2 & \Delta F_3 \\ \mu_1 - rF_1 & \mu_2 - rF_2 & \mu_3 - rF_3 \end{bmatrix} \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} .\tag{10.3.9}$$

Equation (10.3.9) has a nonzero solution only if the rows are linearly dependent. There must be $\lambda_B(S, t), \lambda_J(S, t)$ such that

$$\begin{aligned}(\mu_1 - rF_1) &= \lambda_B\sigma_1^* - \lambda_J\Delta F_1 \\ (\mu_2 - rF_2) &= \lambda_B\sigma_2^* - \lambda_J\Delta F_2 \\ (\mu_3 - rF_3) &= \lambda_B\sigma_3^* - \lambda_J\Delta F_3 .\end{aligned}\tag{10.3.10}$$

(We will show later on that $\lambda_J \geq 0$ to avoid arbitrage). Dropping subscripts, we have

$$(\mu - rF) = \lambda_B\sigma^* - \lambda_J\Delta F\tag{10.3.11}$$

and substituting the definitions of $\mu, \sigma^*, \Delta F$, from equations (10.3.3), we obtain

$$F_t + \frac{b^2}{2}F_{SS} + (a - \lambda_B b)F_S - rF + \lambda_J[F(JS, t) - F(S, t)] = 0 .\tag{10.3.12}$$

Note that λ_J will not be the real world intensity of the Poisson process, but J will be the real world jump size.

In the event that, say, $F_3 = S$ is a traded asset, we note that in this case

$$\begin{aligned}\sigma_3^* &= b \\ \mu_3 &= a \\ \Delta F_3 &= (J - 1)S .\end{aligned}\tag{10.3.13}$$

From equation (10.3.10) we have that

$$(\mu_3 - rF_3) = \lambda_B\sigma_3^* - \lambda_J\Delta F_3 ,\tag{10.3.14}$$

or, using equation (10.3.13),

$$a - \lambda_B b = rS - \lambda_J(J - 1)S .\tag{10.3.15}$$

Substituting equation (10.3.15) into equation (10.3.12) gives

$$F_t + \frac{b^2}{2}F_{SS} + [r - \lambda_J(J - 1)]SF_S - rF + \lambda_J[F(JS, t) - F(S, t)] = 0 .\tag{10.3.16}$$

Note that equation (10.3.12) is valid if the underlying asset cannot be used to hedge, while equation (10.3.16) is valid only if the underlying asset can be used as part of the hedging portfolio.

Let $\tau = T - t$, and set $a = 0, b = 0, r = 0$ in equation (10.3.12), giving

$$F_\tau = \lambda_J [F(JS, \tau) - F(S, \tau)] . \quad (10.3.17)$$

Now, suppose that

$$\begin{aligned} F(S, \tau = 0) &= 0 ; & \text{if } S \geq K \\ &= 1 ; & \text{if } S < K . \end{aligned} \quad (10.3.18)$$

Now, consider the asset value $S^* > K$, and let $J = K/(2 * S^*)$. Imagine solving equation (10.3.17) to an infinitesimal time $\tau = \epsilon \ll 1$. We will obtain the following value for F ,

$$F(S^*, \epsilon) \simeq \epsilon \lambda_J . \quad (10.3.19)$$

Since the payoff is nonnegative, we must have $\lambda_J \geq 0$ to avoid arbitrage.

Now, suppose that there are a finite number of jump states, i.e. after a jump, the asset may jump to to any state $J_i S$

$$S \rightarrow J_i S ; \quad i = 1, \dots, n . \quad (10.3.20)$$

Repeating the above arguments, now using $n + 2$ hedging instruments in the hedging portfolio

$$\Pi = \sum_{i=1}^{i=n+2} n_i F_i \quad (10.3.21)$$

so that the diffusion and jumps are hedged perfectly, we obtain the following PDE

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_B b) F_S - rF + \sum_{i=1}^{i=n} \lambda_J^i [F(J_i S, t) - F(S, t)] = 0 . \quad (10.3.22)$$

If we can use the underlying to hedge, then we get the analogue of equation (10.3.16)

$$F_t + \frac{b^2}{2} F_{SS} + (rS - \sum_{i=1}^{i=n} \lambda_J^i S(J_i - 1)) F_S - rF + \sum_{i=1}^{i=n} \lambda_J^i [F(J_i S, t) - F(S, t)] = 0 . \quad (10.3.23)$$

Now, let

$$\begin{aligned} p(J_i) &= \frac{\lambda_J^i}{\sum_{i=1}^{i=n} \lambda_J^i} \\ \lambda^* &= \sum_{i=1}^{i=n} \lambda_J^i \end{aligned} \quad (10.3.24)$$

then we can write equation (10.3.22) as

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_B b) F_S - rF + \lambda^* \sum_{i=1}^{i=n} p(J_i) [F(J_i S, t) - F(S, t)] = 0 . \quad (10.3.25)$$

Note that since $\lambda_J^i \geq 0$, $p(J_i) \geq 0$, and $\lambda^* \geq 0$.

Taking the limit as the number of jump states tends to infinity, then $p(J)$ tends to a continuous distribution, so that equation (10.3.25) becomes

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_B b) F_S - rF + \lambda^* \int_0^\infty p(J) [F(JS, t) - F(S, t)] dJ = 0 . \quad (10.3.26)$$

It is convenient to rewrite equation (10.3.26) in a slightly different form. Suppose we redefine λ_B as follows

$$\lambda_B = \lambda'_B + \lambda^* \frac{E[J-1]S}{b} \quad (10.3.27)$$

where

$$E[J-1] = \int_0^\infty p(J)(J-1) dJ . \quad (10.3.28)$$

Substituting equation (10.3.27) into equation (10.3.26) gives

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda'_B b - \lambda^* E[J-1]S) F_S - rF + \lambda^* \int_0^\infty p(J)[F(JS, t) - F(S, t)] dJ = 0 . \quad (10.3.29)$$

Note that in the case that $F_{SS} = 0$ (which would normally be the case for $S \rightarrow \infty$), then equation (10.3.29) reduces to

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_B b) F_S - rF = 0 , \quad (10.3.30)$$

so that the term $\lambda^* E[J-1]S$ in the drift term cancels the integral term, leaving the equation independent of λ^* . This is very convenient for applying numerical boundary conditions. The PIDE (10.3.29) can also be written as

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda'_B b) F_S - rF + \lambda^* \int_0^\infty p[J][F(JS, t) - F(S, t) - (J-1)SF_S] dJ = 0 \quad (10.3.31)$$

which is valid for infinite activity processes.

In the case where we can hedge with the underlying asset S , we obtain

$$F_t + \frac{b^2}{2} F_{SS} + (r - \lambda^* E[J-1])SF_S - rF + \lambda^* \int_0^\infty p(J)[F(JS, t) - F(S, t)] dJ = 0 . \quad (10.3.32)$$

Note that λ^* and $p(J)$ are not the real arrival rate and real jump size distributions, since they are based on hedging arguments which eliminate the risk. Consequently, λ^* , $p(J)$ must be obtained by calibration to market data.

10.4 Simulating Jump Diffusion

Let's rewrite equation (10.1.5) as

$$\frac{dS_t}{S_{t-}} = \mu dt + \sigma dZ + (J_t - 1) dq . \quad (10.4.1)$$

where

$$\begin{aligned} dq &= 1 ; & \text{with probability } \lambda dt \\ &= 0 ; & \text{with probability } 1 - \lambda dt. \end{aligned} \quad (10.4.2)$$

and where S_{t-} denotes the value of S immediately before a possible jump occurs, S_t is the value immediately after a jump occurs, and J_t represents a possible jump size at time t .

In other words, if a jump occurs at time t , then equation (10.4.1) can be interpreted as

$$S_t = S_{t-} J_t . \quad (10.4.3)$$

It will be easier to consider $X_t = \log S_t$. Suppose no jump occurs in $[t, t + dt]$, then equation (10.4.1) can be written as (using Ito's Lemma)

$$dX_t = (\mu - \sigma^2/2) dt + \sigma dZ . \quad (10.4.4)$$

If a jump occurs in $[t, t + dt]$, then equation (10.4.3) implies

$$\begin{aligned} X_t &= X_{t-} + \log J_t \\ dX_t &= X_t - X_{t-} \\ dX_t &= \log J_t . \end{aligned} \quad (10.4.5)$$

Putting together equations (10.4.4) and (10.4.5) gives

$$dX_t = (\mu - \sigma^2/2) dt + \sigma dZ + \log J_t dq \quad (10.4.6)$$

or writing this in terms of S_t , we get

$$S_t = S_{t-} e^{(\mu - \sigma^2/2) dt + \sigma(Z(t+dt) - Z(t))} J_t \quad (10.4.7)$$

where we assume that only one possible jump can occur in $[t, t + dt]$, and that $J_t = 1$ if no jump occurs in $[t, t + dt]$.

A common assumption is that

$$\log J_t \sim N(\mu_J, \sigma_J^2) \quad (10.4.8)$$

i.e. that J_t is lognormal, with mean μ_J and variance σ_J^2 .

10.4.1 Compensated Drift

Recall equation (10.4.1)

$$\frac{dS_t}{S_{t-}} = \mu dt + \sigma dZ + (J_t - 1) dq . \quad (10.4.9)$$

Now

$$\begin{aligned} E\left[\frac{dS_t}{S_{t-}}\right] &= \mu dt + 0 + E[J_t - 1]E[dq] \\ &= \mu dt + E[J_t - 1]\lambda dt , \end{aligned} \quad (10.4.10)$$

where we assume that dq and J_t are independent. It is usual to redefine the drift term in terms of the compensated drift μ_c

$$\begin{aligned} \mu &= \mu_c - \lambda\kappa \\ \kappa &= E[J_t - 1] . \end{aligned} \quad (10.4.11)$$

Now, with definition (10.4.11), equation (10.4.9) becomes

$$\frac{dS_t}{S_{t-}} = (\mu_c - \lambda\kappa) dt + \sigma dZ + (J_t - 1) dq , \quad (10.4.12)$$

and

$$\begin{aligned} E\left[\frac{dS_t}{S_{t-}}\right] &= (\mu_c - \lambda\kappa) dt + 0 + \lambda\kappa dt \\ &= \mu_c dt . \end{aligned} \quad (10.4.13)$$

In this way

$$E \left[\frac{dS_t}{S_{t-}} \right] = \mu_c dt , \quad (10.4.14)$$

so that $\mu_c dt$ is the expected return in $[t, t + dt]$.

If $\log J_t \sim N(\mu_J, \sigma_J^2)$, then

$$\kappa = \exp(\sigma_J^2/2 + \mu_J) - 1 . \quad (10.4.15)$$

In terms of $X_t = \log S_t$, process (10.4.12) is

$$dX_t = (\mu_c - \sigma^2/2 - \lambda\kappa) dt + \sigma dZ + \log J_t dq \quad (10.4.16)$$

10.4.2 Contingent Claims Pricing

If we are interested in pricing a contingent claim, then we should compute the expected value in the risk neutral world. The requirement is then that

$$E \left[\frac{dS_t}{S_{t-}} \right] = \mu_c dt = r dt \quad (10.4.17)$$

where r is the risk free rate.

The stochastic process in the risk neutral world is then

$$\frac{dS_t}{S_{t-}} = (r - \lambda^Q \kappa^Q) dt + \sigma dZ + (J_t^Q - 1) dq , \quad (10.4.18)$$

where $\lambda^Q, \kappa^Q, J_t^Q$ are all risk adjusted quantities.

10.5 MATLAB Code: Jump Diffusion

We give MATLAB code for simulation jump diffusions, assuming process (10.4.16), with jump size given by (10.4.8) in (10.5).

We are assuming that

$$\lambda \Delta t \ll 1 \quad (10.5.1)$$

so that the probability of having more than one jump in $[t, t + dt]$ is negligible.

Note that there are more efficient ways of simulating jump diffusions, for special cases (i.e. volatility constant), but algorithm (10.5) is very simple and can be easily generalized.

Vectorized M file For Jump Diffusion

```

randn('state',100);
rand('state', 10);

T = 1.00;           % expiry time
sigma = 0.25;       % volatility
mu = .10;           % P measure drift
S_init = 100;       % initial value

% jump size: log normal distribution

lambda = .1 ;       % jump size arrival rate
jump_vol = .40; ;   % stdrd dev of jump size
jump_mean = -.9;    % mean of jump size

N_sim = 100000;     % number of simulations
N = 250;            % number of timesteps
delt = T/N;         % timestep

% compensated drift E[J-1]

kappa = exp(.5*jump_vol*jump_vol + jump_mean) - 1.;

% compensated drift for X = log(S)

drift = (mu - sigma*sigma/2.0 - lambda*kappa);

% X = log(S)

X_old(1:N_sim,1) = log(S_init);
X_new(1:N_sim,1) = zeros(N_sim, 1);

jump_chek = zeros(N_sim, 1);
jump_size = zeros(N_sim, 1);
jump_mask = zeros( N_sim, 1);

for i=1:N % timestep loop
    jump_chek(:,1) = rand(N_sim,1);
    jump_mask(:,1) = ( jump_chek(:,1) <= lambda*delt);

    jump_size(:,1) = jump_mean + jump_vol*randn(N_sim,1);
    jump_size = jump_size.*jump_mask;

    X_new(:,1) = X_old(:,1) + drift(:,1)*delt +sigma*sqrt(delt)*randn(N_sim,1) +...
                jump_size(:,1);
    X_old(:,1) = X_new(:,1);
end % timestep loop

S(:,1) = exp( X_new(:,1) );

n_bin = 200;
hist(S, n_bin);

stndrd_dev = std(S);
disp(sprintf('standard deviation: %.5g\n',stndrd_dev));
mean_S = mean(S);
disp(sprintf('mean: %.5g\n',mean_S));

```


Chapter 11

Regime Switching

Of course, volatility is not constant in the real world. It is possible to combine jumps in the asset price with jumps in volatility and stochastic volatility. This leads to a two factor pricing PIDE for the option price.

A simpler approach is to assume that the volatility jumps between a number of *regimes* or volatility states. Let the value of a contingent claim be given by $F(\sigma, S, t)$, where we have allowed the volatility σ to vary. Suppose

$$\begin{aligned} dS &= a dt + b dZ + (J_S - 1)S dq \\ d\sigma &= (J_\sigma - 1)\sigma dq, \end{aligned} \quad (11.0.1)$$

where dq is a Poisson process and dZ is the increment of a Weiner process. Here $a = a(\sigma, S, t)$, $b = b(\sigma, S, t)$. Note that the same dq drives the jump in S and the jump in σ . Following the same steps as in deriving equation (10.3.3) we obtain

$$\begin{aligned} dF &= \mu dt + \sigma^* dZ + \Delta F dq \\ \mu &= aF_S + \frac{b^2}{2}F_{SS} + F_t \\ \sigma^* &= bF_S \\ \Delta F &= [F(J_\sigma\sigma, J_S S, t) - F(\sigma, S, t)] . \end{aligned} \quad (11.0.2)$$

We follow the same steps as in the derivation of the jump diffusion PIDE in equations (10.3.5-10.3.12), i.e. we construct a hedge portfolio with three contracts F_1, F_2, F_3 , and we do not assume that we can trade in the underlying. Eliminating the random terms gives rise to the analogue of equation (10.3.9) and hence a solution exists only if one of the equations is a linear combination of the other equations, which results in

$$(\mu - rF) = \lambda_B \sigma^* - \lambda_J \Delta F \quad (11.0.3)$$

and substituting the definitions of σ^*, μ from equation (11.0.2) gives

$$F_t + \frac{b^2}{2}F_{SS} + (a - \lambda_B b)F_S - rF + \lambda_J[F(J_\sigma\sigma, J_S S, t) - F(\sigma, S, t)] = 0 . \quad (11.0.4)$$

In the event that, say, $F_3 = S$ is a traded asset, we note that in this case

$$\begin{aligned} \sigma_3^* &= b \\ \mu_3 &= a \\ \Delta F_3 &= (J_S - 1)S . \end{aligned} \quad (11.0.5)$$

Substituting equation (11.0.5) into equation (11.0.3) gives

$$a - \lambda_B b = rS - \lambda_J(J_S - 1)S . \quad (11.0.6)$$

Substituting equation (11.0.6) into equation (11.0.4) gives

$$F_t + \frac{b^2}{2} F_{SS} + [r - \lambda_J(J_S - 1)] S F_S - rF + \lambda_J[F(J_\sigma \sigma, J_S S, t) - F(\sigma, S, t)] = 0 . \quad (11.0.7)$$

Note that if $J_S = 1$ (no jump in S , but a regime switch) then the term $\lambda_J(J_S - 1)$ disappears in the drift term.

We can repeat the above arguments with jumps from a given regime with volatility σ to several possible regimes $J_\sigma^i \sigma$, $i = 1, \dots, p$. Each possible transition $\sigma \rightarrow J_\sigma^i \sigma$ is driven by a Poisson process dq^i . We assume that dq^i and dq^j are independent. In this case, we have

$$\begin{aligned} dS &= a dt + b dZ + \sum_{i=1}^{i=p} (J_S^i - 1) S dq^i \\ d\sigma &= \sum_{i=1}^{i=p} (J_\sigma^i - 1) \sigma dq^i , \end{aligned} \quad (11.0.8)$$

Following the by now familiar steps, we obtain

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_B b) F_S - rF + \sum_i \lambda_J^i [F(J_\sigma^i \sigma, J_S^i S, t) - F(\sigma, S, t)] = 0 . \quad (11.0.9)$$

Note that in general $\lambda_J^i = \lambda_J^i(\sigma, S)$, $J_\sigma^i = J_\sigma^i(\sigma, S)$, $J_S^i = J_S^i(\sigma, S)$, and $\lambda_B = \lambda_B(\sigma, S)$. Now, suppose we have only a finite number of possible regimes σ_k , $k = 1, \dots, p$. Let

$$\begin{aligned} b(\sigma_k, S, t) &= b_k \\ \lambda_B^k(\sigma_k, S) &= \lambda_B^k(S) \\ F(\sigma_k, S, t) &= F^k(S, t) \\ \lambda_J^i(\sigma_k, S, t) &= \lambda_J^{k \rightarrow i} \\ J_\sigma^i(\sigma_k, S, t) &= J_\sigma^{k \rightarrow i} \\ J_S^i(\sigma_k, S, t) &= J_S^{k \rightarrow i} . \end{aligned} \quad (11.0.10)$$

Rewriting equation (11.0.9) using equation (11.0.10) gives

$$F_t^k + \frac{b_k^2}{2} F_{SS}^k + (a_k - \lambda_B^k b_k) F_S^k - rF^k + \sum_i \lambda_J^{k \rightarrow i} [F^i(J_S^{k \rightarrow i} S, t) - F^k(S, t)] = 0 . \quad (11.0.11)$$

If we can hedge with the underlying, then the usual arguments give

$$a_k - \lambda_B^k b_k = rS - \sum_i \lambda_J^{k \rightarrow i} (J_S^{k \rightarrow i} - 1) S . \quad (11.0.12)$$

Substituting equation (11.0.12) into equation (11.0.11) gives

$$F_t^k + \frac{b_k^2}{2} F_{SS}^k + (r - \sum_i \lambda_J^{k \rightarrow i} (J_S^{k \rightarrow i} - 1)) S F_S^k - rF^k + \sum_i \lambda_J^{k \rightarrow i} [F^i(J_S^{k \rightarrow i} S, t) - F^k(S, t)] = 0 . \quad (11.0.13)$$

If we have only a small number of regimes, we are effectively solving a small number of coupled 1-d PDEs. In principle, the $J_S^{k \rightarrow i}, \sigma_k$ are P measure parameters, while the $\lambda_J^{k \rightarrow i}$ is a Q measure parameter. Often, we choose

$$\begin{aligned} b(\sigma, S, t) &= \sigma S \\ b_k &= \sigma_k S . \end{aligned} \quad (11.0.14)$$

We can also determine the $\sigma_k, J_S^{k \rightarrow i}, \lambda_J^{k \rightarrow i}$ by calibration to market prices.

11.1 Mean Variance Portfolio Optimization

An introduction to Computational Finance would not be complete without some discussion of Portfolio Optimization. Consider a risky asset which follows Geometric Brownian Motion with drift

$$\frac{dS}{S} = \mu dt + \sigma dZ, \quad (11.1.1)$$

where as usual $dZ = \phi\sqrt{dt}$ and $\phi \sim N(0,1)$. Suppose we consider a fixed finite interval Δt , then we can write equation (11.1.1) as

$$\begin{aligned} R &= \mu' + \sigma' \phi \\ R &= \frac{\Delta S}{S} \\ \mu' &= \mu \Delta t \\ \sigma' &= \sigma \sqrt{\Delta t}, \end{aligned} \quad (11.1.2)$$

where R is the actual return on the asset in $[t, t + \Delta t]$, μ' is the expected return on the asset in $[t, t + \Delta t]$, and σ' is the standard deviation of the return on the asset in $[t, t + \Delta t]$.

Now consider a portfolio of N risky assets. Let R^i be the return on asset i in $[t, t + \Delta t]$, so that

$$R^i = \mu'_i + \sigma'_i \phi_i \quad (11.1.3)$$

Suppose that the correlation between asset i and asset j is given by $\rho_{ij} = E[\phi_i \phi_j]$. Suppose we buy x_i of each asset at t , to form the portfolio P

$$P = \sum_{i=1}^{i=N} x_i S_i. \quad (11.1.4)$$

Then, over the interval $[t, t + \Delta t]$

$$\begin{aligned} P + \Delta P &= \sum_{i=1}^{i=N} x_i S_i (1 + R^i) \\ \Delta P &= \sum_{i=1}^{i=N} x_i S_i R^i \\ \frac{\Delta P}{P} &= \sum_{i=1}^{i=N} w_i R^i \\ w_i &= \frac{x_i S_i}{\sum_{j=1}^{j=N} x_j S_j} \end{aligned} \quad (11.1.5)$$

In other words, we divide up our total wealth $W = \sum_{i=1}^{i=N} x_i S_i$ into each asset with weight w_i . Note that $\sum_{i=1}^{i=N} w_i = 1$.

To summarize, given some initial wealth at t , we suppose that an investor allocates a fraction w_i of this wealth to each asset i . We assume that the total wealth is allocated to this risky portfolio P , so that

$$\begin{aligned} \sum_{i=1}^{i=N} w_i &= 1 \\ P &= \sum_{i=1}^{i=N} x_i S_i \\ R_p = \frac{\Delta P}{P} &= \sum_{i=1}^{i=N} w_i R^i. \end{aligned} \quad (11.1.6)$$

The expected return on this portfolio $\overline{R_p}$ in $[t, t + \Delta t]$ is

$$\overline{R_p} = \sum_{i=1}^{i=N} w_i \mu'_i, \quad (11.1.7)$$

while the variance of R_p in $[t, t + \Delta t]$ is

$$Var(R_p) = \sum_{i=1}^{i=N} \sum_{j=1}^{j=N} w_i w_j \sigma'_i \sigma'_j \rho_{ij}. \quad (11.1.8)$$

11.2 Special Cases

Suppose the assets all have zero correlation with one another, i.e. $\rho_{ij} \equiv 0, \forall i \neq j$ (of course $\rho_{ii} = 1$). Then equation (11.1.8) becomes

$$Var(R_p) = \sum_{i=1}^{i=N} (\sigma'_i)^2 (w_i)^2. \quad (11.2.1)$$

Now, suppose we equally weight all the assets in the portfolio, i.e. $w_i = 1/N, \forall i$. Let $\max_i \sigma'_i = \sigma'_{max}$, then

$$\begin{aligned} Var(R_p) &= \frac{1}{N^2} \sum_{i=1}^{i=N} (\sigma'_i)^2 \\ &\leq \frac{N(\sigma'_{max})^2}{N^2} \\ &= O\left(\frac{1}{N}\right), \end{aligned} \quad (11.2.2)$$

so that in this special case, if we diversify over a large number of assets, the standard deviation of the portfolio tends to zero as $N \rightarrow \infty$.

Consider another case: all assets are perfectly correlated, $\rho_{ij} = 1, \forall i, j$. In this case

$$\begin{aligned} Var(R_p) &= \sum_{i=1}^{i=N} \sum_{j=1}^{j=N} w_i w_j \sigma'_i \sigma'_j \\ &= \left[\sum_{j=1}^{j=N} w_j \sigma'_j \right]^2 \end{aligned} \quad (11.2.3)$$

so that if $sd(R) = \sqrt{Var(R)}$ is the standard deviation of R , then, in this case

$$sd(R_p) = \sum_{j=1}^{j=N} w_j \sigma'_j, \quad (11.2.4)$$

which means that in this case the standard deviation of the portfolio is simply the weighted average of the individual asset standard deviations.

In general, we can expect that $0 < |\rho_{ij}| < 1$, so that the standard deviation of a portfolio of assets will be smaller than the weighted average of the individual asset standard deviation, but larger than zero.

This means that diversification will be a good thing (as Martha Stewart would say) in terms of risk versus reward. In fact, a portfolio of as little as 10 – 20 stocks tends to reap most of the benefits of diversification.

11.3 The Portfolio Allocation Problem

Different investors will choose different portfolios depending on how much risk they wish to take. However, all investors like to achieve the highest possible expected return for a given amount of risk. We are assuming that risk and standard deviation of portfolio return are synonymous.

Let the covariance matrix C be defined as

$$[C]_{ij} = C_{ij} = \sigma'_i \sigma'_j \rho_{ij} \quad (11.3.1)$$

and define the vectors $\bar{\mu} = [\mu'_1, \mu'_2, \dots, \mu'_N]^t$, $\bar{w} = [w_1, w_2, \dots, w_N]^t$. In theory, the covariance matrix should be symmetric positive semi-definite. However, measurement errors may result in C having a negative eigenvalue, which should be fixed up somehow.

The expected return on the portfolio is then

$$\overline{R_p} = \bar{w}^t \bar{\mu} , \quad (11.3.2)$$

and the variance is

$$Var(R_p) = \bar{w}^t C \bar{w} . \quad (11.3.3)$$

We can think of portfolio allocation problem as the following. Let α represent the degree with which investors want to maximize return at the expense of assuming more risk. If $\alpha \rightarrow 0$, then investors want to avoid as much risk as possible. On the other hand, if $\alpha \rightarrow \infty$, then investors seek only to maximize expected return, and don't care about risk. The portfolio allocation problem is then (for given α) find $\bar{w}$ which satisfies

$$\min_{\bar{w}} \bar{w}^t C \bar{w} - \alpha \bar{w}^t \bar{\mu} \quad (11.3.4)$$

subject to the constraints

$$\sum_i w_i = 1 \quad (11.3.5)$$

$$L_i \leq w_i \leq U_i ; \quad i = 1, \dots, N . \quad (11.3.6)$$

Constraint (11.3.5) is simply equation (11.1.6), while constraints (11.3.6) may arise due to the nature of the portfolio. For example, most mutual funds can only hold long positions ($w_i \geq 0$), and they may also be prohibited from having a large position in any one asset (e.g. $w_i \leq .20$). Long-short hedge funds will not have these types of restrictions. For fixed α , equations (11.3.4-11.3.6) constitute a *quadratic programming* problem.

Let

$$\begin{aligned} sd(R_p) &= \text{standard deviation of } R_p \\ &= \sqrt{Var(R_p)} \end{aligned} \quad (11.3.7)$$

We can now trace out a curve on the $(sd(R_p), \overline{R_p})$ plane. We pick various values of α , and then solve the quadratic programming problem (11.3.4-11.3.6). Figure 11.3.1 shows a typical curve, which is also known as the *efficient frontier*. The data used for this example is

$$\begin{aligned} \bar{\mu} &= \begin{bmatrix} .15 \\ .20 \\ .08 \end{bmatrix} ; \quad C = \begin{bmatrix} .20 & .05 & -.01 \\ .05 & .30 & .015 \\ -.01 & .015 & .1 \end{bmatrix} \\ L &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} ; \quad U = \begin{bmatrix} \infty \\ \infty \\ \infty \end{bmatrix} \end{aligned} \quad (11.3.8)$$

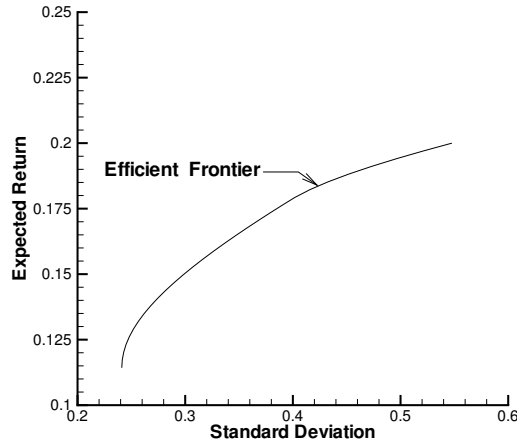


Figure 11.3.1: A typical efficient frontier. This curve shows, for each value of portfolio standard deviation $SD(R_p)$, the maximum possible expected portfolio return R_p . Data in equation (11.3.8).

We have restricted this portfolio to be long only. For a given value of the standard deviation of the portfolio return ($sd(R_p)$), then any point below the curve is not efficient, since there is another portfolio with the same risk (standard deviation) and higher expected return. Only points on the curve are efficient in this manner. In general, a linear combination of portfolios at two points along the efficient frontier will be feasible, i.e. satisfy the constraints. This feasible region will be convex along the efficient frontier. Another way of saying this is that a straight line joining any two points along the curve does not intersect the curve except at the given two points. Why is this the case? If this was not true, then the efficient frontier would not really be efficient. (see Portfolio Theory and Capital Markets, W. Sharpe, McGraw Hill, 1970, reprinted in 2000).

Figure 11.3.2 shows results if we allow the portfolio to hold up to .25 short positions in each asset. In other words, the data is the same as in (11.3.8) except that

$$L = \begin{bmatrix} -.25 \\ -.25 \\ -.25 \end{bmatrix}. \quad (11.3.9)$$

In general, long-short portfolios are more efficient than long-only portfolios. This is the advertised advantage of long-short hedge funds.

Since the feasible region is convex, we can actually proceed in a different manner when constructing the efficient frontier. First of all, we can determine the maximum possible expected return ($\alpha = \infty$ in equation (11.3.4)),

$$\begin{aligned} \min_{\bar{w}} & -\bar{w}^t \bar{\mu} \\ & \sum_i w_i = 1 \\ & L_i \leq w_i \leq U_i \quad ; \quad i = 1, \dots, N \end{aligned} \quad (11.3.10)$$

which is simply a linear programming problem. If the solution weight vector to this problem is $(\bar{w})_{max}$, then the maximum possible expected return is $(\bar{R}_p)^{max} = \bar{w}_{max}^t \bar{\mu}$.

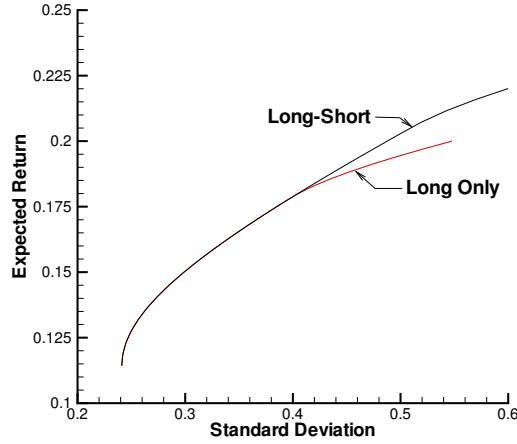


Figure 11.3.2: Efficient frontier, comparing results for long-only portfolio (11.3.8) and a long-short portfolio (same data except that lower bound constraint is replaced by equation (11.3.9)).

Then determine the portfolio with the smallest possible risk, ($\alpha = 0$ in equation (11.3.4))

$$\begin{aligned} \min_{\bar{w}} \bar{w}^t C \bar{w} \\ \sum_i w_i = 1 \\ L_i \leq w_i \leq U_i \quad ; \quad i = 1, \dots, N . \end{aligned} \quad (11.3.11)$$

If the solution weight vector to this quadratic program is given by $\bar{w}_{min}$, then the minimum possible portfolio return is $(\bar{R}_p)^{min} = \bar{w}_{min}^t \bar{\mu}$. We then divide up the range $[(\bar{R}_p)^{min}, (\bar{R}_p)^{max}]$ into a large number of discrete portfolio returns $(\bar{R}_p)_k; k = 1, \dots, N_{pts}$. Let $e = [1, 1, \dots, 1]^t$, and

$$A = \begin{bmatrix} \bar{\mu}^t \\ e^t \end{bmatrix} \quad ; \quad B^k = \begin{bmatrix} (\bar{R}_p)_k \\ 1 \end{bmatrix} \quad (11.3.12)$$

then, for given $(\bar{R}_p)_k$ we solve the quadratic program

$$\begin{aligned} \min_{\bar{w}} \bar{w}^t C \bar{w} \\ A \bar{w} = B^k \\ L_i \leq w_i \leq U_i \quad ; \quad i = 1, \dots, N , \end{aligned} \quad (11.3.13)$$

with solution vector $(\bar{w})_k$ and hence portfolio standard deviation $sd((\bar{R}_p)_k) = \sqrt{(\bar{w})_k^t C (\bar{w})_k}$. This gives us a set of pairs $(sd((\bar{R}_p)_k), (\bar{R}_p)_k), k = 1, \dots, N_{pts}$.

11.4 Adding a Risk-free asset

Up to now, we have assumed that each asset is risky, i.e. $\sigma'_i > 0, \forall i$. However, what happens if we add a risk free asset to our portfolio? This risk-free asset must earn the risk free rate $r' = r \Delta t$, and its standard deviation is zero. The data for this case is (the risk-free asset is added to the end of the weight vector, with

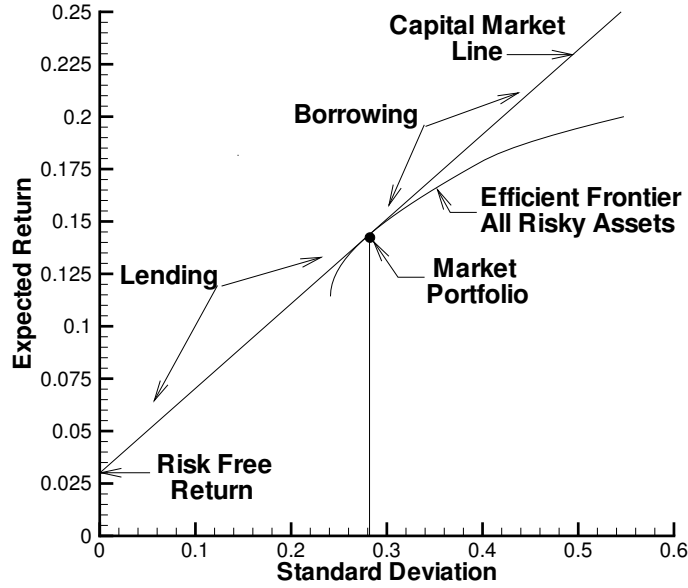


Figure 11.4.1: The efficient frontier from Figure 11.3.1 (all risky assets), and the efficient frontier with the same assets as in Figure 11.3.1, except that we include a risk free asset. In this case, the efficient frontier becomes a straight line, shown as the capital market line.

$r' = .03$).

$$\begin{aligned} \bar{\mu} &= \begin{bmatrix} .15 \\ .20 \\ .08 \\ .03 \end{bmatrix} ; \quad C = \begin{bmatrix} .20 & .05 & -.01 & 0.0 \\ .05 & .30 & .015 & 0.0 \\ -.01 & .015 & .1 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix} \\ L &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ -\infty \end{bmatrix} ; \quad U = \begin{bmatrix} \infty \\ \infty \\ \infty \\ \infty \end{bmatrix} \end{aligned} \quad (11.4.1)$$

where we have assumed that we can borrow any amount at the risk-free rate (a dubious assumption).

If we compute the efficient frontier with a portfolio of risky assets and include one risk-free asset, we get the result labeled *capital market line* in Figure 11.4.1. In other words, in this case the efficient frontier is a straight line. Note that this straight line is always above the efficient frontier for the portfolio consisting of all risky assets (as in Figure 11.3.1). In fact, given the efficient frontier from Figure 11.3.1, we can construct the efficient frontier for a portfolio of the same risky assets plus a risk free asset in the following way. First of all, we start at the point $(0, r')$ in the $(sd(R_p), \bar{R}_p)$ plane, corresponding to a portfolio which consists entirely of the risk free asset. We then draw a straight line passing through $(0, r')$, which touches the all-risky-asset efficient frontier at a single point (the straight line is tangent the all-risky-asset efficient frontier). Let the portfolio weights at this single point be denoted by $\bar{w}_M$. The portfolio corresponding to the weights $\bar{w}_M$ is termed the *market portfolio*. Let $(\bar{R}_p)_M = \bar{w}_M^t \bar{\mu}$ be the expected return on this market portfolio, with corresponding standard deviation $sd((R_p)_M)$. Let w_r be the fraction invested in the risk free asset. Then,

any point along the capital market line has

$$\begin{aligned}\overline{R_p} &= w_r r' + (1 - w_r) (\overline{R_p})_M \\ sd(R_p) &= (1 - w_r) sd((R_p)_M) .\end{aligned}\tag{11.4.2}$$

If $w_r \geq 0$, then we are lending at the risk-free rate. If $w_r < 0$, we are borrowing at the risk-free rate.

Consequently, given a portfolio of risky assets, and a risk-free asset, then all investors should divide their assets between the risk-free asset and the market portfolio. Any other choice for the portfolio is not efficient. Note that the actual fraction selected for investment in the market portfolio depends on the risk preferences of the investor.

The capital market line is so important, that the equation of this line is written as $\overline{R_p} = r' + \lambda_M sd((R_p))$, where λ_M is the market price of risk. In other words, all diversified investors, at any particular point in time, should have diversified portfolios which plot along the capital market line. All portfolios should have the same Sharp ratio

$$\lambda_M = \frac{\overline{R_p} - r'}{sd(R_p)} .\tag{11.4.3}$$

11.5 Criticism

Is mean-variance portfolio optimization the solution to all our problems? Not exactly. We have assumed that μ', σ' are independent of time. This is not likely. Even if these parameters are reasonably constant, they are difficult to estimate. In particular, μ' is hard to determine if the time series of returns is not very long. Remember that for short time series, the noise term (Brownian motion) will dominate. If we have a long time series, we can get a better estimate for μ' , but why do we think μ' for a particular firm will be constant for long periods? Probably, stock analysts should be estimating μ' from company balance sheets, sales data, etc. However, for the past few years, analysts have been too busy hyping stocks and going to lunch to do any real work. So, there will be lots of different estimates of μ', C , and hence many different *optimal* portfolios.

In fact, some recent studies have suggested that if investors simply use the $1/N$ rule, whereby initial wealth is allocated equally between N assets, that this does a pretty good job, assuming that there is uncertainty in the estimates of μ', C .

We have also assumed that risk is measured by standard deviation of portfolio return. Actually, if I am long an asset, I like it when the asset goes up, and I don't like it when the asset goes down. In other words, volatility which makes the price increase is good. This suggests that perhaps it may be more appropriate to minimize downside risk only (assuming a long position).

Perhaps one of the most useful ideas that come from mean-variance portfolio optimization is that diversified investors (at any point in time) expect that any optimal portfolio will produce a return

$$\begin{aligned}\overline{R_p} &= r' + \lambda_M \sigma'_p \\ \overline{R_p} &= \text{Expected portfolio return} \\ r' &= \text{risk-free return in period } \Delta t \\ \lambda_M &= \text{market price of risk} \\ \sigma'_p &= \text{Portfolio volatility ,}\end{aligned}\tag{11.5.1}$$

where different investors will choose portfolios with different σ'_p (volatility), depending on their risk preferences, but λ_M is the same for all investors. Of course, we also have

$$\overline{R_M} = r' + \lambda_M \sigma'_M .\tag{11.5.2}$$

Note: there is a whole field called *Behavioural Finance*, whose adherents don't think much of mean-variance portfolio optimization.

Another recent approach is to compute the optimal portfolio weights using many different perturbed input data sets. The input data (expected returns, and covariances) are determined by *resampling*, i.e. assuming that the observed values have some observational errors. In this way, we can get an some sort of optimal portfolio weights which have some effect of data errors incorporated in the result. This gives us an average efficient frontier, which, it is claimed, is less sensitive to data errors.

11.6 Individual Securities

Equation (11.5.2) refers to an efficient portfolio. What is the relationship between risk and reward for individual securities? Consider the following portfolio: divide all wealth between the market portfolio, with weight w_M and security i , with weight w_i . By definition

$$w_M + w_i = 1, \quad (11.6.1)$$

and we define

$$\begin{aligned} \overline{R^M} &= \text{expected return on the market portfolio} \\ \overline{R^i} &= \text{expected return on asset } i \\ \sigma'_M &= \text{s.d. of return on market portfolio} \\ \sigma'_i &= \text{s.d. of return on asset } i \\ C_{i,M} &= \sigma'_M \sigma'_i \rho_{i,M} \\ &= \text{Covariance between } i \text{ and } M \end{aligned} \quad (11.6.2)$$

Now, the expected return on this portfolio is

$$\begin{aligned} \overline{R_p} = E[R_p] &= w_i \overline{R^i} + w_M \overline{R^M} \\ &= w_i \overline{R^i} + (1 - w_i) \overline{R^M} \end{aligned} \quad (11.6.3)$$

and the variance is

$$\begin{aligned} \text{Var}(R_p) &= (\sigma'_p)^2 = w_i^2 (\sigma'_i)^2 + 2w_i w_M C_{i,M} + w_M^2 (\sigma'_M)^2 \\ &= w_i^2 (\sigma'_i)^2 + 2w_i (1 - w_i) C_{i,M} + (1 - w_i)^2 (\sigma'_M)^2 \end{aligned} \quad (11.6.4)$$

For a set of values $\{w_i\}$, equations (11.6.3-11.6.4) will plot a curve in expected return-standard deviation plane $(\overline{R_p}, \sigma'_p)$ (e.g. Figure 11.4.1). Let's determine the slope of this curve when $w_i \rightarrow 0$, i.e. when this curve intersects the capital market line at the market portfolio.

$$\begin{aligned} 2(\sigma'_p) \frac{\partial(\sigma'_p)}{\partial w_i} &= 2w_i (\sigma'_i)^2 + 2(1 - 2w_i) C_{i,M} + 2(w_i - 1) (\sigma'_M)^2 \\ \frac{\partial \overline{R_p}}{\partial w_i} &= \overline{R^i} - \overline{R^M}. \end{aligned} \quad (11.6.5)$$

Now,

$$\begin{aligned} \frac{\partial \overline{R_p}}{\partial(\sigma'_p)} &= \frac{\frac{\partial \overline{R_p}}{\partial w_i}}{\frac{\partial(\sigma'_p)}{\partial w_i}} \\ &= \frac{(\overline{R^i} - \overline{R^M})(\sigma'_p)}{w_i (\sigma'_i)^2 + (1 - 2w_i) C_{i,M} + (w_i - 1) (\sigma'_M)^2}. \end{aligned} \quad (11.6.6)$$

Now, let $w_i \rightarrow 0$ in equation (11.6.6), then we obtain

$$\frac{\partial \overline{R_p}}{\partial(\sigma'_p)} = \frac{(\overline{R^i} - \overline{R^M})(\sigma'_M)}{C_{i,M} - (\sigma'_M)^2} \quad (11.6.7)$$

But this curve should be tangent to the capital market line, equation (11.5.2) at the point where the capital market line touches the efficient frontier. If this curve is not tangent to the capital market line, then this implies that if we choose $w_i = \pm\epsilon$, then the curve would be above the capital market line, which should not be possible (the capital market line is the most efficient possible portfolio). This assumes that positions with $w_i < 0$ in asset i are possible.

Assuming that the slope of the $\overline{R}_p$ portfolio is tangent to the capital market line gives (from equations (11.5.2, 11.6.7))

$$\frac{\overline{R}^M - r'}{(\sigma'_M)} = \frac{(\overline{R}^i - \overline{R}^M)(\sigma'_M)}{C_{i,M} - (\sigma'_M)^2} \quad (11.6.8)$$

or

$$\begin{aligned} \overline{R}^i &= r' + \beta_i(\overline{R}^M - r') \\ \beta_i &= \frac{C_{i,M}}{(\sigma'_M)^2} . \end{aligned} \quad (11.6.9)$$

The coefficient β_i in equation (11.6.9) has a nice intuitive definition. Suppose we have a time series of returns

$$\begin{aligned} (R^i)_k &= \text{Return on asset } i, \text{ in period } k \\ (R^M)_k &= \text{Return on market portfolio in period } k . \end{aligned} \quad (11.6.10)$$

Typically, we assume that the market portfolio is a broad index, such as the TSX 300. Now, suppose we try to obtain a least squares fit to the above data, using the equation

$$R^i \simeq \alpha_i + b_i R^M . \quad (11.6.11)$$

Carrying out the usual least squares analysis (e.g. do a linear regression of R^i vs. R^M), we find that

$$b_i = \frac{C_{i,M}}{(\sigma'_M)^2} \quad (11.6.12)$$

so that we can write

$$R^i \simeq \alpha_i + \beta_i R^M . \quad (11.6.13)$$

This means that β_i is the slope of the best fit straight line to a $((R^i)_k, (R^M)_k)$ scatter plot. An example is shown in Figure 11.6.1. Now, from equation (11.6.9) we have that

$$\overline{R}^i = r' + \beta_i(\overline{R}^M - r') \quad (11.6.14)$$

which is consistent with equation (11.6.13) if

$$\begin{aligned} R^i &= \alpha_i + \beta_i R^M + \epsilon_i \\ E[\epsilon_i] &= 0 \\ \alpha_i &= r'(1 - \beta_i) \\ E[\epsilon_i, R^M] &= 0 , \end{aligned} \quad (11.6.15)$$

since

$$E[R^i] = \overline{R}^i = \alpha_i + \beta_i \overline{R}^M . \quad (11.6.16)$$

Equation (11.6.16) has the interpretation that the return on asset i can be decomposed into a drift component, a part which is correlated to the market portfolio (the broad index), and a random part uncorrelated with the index. Make the following assumptions

$$\begin{aligned} E[\epsilon_i \epsilon_j] &= 0 ; \quad i \neq j \\ &= e_i^2 ; \quad i = j \end{aligned} \quad (11.6.17)$$

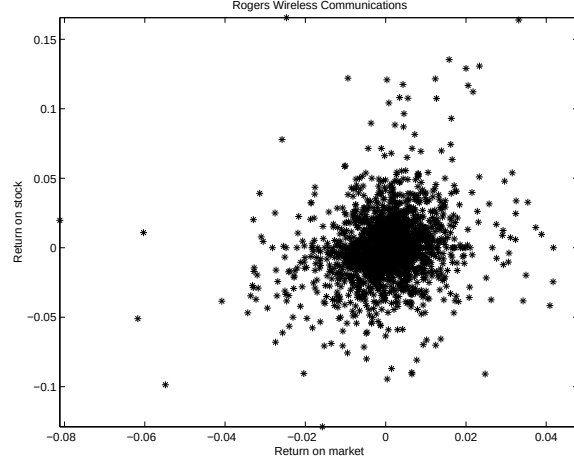


Figure 11.6.1: Return on Rogers Wireless Communications versus return on TSE 300. Each point represents pairs of daily returns. The vertical axis measures the daily return on the stock and the horizontal axis that of the TSE300.

e.g. that returns on each asset are correlated only through their correlation with the index. Consider once again a portfolio where the wealth is divided amongst N assets, each asset receiving a fraction w_i of the initial wealth. In this case, the return on the portfolio is

$$\begin{aligned} R_p &= \sum_{i=1}^{i=N} w_i R^i \\ \overline{R_p} &= \sum_{i=1}^{i=N} w_i \alpha_i + \overline{R^M} \sum_{i=1}^{i=N} w_i \beta_i \end{aligned} \quad (11.6.18)$$

and

$$\begin{aligned} s.d.(R_p) &= \sqrt{(\sigma'_M)^2 \sum_{i=1}^{i=N} \sum_{j=1}^{j=N} w_i w_j \beta_i \beta_j + \sum_{i=1}^{i=N} w_i^2 e_i^2} \\ &= \sqrt{(\sigma'_M)^2 \left(\sum_{i=1}^{i=N} w_i \beta_i \right)^2 + \sum_{i=1}^{i=N} w_i^2 e_i^2} . \end{aligned} \quad (11.6.19)$$

Now, if $w_i = O(1/N)$, then

$$\sum_{i=1}^{i=N} w_i^2 e_i^2 \quad (11.6.20)$$

is $O(1/N)$ as N becomes large, hence equation (11.6.19) becomes

$$s.d.(R_p) \simeq \sigma'_M \left| \sum_{i=1}^{i=N} w_i \beta_i \right| . \quad (11.6.21)$$

Note that if we write

$$\overline{R^i} = r' + \lambda_i \sigma'_i \quad (11.6.22)$$

then we also have that

$$\overline{R^i} = r' + \beta_i(\overline{R^M} - r') \quad (11.6.23)$$

so that the market price of risk of security i is

$$\lambda_i = \frac{\beta_i(\overline{R^M} - r')}{\sigma'_i} \quad (11.6.24)$$

which is useful in real options analysis.

Chapter 12

Derivative Contracts on non-traded Assets and Real Options

The hedging arguments used in previous sections use the underlying asset to construct a hedging portfolio. What if the underlying asset cannot be bought and sold, or is non-storable? If the underlying variable is an interest rate, we can't store this. Or if the underlying asset is bandwidth, we can't store this either. However, we can get around this using the following approach.

12.1 Derivative Contracts

Let the underlying variable follow

$$dS = a(S, t)dt + b(S, t)dZ, \quad (12.1.1)$$

and let $F = F(S, t)$, so that from Ito's Lemma

$$dF = \left[aF_S + \frac{b^2}{2}F_{SS} + F_t \right] dt + bF_S dZ, \quad (12.1.2)$$

or in shorter form

$$\begin{aligned} dF &= \mu dt + \sigma^* dZ \\ \mu &= aF_S + \frac{b^2}{2}F_{SS} + F_t \\ \sigma^* &= bF_S. \end{aligned} \quad (12.1.3)$$

Now, instead of hedging with the underlying asset, we will hedge one contract with another. Suppose we have two contracts F_1, F_2 (they could have different maturities for example). Then

$$\begin{aligned} dF_1 &= \mu_1 dt + \sigma_1^* dZ \\ dF_2 &= \mu_2 dt + \sigma_2^* dZ \\ \mu_i &= a(F_i)_S + \frac{b^2}{2}(F_i)_{SS} + (F_i)_t \\ \sigma_i^* &= b(F_i)_S \quad ; \quad i = 1, 2. \end{aligned} \quad (12.1.4)$$

Consider the portfolio Π

$$\Pi = n_1 F_1 + n_2 F_2 \quad (12.1.5)$$

so that

$$\begin{aligned}
d\Pi &= n_1 dF_1 + n_2 dF_2 \\
&= n_1(\mu_1 dt + \sigma_1^* dZ) + n_2(\mu_2 dt + \sigma_2^* dZ) \\
&= (n_1\mu_1 + n_2\mu_2) dt + (n_1\sigma_1^* + n_2\sigma_2^*) dZ .
\end{aligned} \tag{12.1.6}$$

Now, to eliminate risk, choose

$$(n_1\sigma_1^* + n_2\sigma_2^*) = 0 \tag{12.1.7}$$

which means that Π is riskless, hence

$$d\Pi = r\Pi dt , \tag{12.1.8}$$

so that, using equations (12.1.6-12.1.8), we obtain

$$(n_1\mu_1 + n_2\mu_2) = r(n_1F_1 + n_2F_2). \tag{12.1.9}$$

Putting together equations (12.1.7) and (12.1.9) gives

$$\begin{bmatrix} \sigma_1^* & \sigma_2^* \\ \mu_1 - rF_1 & \mu_2 - rF_2 \end{bmatrix} \begin{bmatrix} n_1 \\ n_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} . \tag{12.1.10}$$

Now, equation (12.1.10) only has a nonzero solution if the two rows of equation (12.1.10) are linearly dependent. In other words, there must be a $\lambda_S = \lambda_S(S, t)$ (independent of the type of contract) such that

$$\begin{aligned}
\mu_1 - rF_1 &= \lambda_S \sigma_1^* \\
\mu_2 - rF_2 &= \lambda_S \sigma_2^* .
\end{aligned} \tag{12.1.11}$$

Dropping the subscripts, we obtain

$$\frac{\mu - rF}{\sigma^*} = \lambda_S \tag{12.1.12}$$

Substituting μ, σ^* from equations (12.1.3) into equation (12.1.12) gives

$$F_t + \frac{b^2}{2} F_{SS} + (a - \lambda_S b) F_S - rF = 0 . \tag{12.1.13}$$

Equation (12.1.13) is the PDE satisfied by a derivative contract on any asset S . Note that it does not matter if we cannot trade S .

Suppose that $F_2 = S$ is a traded asset. Then we can hedge with S , and from equation (12.1.11) we have

$$\mu_2 - rS = \lambda_S \sigma_2^* \tag{12.1.14}$$

and from equations (12.1.1) and (12.1.3) we have

$$\begin{aligned}
\sigma_2^* &= b \\
\mu_2 &= a
\end{aligned} \tag{12.1.15}$$

and so, using equations (12.1.11) and (12.1.15), we have that

$$\lambda_S = \frac{a - rS}{b} \tag{12.1.16}$$

and equation (12.1.13) reduces to

$$F_t + \frac{b^2}{2} F_{SS} + rSF_S - rF = 0 . \tag{12.1.17}$$

Suppose

$$\begin{aligned}\mu &= F\mu' \\ \sigma^* &= F\sigma'\end{aligned}\tag{12.1.18}$$

so that we can write

$$dF = F\mu'dt + F\sigma'dZ\tag{12.1.19}$$

then using equation (12.1.18) in equation (12.1.12) gives

$$\mu' = r + \lambda_S\sigma'\tag{12.1.20}$$

which has the convenient interpretation that the expected return on holding (not hedging) the derivative contract F is the risk-free rate plus extra compensation due to the riskiness of holding F . The extra return is $\lambda_S\sigma'$, where λ_S is the market price of risk of S (which should be the same for all contracts depending on S) and σ' is the volatility of F . Note that the volatility and drift of F are not the volatility and drift of the underlying asset S .

If we believe that the Capital Asset Pricing Model holds, then a simple minded idea is to estimate

$$\lambda_S = \rho_{SM}\lambda_M\tag{12.1.21}$$

where λ_M is the price of risk of the market portfolio, and ρ_{SM} is the correlation of returns between S and the returns of the market portfolio.

Another idea is the following. Suppose we can find some companies whose main source of business is based on S . Let q_i be the price of this companies stock at $t = t_i$. The return of the stock over $t_i - t_{i-1}$ is

$$R_i = \frac{q_i - q_{i-1}}{q_{i-1}}.$$

Let R_i^M be the return of the market portfolio (i.e. a broad index) over the same period. We compute β as the best fit linear regression to

$$R_i = \alpha + \beta R_i^M$$

which means that

$$\beta = \frac{Cov(R, R^M)}{Var(R^M)}.\tag{12.1.22}$$

Now, from CAPM we have that

$$E(R) = r + \beta [E(R^M) - r]\tag{12.1.23}$$

where $E(\dots)$ is the expectation operator. We would like to determine the *unlevered* β , denoted by β^u , which is the β for an investment made using equity only. In other words, if the firm we used to compute the β above has significant debt, its riskiness with respect to S is amplified. The unlevered β can be computed by

$$\beta^u = \frac{E}{E + (1 - T_c)D}\beta\tag{12.1.24}$$

where

$$\begin{aligned}D &= \text{long term debt} \\ E &= \text{Total market capitalization} \\ T_c &= \text{Corporate Tax rate}.\end{aligned}\tag{12.1.25}$$

So, now the expected return from a pure equity investment based on S is

$$E(R^u) = r + \beta^u [E(R^M) - r] . \quad (12.1.26)$$

If we assume that F in equation (12.1.19) is the company stock, then

$$\begin{aligned} \mu' &= E(R^u) \\ &= r + \beta^u [E(R^M) - r] \end{aligned} \quad (12.1.27)$$

But equation (12.1.20) says that

$$\mu' = r + \lambda_S \sigma' . \quad (12.1.28)$$

Combining equations (12.1.27-12.1.28) gives

$$\lambda_S \sigma' = \beta^u [E(R^M) - r] . \quad (12.1.29)$$

Recall from equations (12.1.3) and (12.1.18) that

$$\begin{aligned} \sigma^* &= F \sigma' \\ \sigma^* &= b F_S , \end{aligned}$$

or

$$\sigma' = \frac{b F_S}{F} . \quad (12.1.30)$$

Combining equations (12.1.29-12.1.30) gives

$$\lambda_S = \frac{\beta^u [E(R^M) - r]}{\frac{b F_S}{F}} . \quad (12.1.31)$$

In principle, we can now compute λ_S , since

- The unleveraged β^u is computed as described above. This can be done using market data for a specific firm, whose main business is based on S , and the firms balance sheet.
- $b(S, t)/S$ is the volatility rate of S (equation (12.1.1)).
- $[E(R^M) - r]$ can be determined from historical data. For example, the expected return of the market index above the risk free rate is about 6% for the past 50 years of Canadian data.
- The risk free rate r is simply the current T-bill rate.
- F_S can be estimated by computing a linear regression of the stock price of a firm which invests in S , and S . Now, this may have to be unlevered, to reduce the effect of debt. If we are going to now value the real option for a specific firm, we will have to make some assumption about how the firm will finance a new investment. If it is going to use pure equity, then we are done. If it is a mixture of debt and equity, we should relever the value of F_S . At this point, we need to talk to a Finance Professor to get this right.

12.2 A Forward Contract

A forward contract is a special type of derivative contract. The holder of a forward contract agrees to buy or sell the underlying asset at some delivery price K in the future. K is determined so that the cost of entering into the forward contract is zero at its inception.

The payoff of a (long) forward contract expiring at $t = T$ is then

$$V(S, \tau = 0) = S(T) - K . \quad (12.2.1)$$

Note that there is no optionality in a forward contract.

The value of a forward contract is a contingent claim. and its value is given by equation (12.1.13)

$$V_t + \frac{b^2}{2} V_{SS} + (a - \lambda_S b) V_S - rV = 0 . \quad (12.2.2)$$

Now we can also use a simple no-arbitrage argument to express the value of a forward contract in terms of the original delivery price K , (which is set at the inception of the contract) and the current forward price $f(S, \tau)$. Suppose we are long a forward contract with delivery price K . At some time $t > 0$, ($\tau < T$), the forward price is no longer K . Suppose the forward price is $f(S, \tau)$, then the payoff of a long forward contract, entered into at (τ) is

$$\text{Payoff} = S(T) - f(S(\tau), \tau) .$$

Suppose we are long the forward contract struck at $t = 0$ with delivery price K . At some time $t > 0$, we hedge this contract by going short a forward with the current delivery price $f(S, \tau)$ (which costs us nothing to enter into). The payoff of this portfolio is

$$S - K - (S - f) = f - K \quad (12.2.3)$$

Since f, K are known with certainty at (S, τ) , then the value of this portfolio today is

$$(f - K)e^{-r\tau} . \quad (12.2.4)$$

But if we hold a forward contract today, we can always construct the above hedge at no cost. Therefore,

$$V(S, \tau) = (f - K)e^{-r\tau} . \quad (12.2.5)$$

Substituting equation (12.2.5) into equation (12.2.2), and noting that K is a constant, gives us the following PDE for the forward price (the delivery price which makes the forward contract worth zero at inception)

$$f_\tau = \frac{b^2}{2} f_{SS} + (a - \lambda_S b) f_S \quad (12.2.6)$$

with terminal condition

$$f(S, \tau = 0) = S \quad (12.2.7)$$

which can be interpreted as the fact that the forward price must equal the spot price at $t = T$.

Suppose we can estimate a, b in equation (12.2.6), and there are a set of forward prices available. We can then estimate λ_S by solving equation (12.2.6) and adjusting λ_S until we obtain a good fit for the observed forward prices.

12.2.1 Convenience Yield

We can also write equation (12.2.6) as

$$f_t + \frac{b^2}{2} f_{SS} + (r - \delta) S f_S = 0 \quad (12.2.8)$$

where δ is defined as

$$\delta = r - \frac{a - \lambda_S b}{S} . \quad (12.2.9)$$

In this case, we can interpret δ as the *convenience yield* for holding the asset. For example, there is a convenience to holding supplies of natural gas in reserve.

12.2.2 Volatility of Forward Prices

From equation (12.2.6) we have that the forward price for a contract expiring at time T , at current time t , spot price $S(t)$ is given by

$$f(S, t) = E^Q[S(T)] \quad (12.2.10)$$

where S follows the risk neutral process

$$dS = (a - \lambda_S b) dt + b dZ . \quad (12.2.11)$$

In other words, the *forward price is the risk neutral expected spot price at expiry*.

Now, using Ito's Lemma and assuming the risk neutral spot process (12.2.11) gives

$$df = \left(f_t + \frac{b^2}{2} f_{SS} + (a - \lambda_S b) f_S \right) dt + f_{Sb} dZ . \quad (12.2.12)$$

But since f satisfies equation (12.2.6), equation (12.2.12) becomes

$$\begin{aligned} df &= f_{Sb} dZ \\ &= \hat{\sigma} f dZ , \end{aligned} \quad (12.2.13)$$

where the *effective volatility* of the forward price is

$$\hat{\sigma} = \frac{f_{Sb}}{f} . \quad (12.2.14)$$

Note that from equation (12.2.13), the forward price has zero drift.

Chapter 13

Further Reading

13.1 General Interest

- Peter Bernstein, *Capital Ideas: the improbable origins of modern Wall street*, The Free Press, New York, 1992.
- Peter Bernstein, *Against the Gods: the remarkable story of risk*, John Wiley, New York, 1998, ISBN 0-471-29563-9.
- Benjamin Graham, *The intelligent investor*, various editions. Warren Buffet's bible. There is a Collins edition, of the 1973 version, with commentary by Jason Zweig.
- Burton Malkiel, *A random walk down Wall Street*, W.W. Norton, New York, various editions.
- N. Taleb, *Fooled by Randomness*, Texere, 2001, ISBN 1-58799-071-7.
- S. Das, *Traders, Guns and Money*, Prentice Hall, 2006.
- N. Taleb, *The black swan*, Random House, 2007.

13.2 More Background

- A. Dixit and R. Pindyck, *Investment under uncertainty*, Princeton University Press, 1994.
- John Hull, *Options, futures and other derivatives*, Prentice-Hall, 1997, ISBN 0-13-186479-3.
- S. Ross, R. Westerfield, J. Jaffe, *Corporate Finance*, McGraw-Hill Irwin, 2002, ISBN 0-07-283137-5.
- W. Sharpe, *Portfolio Theory and Capital Markets*, Wiley, 1970, reprinted in 2000, ISBN 0-07-135320-8. (Still a classic).
- Lenos Trigeorgis, *Real Options: Managerial Flexibility and Strategy for Resource Allocation*, MIT Press, 1996, ISBN 0-262-20102-X.

13.3 More Technical

- P. Brandimarte, *Numerical Methods in Finance: A MATLAB Introduction*, Wiley, 2002, ISBN 0-471-39686-9.
- Boyle, Broadie, Glasserman, *Monte Carlo methods for security pricing*, J. Econ. Dyn. Con., 21:1267-1321 (1997)

- P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer (2004) ISBN 0-387-00451-3.
- D. Higham, *An Introduction to Financial Option Valuation*, Cambridge (2004) ISBN 0-521-83884-3.
- P. Jackel, *Monte Carlo Methods in Finance*, Wiley, 2002, ISBN 0-471-49741-X.
- Y.K. Kwok, *Mathematical Models of Finance*, Springer Singapore, 1998, ISBN 981-3083-565.
- S. Neftci, *An Introduction to the Mathematics of Financial Derivatives*, Academic Press (2000) ISBN 0-12-515392-9.
- R. Seydel, *Tools for Computational Finance*, Springer, 2002, ISBN 3-540-43609-X.
- D. Tavella and C. Randall, *Pricing Financial Instruments: the Finite Difference Method*, Wiley, 2000, ISBN 0-471-19760-2.
- D. Tavella, *Quantitative Methods in Derivatives Pricing: An Introduction to Computational Finance*, Wiley (2002) ISBN 0-471-39447-5.
- N. Taleb, *Dynamic Hedging*, Wiley, 1997, ISBN 0-471-15280-3.
- P. Wilmott, S. Howison, J. Dewynne, *The mathematics of financial derivatives: A student introduction*, Cambridge, 1997, ISBN 0-521-49789-2.
- Paul Wilmott, *Paul Wilmott on Quantitative Finance*, Wiley, 2000, ISBN 0-471-87438-8.